



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática – Ingeniería del Software

GEO-Audit: diseño e implementación de un proceso de migración web orientado a SEO técnico y AISO (optimización de búsquedas para motores de IA generativa), aplicado al caso Programamos.es

**Realizado por
Ángel Manuel Ferrer Álvarez**

**Dirigido por
Jesús Moreno León y Miguel Camacho Ruiz**

**Departamento
Departamento de Lenguajes y Sistemas Informáticos**

Sevilla, Mayo de 2026

Resumen

Los motores de búsqueda generativos, como ChatGPT, Gemini, Perplexity o Claude, están transformando la forma en que los usuarios acceden a la información en internet. A diferencia de los buscadores tradicionales, que devuelven una lista de enlaces, estos motores generan respuestas completas citando selectivamente las fuentes que consideran más relevantes. Para los sitios web, esto plantea un desafío fundamental: la citación se convierte en el principal punto de contacto con el usuario. Una marca puede aparecer también desde el conocimiento que el modelo adquirió en su entrenamiento, sin cita alguna; por ello este trabajo mide tanto las citas como las menciones de marca.

Este Trabajo Fin de Grado presenta GEO-Audit, un sistema de auditoría de visibilidad que mide y diagnostica la presencia de un sitio web en respuestas generadas por inteligencia artificial, y genera recomendaciones priorizadas para mejorarla. El sistema se valida mediante un caso de estudio real: Programamos.es, una asociación educativa sin ánimo de lucro que enseña programación a jóvenes en España.

Las contribuciones principales del trabajo son:

1. Un marco de 8 métricas de visibilidad GEO respaldadas por literatura académica: Visibilidad, Share of Model, Ranking, PAWC, Citation Rate, Brand Mentions, Sentiment y Engine Coverage.
2. Un pipeline experimental basado en RAG que, manteniendo congelados competidores y modelo juez, permite estimar la *dirección* del cambio en visibilidad al modificar el contenido del sitio web. Los valores absolutos tienden a estar inflados porque el simulador no incorpora señales externas presentes en motores reales, como autoridad de dominio, popularidad, presencia previa en corpus de entrenamiento o mecanismos propietarios de recuperación.
3. Un sistema de descubrimiento automático de competidores utilizando Gemini 2.5 Flash con Google Search grounding.
4. Un protocolo experimental con 100 consultas organizadas en un sistema de rotación (20 core + 4 bloques de 20).
5. Una evaluación en producción con mediciones en motores generativos reales (Gemini, ChatGPT y Claude), que permite medir la visibilidad efectiva del sitio en los motores que utilizan los usuarios y complementar los resultados del simulador RAG controlado.
6. Un agente de optimización (GEOOptimizer) que, a partir de los resultados de las métricas y el análisis de contenido de competidores, genera recomendaciones GEO priorizadas y ancladas a evidencia concreta para orientar futuras mejoras de contenido.

Palabras clave: Generative Engine Optimization, GEO, visibilidad en IA, RAG, métricas de citación, motores generativos, SEO, LLM, FAISS, optimización de contenido web.

Abstract

Generative search engines, such as ChatGPT, Gemini, Perplexity or Claude, are transforming how users access information on the internet. Unlike traditional search engines that return ranked lists of links, generative engines produce complete responses that selectively cite sources deemed most relevant. For websites, this poses a fundamental challenge: citation becomes the main point of contact with the user. A brand may also surface from knowledge the model acquired during training, without any citation; for this reason, this work measures both citations and brand mentions.

This undergraduate thesis presents GEO-Audit, a visibility auditing system that measures and diagnoses a website's presence in AI-generated responses, and produces prioritized recommendations to improve it. The system is validated through a real-world case study: Programamos.es, a non-profit educational association that teaches programming to young people in Spain.

The main contributions of this work are:

1. A framework of 8 GEO visibility metrics grounded in academic literature: Visibility, Share of Model, Ranking, PAWC, Citation Rate, Brand Mentions, Sentiment, and Engine Coverage.
2. An experimental pipeline based on Retrieval-Augmented Generation (RAG) that, by keeping competitors and the judge model frozen, estimates the *direction* of visibility change when the target website's content is modified. Absolute values tend to be inflated because the simulator does not model external signals used by real engines, such as domain authority, popularity, prior presence in training corpora, or proprietary retrieval mechanisms.
3. An automated competitor discovery system using Gemini 2.5 Flash with Google Search grounding.
4. An experimental protocol with 100 queries organized in a rotation system (20 core + 4 blocks of 20).
5. A production evaluation with live measurements across real generative engines (Gemini, ChatGPT, and Claude), which measures the website's effective visibility in the engines used by real users and complements the results of the controlled RAG simulator.
6. An optimization agent (GEOOptimizer) that, based on the metric results and the analysis of competitor content, generates prioritized GEO recommendations grounded in concrete evidence to guide future content improvements.

Keywords: Generative Engine Optimization, GEO, AI visibility, RAG, citation metrics, generative engines, SEO, LLM, FAISS, web content optimization.

Índice general

Resumen	i
Abstract	ii
Índice general	iii
Índice de tablas	viii
Índice de figuras	x
Índice de código	xi
1 Introducción	1
1.1 Contexto y motivación	1
1.2 Caso de estudio: Programamos.es	2
1.3 Objetivos	2
1.3.1 Objetivo general	2
1.3.2 Objetivos específicos	2
1.4 Alcance y limitaciones	3
1.4.1 Alcance del trabajo	3
1.4.2 Limitaciones	4
1.5 Estructura del documento	4
2 Estado del arte	6
2.1 De buscadores tradicionales a motores generativos	6
2.1.1 Del paradigma clásico al motor generativo	6
2.1.2 Arquitectura de los principales motores generativos	6
2.2 Generative Engine Optimization	7
2.2.1 Definición y taxonomía	7
2.2.2 Métricas de visibilidad	7
2.2.3 Estrategias de optimización	9
2.3 Sistemas RAG	10
2.3.1 Arquitectura y componentes	10
2.3.2 Chunking y configuración de referencia	10
2.3.3 Estructura HTML y citabilidad	11
2.4 Trabajos relacionados	11
2.4.1 Benchmarks y marcos de evaluación GEO	11
2.4.2 Limitaciones del estado del arte	11

3	Planificación y metodología	13
3.1	Metodología de desarrollo	13
3.1.1	Enfoque iterativo adaptado	13
3.1.2	Patrones de diseño	13
3.1.3	Dimensión investigadora de la metodología	14
3.2	Alcance del proyecto	14
3.2.1	Catálogo de requisitos de gestión	14
3.2.2	Línea base del alcance	17
3.3	Planificación temporal	19
3.3.1	Descripción de las fases	19
3.3.2	Cronograma y esfuerzo	21
3.4	Costes y recursos	24
3.4.1	Infraestructura de cómputo	24
3.4.2	Costes de APIs	25
3.4.3	Software y dependencias	25
3.5	Gestión del repositorio	26
3.5.1	Estrategia de ramificación	26
3.5.2	Integración continua	26
3.5.3	Versionado de configuración y prompts	26
3.5.4	Organización de datos	27
3.6	Gestión de riesgos	27
3.6.1	Riesgo crítico: inmutabilidad del modelo de embeddings	27
3.6.2	Plan de contingencia general	27
3.7	Seguimiento y cierre	28
3.7.1	Seguimiento temporal por fase	28
3.7.2	Comentario de desviaciones	29
3.7.3	Cierre del proyecto	29
4	Diseño del sistema	31
4.1	Visión general de la arquitectura	31
4.1.1	Casos de uso del sistema	31
4.1.2	Modo experimental	32
4.1.3	Modo live	32
4.1.4	Comparativa de modos	33
4.1.5	Pipeline de métricas SEO	33
4.1.6	Organización del código fuente	34
4.1.7	Dashboard de visualización	34
4.1.8	Vista de componentes y despliegue	35
4.2	Diseño experimental	38
4.2.1	Variables del experimento	38
4.2.2	Separación Discovery vs. Runs experimentales	39
4.2.3	Sistema de queries con rotación	40
4.3	Marco de métricas GEO	41
4.3.1	Definición formal de las métricas	41
4.3.2	Jerarquía de métricas	44
4.4	Decisiones arquitectónicas	45
4.4.1	ADR-003/ADR-016/ADR-017: Modelo de embeddings, evolución a Gemini API	45
4.4.2	ADR-004: Competidores descubiertos desde LLMs reales	45
4.4.3	ADR-005: FAISS local para el vectorstore	46
4.4.4	ADR-006: Separación discovery (una vez) vs. runs (N veces)	46
4.4.5	ADR-007: Procesamiento HTML-aware	46

4.4.6	ADR-010: Discovery con Gemini + Google Search grounding	46
4.4.7	ADR-011: Chunking 256/64 alineado con SAGEO Arena	47
4.4.8	ADR-012: RAG Judge como agente con herramienta de búsqueda FAISS	47
4.4.9	ADR-013: 100 queries con sistema de rotación	47
4.5	Agente de optimización: GEOOptimizer	48
4.5.1	Motivación y posición en el sistema	48
4.5.2	Entradas del agente	48
4.5.3	Palancas de optimización	48
4.5.4	Formato de salida	48
5	Implementación	50
5.1	Pipeline de procesamiento de contenido web	50
5.1.1	StructuredWebLoader	51
5.1.2	SiteCrawler	52
5.1.3	TokenAwareChunker	53
5.1.4	Embeddings via API de Google	53
5.2	Descubrimiento de competidores	54
5.2.1	CompetitorFinder con Google Search grounding	54
5.2.2	Agregación y selección de competidores	55
5.3	RAG Judge	56
5.3.1	Herramienta de búsqueda FAISS	56
5.3.2	Agente RAG con Gemini	57
5.3.3	Validación y parseo de salida JSON	57
5.4	Extracción de métricas GEO	58
5.4.1	Métricas calculadas	58
5.4.2	Cálculo de PAWC	59
5.4.3	Coincidencia de URLs y Citation Rate	59
5.4.4	Detección de menciones de marca	60
5.5	Evaluación en vivo sobre motores reales	60
5.5.1	Consulta a tres motores con búsqueda web	61
5.5.2	Métricas y operativa del modo live	61
5.6	Agente de optimización GEOOptimizer	62
5.6.1	Recopilación de evidencia con Serper	62
5.6.2	Generación de recomendaciones con Claude	62
5.6.3	Validación y crítica de las recomendaciones	63
5.7	Recolección automatizada de métricas SEO	63
5.8	Configuración y prompts	64
5.8.1	Sistema de configuración centralizado	64
5.8.2	Cargador de configuración	64
5.8.3	Registro de prompts versionado	65
5.9	Scripts de automatización	65
5.9.1	Script de descubrimiento	65
5.10	Dashboard web	65
5.11	Métricas del código	68
5.12	Matriz de trazabilidad	69
6	Pruebas y calidad	70
6.1	Estrategia de pruebas	70
6.2	Pruebas automatizadas del backend	71
6.3	Pruebas del frontend	72
6.4	Integración continua	73

6.5	Análisis de calidad del código	73
6.5.1	Métricas de complejidad y mantenibilidad	73
6.5.2	Análisis continuo con SonarCloud	73
6.6	Validación del núcleo y del experimento	74
6.6.1	Casos unitarios del CitationExtractor	74
6.6.2	Reproducibilidad	75
6.6.3	Validación experimental frente a evaluación en vivo	75
7	Resultados y discusión	76
7.1	Resultados del modo experimental	76
7.1.1	Coste de ejecución	76
7.1.2	Métricas de visibilidad	77
7.2	Resultados del modo live	77
7.2.1	Desglose por categoría de consulta	78
7.2.2	Tono de las menciones	79
7.3	Resultados SEO	79
7.4	Discusión: el simulador frente a la realidad	80
7.5	Evaluación del agente optimizador	80
8	Conclusiones y trabajo futuro	82
8.1	Conclusiones	82
8.2	Cumplimiento de objetivos	83
8.3	Contribuciones	83
8.4	Limitaciones	84
8.5	Lecciones aprendidas	84
8.6	Trabajo futuro	85
A	Registro de decisiones arquitectónicas (ADR)	86
B	Catálogo de consultas experimentales	87
C	Prompts del sistema	91
C.1	RAG Judge en modo agente (rag_judge_agent v1.0.0)	91
C.2	Analizador de sentimiento (sentiment_analyzer v1.0.0)	92
C.3	GEOOptimizer (geo_optimizer v3.2.0)	92
D	Manual de despliegue	94
D.1	Requisitos	94
D.2	Descubrimiento de competidores (única vez)	94
D.3	Ejecución de un <i>run</i> experimental	95
D.4	Evaluación en vivo	95
D.5	Recolección diaria de métricas SEO	95
D.6	Dashboard web	95
E	Manual de usuario del dashboard	97
E.1	Acceso y navegación	97
E.2	Pantallas	97
E.2.1	Visión general	97
E.2.2	Experimental	97
E.2.3	Detalle de run	97
E.2.4	Comparar	98
E.2.5	Live	98
E.2.6	SEO	98

E.2.7 Optimizador	98
E.2.8 Jobs	98
E.3 Flujos de uso habituales	98
F Informe del agente optimizador	100
F.1 Contexto de la ejecución	100
F.2 Estructura de cada recomendación	100
F.3 Recomendaciones generadas	100
F.4 Prompt de optimización (L2)	101
G Listados de código	102
G.1 Pipeline de procesamiento de contenido web	102
G.2 Descubrimiento de competidores	105
G.3 RAG Judge	105
G.4 Extracción de métricas GEO	107
G.5 Evaluación en vivo sobre motores reales	107
G.6 Agente de optimización GEOOptimizer	108
G.7 Recolección automatizada de métricas SEO	109
G.8 Configuración y prompts	109
G.9 Scripts de automatización	112
Referencias	114

Índice de tablas

2.1	Comparativa de métricas de visibilidad GEO en la literatura.	8
2.2	Impacto cuantitativo de las estrategias de optimización GEO.	10
3.1	Diccionario de la EDT (paquetes hoja).	19
3.2	Fechas de inicio y fin de las fases del proyecto.	21
3.3	Diagrama de Gantt del proyecto (15 semanas, feb–may 2026).	22
3.4	Backlog detallado de actividades por fase.	23
3.5	Estimación de esfuerzo por fase.	24
3.6	Desglose de costes de APIs.	25
3.7	Principales dependencias del proyecto.	25
3.8	Matriz de riesgos del proyecto.	28
3.9	Seguimiento del esfuerzo: estimado vs. real por fase.	29
4.1	Comparativa entre el modo experimental y el modo live.	33
4.2	Organización del código fuente en módulos.	34
4.3	Variables del diseño experimental de GEO-Audit.	38
4.4	Composición del banco de consultas experimentales.	40
4.5	Palancas de optimización GEO del agente GEOOptimizer.	49
5.1	Pantallas del dashboard: función, datos consumidos y objetivo cubierto. . .	68
5.2	Tamaño del código fuente por componente (medido con <code>cloc</code>).	68
5.3	Matriz de trazabilidad: objetivos → diseño → implementación → pruebas → resultados.	69
6.1	Niveles de la estrategia de pruebas.	70
6.2	Distribución de las pruebas por tipo.	71
6.3	Módulos de pruebas del backend.	71
6.4	Casos representativos de las pruebas del backend.	72
6.5	Cobertura de pruebas del backend por módulo (medida con <code>coverage.py</code>). .	72
6.6	Resultados del análisis en SonarCloud tras la iteración de corrección (pro- yecto <code>angelmanuelferrer_TFG</code>).	74
6.7	Casos de coincidencia de URLs con el dominio objetivo.	75
6.8	Controles de reproducibilidad del modo experimental.	75
7.1	Agregados del modo experimental (run <code>run_20260528_140843</code> , bloque R1). .	77
7.2	Desglose por categoría del run experimental (bloque R1).	77
7.3	Promedios de la evaluación en vivo sobre cinco semanas (W14–W18). Engine Coverage medio: 33,2 %.	78
7.4	Tasa de visibilidad por categoría de consulta y motor (datos agrupados de W14–W18).	79

7.5	Métricas SEO de PageSpeed Insights: media [mín-máx] de las 89 mediciones (feb-may 2026).	79
7.6	Recomendaciones generadas por el GEOOptimizer para las cuatro consultas perdidas analizadas.	81
8.1	Grado de cumplimiento de los objetivos específicos.	83
B.1	Distribución del catálogo de 100 consultas por categoría y bloque de rotación.	87
B.2	Catálogo completo de las 100 consultas (<code>queries.json</code> v2.0.0).	87

Índice de figuras

3.1	Diagrama de componentes del pipeline GEO-Audit.	13
3.2	Estructura de Descomposición del Trabajo (EDT) del proyecto.	18
4.1	Diagrama de casos de uso de GEO-Audit.	31
4.2	Diagrama de componentes UML de GEO-Audit.	36
4.3	Diagrama de despliegue UML de GEO-Audit.	37
4.4	Pipeline de Discovery (ejecución única): identifica los competidores y construye el vectorstore FAISS que permanece congelado durante todo el estudio.	39
4.5	Pipeline de un run experimental (ejecución N veces): re-procesa únicamente el contenido del sitio objetivo y reutiliza el vectorstore de competidores congelado.	40
4.6	Jerarquía de las métricas GEO. Las flechas indican dependencia: si $V(q) = 0$, las métricas hijas carecen de valor informativo para esa consulta. Las cajas en línea discontinua son agregaciones que se calculan sobre $V(q)$ a distintos niveles (categoría, motor o eficiencia de retrieval).	44
5.1	Vista de visión general del dashboard: instantánea combinada de los modos experimental, live y SEO.	66
5.2	Detalle de una ejecución experimental: métricas GEO por consulta y agregados por categoría.	67
5.3	Panel del optimizador: consultas priorizadas y recomendaciones GEO generadas por el agente.	67
7.1	Evolución de la tasa de visibilidad de programamos.es por motor (W14–W18).	78
7.2	Puntuaciones medias de PageSpeed Insights por dispositivo (media de 89 mediciones).	80

Índice de código

4.1	Selección de queries por bloque rotativo.	41
5.1	Listas de etiquetas y selectores de ruido en StructuredWebLoader.	51
5.2	Extracción del contenido principal de la página HTML.	51
5.3	Separadores jerárquicos del TokenAwareChunker.	53
5.4	Método <code>_query_gemini</code> de CompetitorFinder con Google Search grounding.	54
5.5	Resolución de URLs proxy de Google Search grounding.	55
5.6	Agregación de URLs competidoras por dominio con puntuación ponderada.	55
5.7	Método <code>_init_llm</code> del RAGJudge.	57
5.8	Validación de esquema JSON de la salida del RAGJudge.	57
5.9	Cálculo de PAWC en CitationExtractor.	59
5.10	Normalización y coincidencia de URLs en CitationExtractor.	59
5.11	Cálculo del Citation Rate en CitationExtractor.	60
5.12	Adaptación de métricas y cálculo de Engine Coverage por consulta.	61
5.13	Llamada a Claude con tool use para salida estructurada.	62
5.14	Función de recolección de métricas PageSpeed Insights.	63
5.15	Formato v2 del banco de consultas queries.json.	64
C.1	Prompt de sistema de <code>rag_judge_agent</code>	91
C.2	Prompt de sistema de <code>sentiment_analyzer</code>	92
G.1	Método de descarga con reintentos exponenciales de StructuredWebLoader.	102
G.2	Método principal del crawler para rastrear sitios web.	102
G.3	Clase TokenAwareChunker con fragmentación basada en tokens.	103
G.4	Clase GoogleGenAIEmbeddings para generación de vectores vía API de Google.	104
G.5	Extracción de URLs de búsqueda y citación desde grounding_metadata.	105
G.6	Herramienta de búsqueda FAISS encapsulada como tool de LangChain.	105
G.7	Método <code>generate_answer_with_agent</code> del agente RAGJudge.	106
G.8	Extracción robusta de JSON de la salida del agente mediante tres estrategias.	106
G.9	Detección de menciones de marca en la respuesta y citas.	107
G.10	Adaptador de Claude Haiku con búsqueda web en LiveEvaluator.	108
G.11	Búsqueda de fragmentos restringida a los competidores congelados.	108
G.12	Workflow de GitHub Actions para la recolección diaria de métricas SEO.	109
G.13	Fichero de configuración del experimento <code>experiment_config.json</code>	109
G.14	Función <code>get_queries_for_run</code> con soporte de rotación.	110
G.15	Registro centralizado de prompts con versionado.	111
G.16	Orquestación del pipeline de descubrimiento en <code>run_discovery.py</code>	112
G.17	Orquestación del pipeline experimental en <code>run_experimental.py</code>	112

Introducción

1.1— Contexto y motivación

En 2022, el lanzamiento de ChatGPT marcó un cambio profundo en la forma en que los usuarios acceden a la información en internet. Desde entonces, los motores generativos se han incorporado al ecosistema de búsqueda mediante herramientas como Google AI Overviews, Perplexity, ChatGPT o Claude. Para cualquier organización cuya visibilidad online sea estratégica, ya sea una empresa, una ONG o una institución académica, esta transición plantea una pregunta central: *¿qué ocurre con nuestra presencia digital cuando los usuarios preguntan a una IA en lugar de buscar en Google?*

La pregunta es reciente, pero el problema de fondo, competir por la visibilidad digital, lleva dos décadas articulándose en torno al SEO. Los buscadores tradicionales indexan páginas, las ordenan por relevancia y ofrecen al usuario una lista de enlaces. En ese modelo, competir por la visibilidad significa aparecer entre los primeros resultados (Pan y cols., 2007). Los motores generativos modifican esta lógica: sintetizan una respuesta única en lenguaje natural y citan solo algunas de las fuentes que consideran relevantes. Si un sitio no aparece citado, queda fuera del punto de contacto principal con el usuario. La citación no es, con todo, el único canal de presencia: un modelo puede mencionar una marca a partir del conocimiento adquirido en su entrenamiento, sin citar fuente alguna. Ese canal paramétrico existe, pero no es controlable desde el contenido actual del sitio ni atribuible a él; por eso este trabajo lo registra mediante la métrica de menciones de marca y centra la optimización en la citación.

Esta diferencia ha impulsado la aparición de la Generative Engine Optimization (GEO), un campo emergente que estudia cómo mejorar la visibilidad de contenidos web en respuestas generadas por LLMs. La literatura inicial muestra que las técnicas clásicas de SEO no se trasladan de forma directa al contexto generativo: factores como la claridad semántica, la presencia de datos verificables, las citas de autoridad o la estructura del contenido pueden influir en la probabilidad de citación (Aggarwal y cols., 2024).

A pesar de la relevancia del fenómeno, las herramientas para medir la visibilidad en motores generativos son todavía limitadas, comerciales y metodológicamente opacas. No existe un equivalente a Google Search Console para respuestas generadas por IA. En este vacío se sitúa el presente Trabajo Fin de Grado: el desarrollo de GEO-Audit, un sistema de auditoría que mide y diagnostica la citación de un sitio web en motores generativos, y genera recomendaciones priorizadas para orientar futuras mejoras de contenido.

1.2– Caso de estudio: Programamos.es

El profesor Jesús Moreno León, tutor de este TFG, ofreció Programamos (<https://programamos.es>) como caso de estudio para este trabajo. La asociación, fundada en 2013, enseña programación y pensamiento computacional a niños y jóvenes de 6 a 18 años, y cuenta con un corpus amplio de contenido educativo original referenciado en publicaciones académicas.

Programamos reúne las condiciones ideales para una auditoría GEO: contenido de calidad real referenciado en publicaciones académicas, buena presencia en búsqueda tradicional, pero visibilidad irregular en motores generativos. Un patrón detectado durante la fase de diseño ilustra el problema: ante la pregunta «¿Qué proyectos educativos de programación operan en Andalucía?» Gemini cita a Programamos, pero ante «¿Qué proyectos existen para enseñar programación a niños en España?» lo omite y nombra en su lugar a Code.org, Scratch Foundation o la Fundación Telefónica. El contenido no cambia entre consultas; cambia la formulación, y con ella la decisión del modelo sobre qué fuentes citar. Esto es exactamente lo que GEO-Audit pretende medir, diagnosticar y convertir en recomendaciones de mejora.

1.3– Objetivos

1.3.1. Objetivo general

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y validar un sistema de auditoría automatizada que permita medir y diagnosticar la visibilidad de un sitio web en las respuestas generadas por motores de búsqueda basados en inteligencia artificial, y generar recomendaciones priorizadas para mejorarla, aplicándolo como caso de estudio a Programamos.es.

1.3.2. Objetivos específicos

A partir del objetivo general, se derivan los siguientes ocho objetivos específicos, cada uno de los cuales aborda un componente concreto del sistema:

- OE1.** Diseñar un marco de métricas GEO fundamentado en la literatura académica (Aggarwal y cols., 2024; Kim, Jeong, Kim, Lee, y Lee, 2026; Lüttgenau, Colic, y Ramirez, 2025), que capture las distintas dimensiones de la visibilidad de un sitio web en motores generativos: desde su aparición y posición en las respuestas hasta el tono con el que se le menciona. La definición detallada de las ocho métricas que componen el marco se presenta en el capítulo 4.
- OE2.** Implementar un pipeline experimental basado en RAG que aisle la calidad del contenido como variable independiente. Construir un simulador de motor generativo que utilice Retrieval-Augmented Generation con un vectorstore FAISS congelado de competidores. El diseño experimental garantiza que la única variable que cambia entre ejecuciones es el contenido del sitio objetivo (Programamos.es), mientras que los competidores, las consultas, el modelo juez y los parámetros de recuperación permanecen constantes. Esto permite atribuir los cambios en las métricas GEO exclusivamente a las intervenciones realizadas sobre el contenido del objetivo.
- OE3.** Construir un sistema de descubrimiento de competidores basado en Gemini con Google Search grounding. Desarrollar un módulo que identifique automáticamente los competidores reales del sitio objetivo en el contexto de motores generativos, consultando a Gemini con acceso a búsqueda de Google y extrayendo las URLs

que el modelo cita en sus respuestas. Este enfoque garantiza que los competidores seleccionados son aquellos que realmente aparecen en las respuestas de motores generativos, en lugar de una selección manual susceptible a sesgos.

- OE4.** Definir un protocolo experimental con 100 consultas y sistema de rotación. Diseñar un conjunto de 100 consultas distribuidas en categorías (informacionales, comparativas y navegacionales) con un sistema de rotación que permita cubrir todo el espacio de consultas a lo largo de múltiples ejecuciones. El sistema utiliza un núcleo fijo de 20 consultas que se ejecuta en todas las iteraciones garantizando comparabilidad longitudinal, complementado con cuatro bloques rotativos de 20 consultas. Cada ejecución procesa 40 consultas en total (núcleo + un bloque rotativo).
- OE5.** Implementar un módulo de evaluación en vivo sobre motores generativos reales. Desarrollar un pipeline que consulte directamente a motores generativos comerciales (Gemini, ChatGPT y Claude) con las consultas del protocolo experimental y extraiga métricas GEO de citación a partir de sus respuestas reales. Este módulo proporciona una referencia directa de visibilidad sin intermediación del simulador y permite caracterizar la presencia del sitio en los motores que los usuarios reales utilizan.
- OE6.** Automatizar la recolección diaria de métricas SEO mediante GitHub Actions. Implementar un workflow de integración continua que ejecute diariamente auditorías de rendimiento web (PageSpeed Insights) sobre el sitio objetivo, almacenando las series temporales de métricas en el repositorio del proyecto. Esto permite correlacionar los indicadores técnicos del sitio (velocidad de carga, accesibilidad, mejores prácticas) con la evolución de las métricas GEO.
- OE7.** Desarrollar un agente de optimización GEO (GEOOptimizer). Implementar un agente basado en LLM que, a partir de los resultados de las métricas GEO y el análisis de contenido de competidores con alta visibilidad, genere recomendaciones de optimización priorizadas y ancladas a evidencia concreta. Cada recomendación especifica la palanca GEO a aplicar (citabilidad, autoridad, baja perplejidad, etc.), la ubicación exacta en el sitio web, la evidencia en competidores que la justifica, y el impacto y esfuerzo estimados.
- OE8.** Implementar un dashboard web para la ejecución, visualización y seguimiento de las métricas GEO. Desarrollar una aplicación full-stack (backend FastAPI + frontend React + TypeScript) que integre los resultados de los pipelines experimental y en vivo en un panel de control con visualizaciones longitudinales, comparativas entre ejecuciones y agregaciones por categoría de consulta.

1.4– Alcance y limitaciones

1.4.1. Alcance del trabajo

GEO-Audit se concibe como un sistema integrado con cuatro componentes principales, todos implementados y aplicados al caso de estudio de Programamos.es:

Pipeline experimental. Opera con un diseño controlado en el que el vectorstore de competidores, el modelo juez y los parámetros de recuperación permanecen congelados entre ejecuciones. La única variable que cambia es el contenido del sitio objetivo. Permite estimar la dirección del impacto de las intervenciones de optimización, aunque los valores absolutos están sesgados al alza respecto a motores reales.

Evaluación en vivo. Consulta directa a motores generativos comerciales (Gemini, ChatGPT y Claude) para obtener métricas de citación sobre respuestas reales, sin intermediación del simulador.

Agente de optimización (GEOOptimizer). Analiza los resultados de las métricas y el contenido de competidores para generar recomendaciones de mejora priorizadas y accionables.

Dashboard web. Aplicación full-stack que integra y visualiza los resultados de los pipelines experimental y en vivo, permitiendo el seguimiento longitudinal de las métricas.

El alcance del trabajo no incluye convertir GEO-Audit en una plataforma SaaS multi-sitio ni aplicar automáticamente las recomendaciones generadas sobre el contenido del sitio. El sistema desarrollado se centra en el caso de estudio de Programamos.es y cubre el ciclo de medición, diagnóstico y generación de recomendaciones accionables; la aplicación de dichas recomendaciones y la ejecución de nuevas mediciones post-intervención quedan como trabajo futuro.

1.4.2. Limitaciones

Este trabajo está sujeto a varias limitaciones propias del dominio GEO y de las restricciones prácticas de un TFG:

Variabilidad de los modelos generativos. Los LLMs pueden producir respuestas distintas entre ejecuciones, incluso con temperatura fijada en 0.0. Esta variabilidad afecta a las métricas de citación, por lo que los resultados deben interpretarse como estimaciones observadas y no como valores absolutos perfectamente reproducibles.

Sesgo del simulador RAG. El simulador utiliza recuperación vectorial sobre un corpus controlado, mientras que los motores generativos reales combinan índices propietarios, señales de autoridad, popularidad, actualidad y conocimiento paramétrico. Por ello, el simulador sirve para comparar variantes de contenido en condiciones controladas, pero no predice de forma exacta la visibilidad real.

Caso de estudio único. Todo el trabajo y la evaluación se han desarrollado específicamente para Programamos.es. Aunque el sistema podría adaptarse para su uso en otro entorno, la implementación actual está diseñada y fijada exclusivamente para las necesidades de esta plataforma.

Evolución del ecosistema. Los motores generativos cambian con rapidez: pueden modificar sus modelos, sus fuentes de recuperación, sus políticas de citación o sus criterios de selección de fuentes. Los resultados deben entenderse como una instantánea del periodo de experimentación.

Ausencia de ground truth absoluto. No existe un oráculo que determine qué fuentes debería citar un motor generativo. Las métricas GEO miden visibilidad observada, no calidad intrínseca ni visibilidad “merecida”.

1.5— Estructura del documento

El presente documento se estructura en ocho capítulos y un conjunto de apéndices, organizados de la siguiente manera:

Capítulo 1. Introducción. Presenta el contexto y la motivación del trabajo, describe el caso de estudio seleccionado, formula los objetivos general y específicos, delimita el alcance y las limitaciones, y ofrece esta visión general de la estructura del documento.

- Capítulo 2.** Estado del arte. Revisa la literatura académica y el estado de la práctica en tres ejes: (1) la evolución de los motores de búsqueda hacia sistemas generativos, (2) las técnicas de Retrieval-Augmented Generation y su aplicación a la evaluación de contenidos, y (3) los trabajos existentes en Generative Engine Optimization, incluyendo las métricas propuestas, los benchmarks disponibles y las herramientas comerciales. Se identifican las lagunas que justifican la contribución de este TFG.
- Capítulo 3.** Planificación y metodología. Describe la metodología híbrida (ingeniería del software más investigación experimental), el alcance del proyecto (requisitos de gestión y EDT), la planificación temporal y de esfuerzo, los costes, la gestión del repositorio y de riesgos, y el seguimiento y cierre.
- Capítulo 4.** Diseño del sistema. Describe la arquitectura en sus dos modos de operación, las decisiones de diseño documentadas como Architecture Decision Records (ADR), el diseño del pipeline experimental, el marco de métricas GEO y el protocolo experimental con el sistema de consultas y rotación.
- Capítulo 5.** Implementación. Detalla la implementación técnica de cada módulo del sistema: el descubridor de competidores con Google Search grounding, el procesador HTML-aware, el chunker basado en tokens, el sistema de embeddings vía API de Google (gemini-embedding-001), el RAG Judge agente, el extractor de métricas GEO, el pipeline de evaluación en vivo, el agente de optimización GEOOptimizer, el dashboard web y el workflow de CI/CD para métricas SEO. En el capítulo se presentan fragmentos de código breves y se discuten las decisiones de implementación; los listados extensos se recogen en el Anexo G.
- Capítulo 6.** Pruebas y calidad. Detalla la estrategia de pruebas, los tests automatizados de backend y frontend, la cadena de integración continua y la validación del núcleo del pipeline y del protocolo experimental.
- Capítulo 7.** Resultados y discusión. Presenta los resultados del modo experimental y de la evaluación en vivo, los resultados SEO, la discusión sobre la relación entre el simulador y los motores reales, y la evaluación cualitativa del agente optimizador.
- Capítulo 8.** Conclusiones y trabajo futuro. Resume las contribuciones del trabajo, evalúa el grado de cumplimiento de cada objetivo y propone líneas de trabajo futuro, incluyendo la extensión al modo plataforma multi-sitio, la generación automática de contenido optimizado y la ampliación de la serie temporal de evaluación en vivo.
- Apéndices.** Incluyen material complementario: el registro de decisiones arquitectónicas (ADR), el catálogo completo de las 100 consultas experimentales con sus categorías y bloques de rotación, los prompts implementados del sistema, un manual de despliegue con las instrucciones de instalación y ejecución, un manual de usuario del dashboard, el informe completo de recomendaciones del agente optimizador y los listados de código extensos de la implementación.

Estado del arte

2.1— De buscadores tradicionales a motores generativos

2.1.1. Del paradigma clásico al motor generativo

Durante más de dos décadas, la búsqueda de información en internet ha estado dominada por el modelo de recuperación y clasificación (*retrieval and ranking*): el motor indexa páginas, un algoritmo las ordena por relevancia y el usuario recibe una lista de enlaces (el paradigma *ten blue links*) donde los diez primeros resultados acaparan más del 90 % de los clics (Chitika Inc., 2013; Advanced Web Ranking, 2024). El SEO (*Search Engine Optimization*) ha codificado este modelo en una disciplina que combina optimización técnica (velocidad de carga, estructura HTML, datos estructurados), de contenido (relevancia temática, calidad editorial) y de autoridad (backlinks, reputación de dominio) para mejorar el posicionamiento orgánico (Fishkin, 2023).

La irrupción de los LLMs a partir de 2022 ha catalizado un cambio de paradigma. Los denominados *motores generativos*, como ChatGPT (OpenAI), Gemini (Google), Perplexity o Claude (Anthropic), no recuperan y ordenan documentos: generan respuestas completas, sintetizando información de múltiples fuentes en texto coherente y directamente consumible (Metzler, Tay, Bahri, y Najork, 2021; Aggarwal y cols., 2024). La decisión de citar no es una función de ranking basada en señales SEO convencionales, sino una decisión del modelo en la etapa de generación, influida por la relevancia del contenido recuperado, la confianza del modelo en la fuente y la capacidad del contenido para integrarse en una respuesta coherente.

2.1.2. Arquitectura de los principales motores generativos

Todos los motores generativos relevantes para este trabajo comparten el mismo patrón: recuperación de información web seguida de síntesis mediante un LLM. Sus diferencias radican en el índice de búsqueda subyacente y en la transparencia de los metadatos de citación.

ChatGPT accede al índice de Bing cuando determina que una consulta requiere información actualizada. La respuesta sintetiza la información recuperada e incluye citas con enlaces, pero el proceso de selección de fuentes opera de forma opaca (OpenAI, 2024).

Gemini dispone de *Google Search grounding*, que permite al modelo acceder al índice de Google durante la generación. Su principal ventaja para la investigación en GEO es que expone metadatos estructurados de las fuentes, distinguiendo entre URLs recuperadas y fragmentos efectivamente citados en la respuesta, lo que facilita el cálculo de métricas de citación (Google LLC, 2025).

Claude integra una herramienta de búsqueda web que ejecuta consultas contra índices web y devuelve resultados estructurados con URLs y fragmentos, que el modelo incorpora con citas en su respuesta (Anthropic, 2025).

2.2— Generative Engine Optimization

2.2.1. Definición y taxonomía

El término *Generative Engine Optimization* (GEO) fue formalizado por Aggarwal y cols. (2024) en su trabajo seminal, donde lo definen como el conjunto de estrategias para “optimizar el contenido web con el objetivo de mejorar su visibilidad en las respuestas generadas por motores de búsqueda basados en modelos de lenguaje generativo”. Esta definición establece la distinción fundamental con el SEO clásico: mientras que el SEO busca mejorar la *posición* de un enlace en una lista de resultados, el GEO busca maximizar la *probabilidad de citación* en un texto generado.

La literatura emergente coincide en clasificar las estrategias GEO en tres niveles que permiten organizar los hallazgos empíricos:

1. **Estrategias a nivel de contenido** (*content-level*). Se centran en las propiedades intrínsecas del texto: inclusión de estadísticas y datos cuantitativos, citas a fuentes autoritativas, escritura con baja perplejidad (textos que el LLM encuentra “naturales” y fáciles de procesar), y párrafos autocontenidos que pueden ser extraídos e integrados en una respuesta sin perder coherencia.
2. **Estrategias a nivel estructural** (*structural-level*). Operan sobre la organización y el marcado del contenido: uso semántico de HTML (encabezados jerárquicos, listas, tablas), implementación de datos estructurados mediante Schema.org (JSON-LD), estructura de párrafos con oraciones temáticas (*topic sentences*) y uso de secciones FAQ con formato pregunta-respuesta.
3. **Estrategias a nivel de autoridad** (*authority-level*). Se refieren a señales externas al contenido: backlinks de dominios de alta autoridad, menciones de marca en fuentes reconocidas, presencia en bases de conocimiento (Wikipedia, Wikidata) y patrones de citación en la literatura académica.

Esta taxonomía tripartita revela una diferencia adicional entre SEO y GEO. En SEO, las señales de autoridad convencionales (backlinks, Domain Authority) han sido históricamente las más influyentes en el ranking (Moz, 2024). En GEO, por el contrario, las estrategias a nivel de contenido son las que mayor impacto demuestran, porque el LLM toma la decisión de citación basándose primariamente en la calidad del fragmento recuperado, priorizando esta sobre las señales de autoridad de dominio convencionales como el PageRank o el Domain Authority (Aggarwal y cols., 2024).

2.2.2. Métricas de visibilidad

La medición de la visibilidad en motores generativos requiere métricas específicas que capturen dimensiones no contempladas por las métricas SEO tradicionales. La literatura reciente ha propuesto diversas métricas, a menudo con nombres diferentes para conceptos similares. La Tabla 2.1 presenta una comparativa sistemática de las métricas propuestas por los principales trabajos, junto con la denominación adoptada en este TFG.

Tabla 2.1: Comparativa de métricas de visibilidad GEO en la literatura.

Métrica	Aggarwal (2024)	Kim et al. (2026, SAGEO)	Este trabajo
Visibilidad binaria	Source-level Visibility	Visibility	Visibility
Cuota de citación	Subjective Impression	Source Share	Share of Model
Posición	–	–	Ranking
Recuento pond. por posición	Position-Adj. Word Count	–	PAWC
Tasa de citación	–	Citation Rate	Citation Rate
Menciones de marca	–	–	Brand Mentions
Sentimiento	–	–	Sentiment
Cobertura entre motores	–	–	Engine Coverage

A continuación se definen formalmente las métricas adoptadas en este trabajo.

Visibilidad (*Visibility*). Métrica binaria que indica si el sitio objetivo aparece citado en la respuesta generada para una consulta dada. Formalmente, para una consulta q y un sitio objetivo t :

$$\text{Visibility}(q, t) = \begin{cases} 1 & \text{si } t \in \text{Citations}(q) \\ 0 & \text{en caso contrario} \end{cases} \quad (2.1)$$

El *Visibility Rate* agregado sobre un conjunto de consultas Q es simplemente la proporción de consultas donde el sitio es visible:

$$\text{Visibility Rate}(Q, t) = \frac{1}{|Q|} \sum_{q \in Q} \text{Visibility}(q, t) \quad (2.2)$$

Share of Model (*SoM*). Analogía del *Share of Voice* en marketing digital. Mide la proporción de citas que corresponden al sitio objetivo sobre el total de citas en la respuesta. Para una consulta q con n citas totales, de las cuales n_t corresponden al sitio objetivo:

$$\text{SoM}(q, t) = \frac{n_t}{n} \times 100 \quad (2.3)$$

Ranking. Posición ordinal (1-indexed) de la primera citación del sitio objetivo en la respuesta. Las citas se ordenan por su aparición en el texto. Un valor menor indica mayor prominencia. Se asigna None cuando el sitio no es citado.

PAWC (*Position-Adjusted Word Count*). Métrica propuesta por Aggarwal y cols. (2024) que pondera el recuento de palabras de cada cita del sitio objetivo por su posición, aplicando un factor de descuento logarítmico similar al DCG (*Discounted Cumulative Gain*) en recuperación de información:

$$\text{PAWC}(q, t) = \sum_{i \in \text{Citas}_t} \frac{w_i}{\log_2(i + 1)}$$

donde w_i es el número de palabras del fragmento citado (*quote*) en la posición i -ésima.

Citation Rate. Propuesta por Kim y cols. (2026) en el benchmark SAGEO Arena. Mide la eficiencia de conversión de recuperación a citación: la proporción de veces que un sitio es efectivamente citado respecto al total de veces que fue recuperado (es decir, estuvo disponible en el contexto del LLM):

$$\text{Citation Rate}(q, t) = \frac{|\text{Citado}(q, t)|}{|\text{Recuperado}(q, t)|} \times 100$$

Esta métrica es especialmente relevante porque permite distinguir entre problemas de recuperación (el contenido no llega al contexto del LLM) y problemas de generación (el contenido llega pero no es citado), orientando así las estrategias de optimización.

Brand Mentions. Recuento de apariciones explícitas del nombre de la marca o del dominio objetivo en el texto generado, con independencia de si van acompañadas de un enlace citado. Captura la presencia de marca en respuestas donde el motor generativo menciona la fuente en prosa pero no la formaliza como cita.

Sentiment. Clasificación del tono con el que el motor generativo menciona al sitio o marca objetivo (positivo, neutro, negativo). Aunque no es estrictamente una métrica de visibilidad, complementa la evaluación cualitativa de cómo el LLM percibe y presenta la marca.

Engine Coverage. Proporción de motores generativos evaluados en los que el sitio es visible. Relevante en evaluaciones multi-motor para identificar dependencias de plataforma.

2.2.3. Estrategias de optimización

La literatura empírica ha identificado diversas estrategias de contenido que mejoran significativamente la visibilidad en motores generativos. A continuación se resumen los hallazgos más relevantes, organizados por tipo de estrategia.

Inclusión de estadísticas y datos cuantitativos. Aggarwal y cols. (2024) demuestran que la inclusión de datos estadísticos en el contenido web aumenta la visibilidad hasta un 40 % sobre la línea base, situándose entre las palancas más efectivas de las nueve evaluadas en su estudio. La explicación propuesta es que los LLMs, al generar respuestas, priorizan fragmentos que contienen datos concretos porque estos aportan valor informativo diferencial y son fáciles de integrar en una respuesta factual.

Citas a fuentes autoritativas. La inclusión de referencias a estudios académicos, informes institucionales u otras fuentes reconocidas en el propio contenido web es una de las palancas que Aggarwal y cols. (2024) identifican como efectivas, y reportan que la combinación de citas, comillas y estadísticas eleva la visibilidad por encima del 40 %. Este efecto se explica por el sesgo de autoridad (*authority bias*) documentado en LLMs: los modelos tienden a confiar más en las fuentes que a su vez citan otras fuentes de calidad reconocida, y por tanto a citarlas con mayor frecuencia.

Escritura con baja perplejidad. Aggarwal y cols. (2024) analizan también la *fluency optimization*: reescribir el contenido de modo que su perplejidad para un LLM sea menor (es decir, que el texto resulte más predecible y fluido al modelo) aumenta la probabilidad de citación hasta un 37 %. La hipótesis subyacente es que el LLM encuentra más fácil integrar fragmentos “fluidos” en su propia generación, frente a textos con vocabulario inusual o construcciones sintácticas atípicas.

Estructura semántica HTML. Tan y cols. (2025), en su trabajo *HtmlRAG*, demuestra que la estructura HTML del contenido proporciona señales semánticas valiosas para los LLMs. Encabezados jerárquicos (<h1>-<h4>), listas ordenadas y desordenadas (,), tablas (<table>) y otros elementos estructurales ayudan al modelo a comprender la organización del contenido y a identificar los fragmentos más relevantes. Esta línea de investigación extiende un principio ya presente en SEO, según el cual el contenido bien estructurado es más fácil de procesar para las máquinas, a un escenario más exigente: no solo facilitar la indexación del crawler, sino permitir que el LLM extraiga, sintetice y cite fragmentos coherentes.

Párrafos autocontenidos. Aggarwal y cols. (2024) reportan que reescribir el contenido en párrafos autocontenidos, donde cada párrafo puede ser comprendido de forma aislada sin depender del contexto previo, mejora la citabilidad de forma moderada respecto a la línea base. Esta propiedad es crítica en el contexto de sistemas RAG, donde los fragmentos son recuperados individualmente y presentados al LLM fuera de su contexto original.

Datos estructurados Schema.org. La implementación de datos estructurados en formato JSON-LD (FAQPage, HowTo, Article, Organization) facilita que los motores generativos extraigan información factual de forma fiable. Aunque el impacto cuantitativo

específico en GEO no ha sido evaluado de forma aislada en la literatura, el uso de Schema.org es una recomendación consistente tanto en guías de GEO como en las documentaciones oficiales de los principales motores (Google Search Central, 2025).

llms.txt. Una propuesta emergente, todavía sin estandarización formal, es el archivo `llms.txt`: un fichero de texto plano ubicado en la raíz del sitio web (análogamente a `robots.txt`) que proporciona al motor generativo un resumen estructurado del contenido del sitio, sus páginas principales y la información que el propietario considera más relevante para ser citada. A diferencia de Schema.org, no requiere marcado HTML y está diseñado específicamente para ser consumido por LLMs durante la fase de recuperación. Su impacto empírico aún no ha sido cuantificado en la literatura.

La Tabla 2.2 resume el impacto cuantitativo de las principales estrategias documentadas.

Tabla 2.2: Impacto cuantitativo de las estrategias de optimización GEO.

Estrategia	Mejora de visibilidad	Fuente
Estadísticas y datos cuantitativos	Hasta +40 %	(Aggarwal y cols., 2024)
Citas a fuentes autoritativas	>40 % combinada	(Aggarwal y cols., 2024)
Baja perplejidad en escritura	Hasta +37 % citabilidad	(Aggarwal y cols., 2024)
Estructura semántica HTML	Mejora significativa	(Tan y cols., 2025)
Párrafos autocontenidos	Moderado	(Aggarwal y cols., 2024)
Datos estructurados (Schema.org)	No cuantificado aisladamente	(Google LLC, 2025)

2.3— Sistemas RAG

2.3.1. Arquitectura y componentes

La *Retrieval-Augmented Generation* (RAG) es una arquitectura que combina recuperación de información con generación de texto mediante LLMs (Lewis y cols., 2020). Todos los motores generativos descritos en la sección anterior implementan internamente alguna variante de RAG: recuperan información de la web y la utilizan como contexto para la generación de la respuesta.

La arquitectura RAG clásica consta de cinco etapas: (1) ingestión de documentos fuente con extracción de contenido textual; (2) fragmentación (*chunking*) en bloques de tamaño controlado; (3) vectorización (*embedding*) de cada fragmento en un espacio semántico de alta dimensionalidad; (4) búsqueda vectorial por similitud coseno ante una consulta; y (5) generación con los fragmentos recuperados como contexto. La calidad de cada etapa se propaga al resultado final: contenido ruidoso produce fragmentos de baja calidad, embeddings menos representativos y respuestas degradadas (Barnett, Kurniawan, Thudumu, Brannelly, y Abdelrazek, 2024).

2.3.2. Chunking y configuración de referencia

La fragmentación es una de las decisiones de diseño más influyentes en un sistema RAG. El tamaño del fragmento determina la granularidad de las citas: fragmentos muy grandes mezclan múltiples temas diluyendo la relevancia, mientras que fragmentos muy pequeños pierden contexto necesario para su comprensión.

Kim y cols. (2026), en el benchmark SAGEO Arena, establecen como configuración de referencia 256 tokens con 64 tokens de solapamiento, alineada con la ventana de contexto que los motores generativos reales utilizan al seleccionar fuentes (BrightEdge Research, 2024). Esta configuración se ha adoptado como estándar en la comunidad investigadora en GEO y es la utilizada en el presente trabajo.

2.3.3. Estructura HTML y citabilidad

Tan y cols. (2025) demuestran empíricamente en su trabajo *HtmlRAG* que “HTML es mejor que texto plano para RAG”: los LLMs son capaces de aprovechar las etiquetas HTML (encabezados, listas, tablas) para comprender la jerarquía y relevancia del contenido con mayor efectividad que con texto plano. Los encabezados establecen jerarquía temática, las listas y tablas codifican relaciones entre datos, y los datos estructurados JSON-LD proporcionan información factual verificada directamente procesable por el LLM.

Para un sistema de auditoría GEO, esto implica que la conversión de HTML a texto debe ser *selectiva*: eliminar ruido (scripts, estilos, navegación) pero preservar la estructura semántica (encabezados, listas, tablas). Una conversión agresiva a texto plano descarta información relevante tanto para la recuperación como para la generación.

2.4– Trabajos relacionados

2.4.1. Benchmarks y marcos de evaluación GEO

La evaluación sistemática de la visibilidad en motores generativos es un campo de investigación reciente, con los primeros trabajos formales aparecidos en 2023. A continuación se revisan los principales marcos de evaluación y se posiciona el presente trabajo en relación con ellos.

GEO-Bench (Aggarwal y cols. (2024)). Es el primer benchmark formal para la evaluación de estrategias GEO. Propone nueve estrategias de optimización de contenido y las evalúa utilizando un RAG simulado sobre un corpus de documentos web. Las métricas empleadas son *Source-level Visibility*, *Position-Adjusted Word Count* (PAWC) y *Subjective Impression*. GEO-Bench establece la metodología de evaluación mediante RAG simulado que ha sido adoptada por trabajos posteriores, incluyendo el presente.

SAGEO Arena (Kim y cols. (2026)). Extiende la evaluación GEO con un benchmark más completo que incluye múltiples configuraciones de RAG (diferentes tamaños de fragmento, modelos de embedding, modelos juez). Su principal contribución es la métrica *Citation Rate* y la estandarización de una configuración de referencia (256 tokens / 64 overlap) que permite la comparabilidad entre estudios.

Lüttgenau y cols. (2025). En *Beyond SEO* proponen un enfoque basado en *transformers* para reescribir automáticamente contenido web con el objetivo de incrementar su presencia en respuestas generativas; reportan mejoras del orden del 15,63 % en *absolute word count* sobre contenido optimizado frente a su versión original. Su contribución se sitúa en el lado de la generación automática de variantes, complementario al enfoque de auditoría y recomendación priorizada de GEO-Audit.

2.4.2. Limitaciones del estado del arte

El análisis de la literatura revela dos limitaciones sistémicas que motivan el presente trabajo:

1. **Descubrimiento manual de competidores.** Todos los benchmarks existentes definen manualmente los dominios competidores, introduciendo un sesgo de selección que puede no reflejar el ecosistema competitivo real en motores generativos. Los competidores que un humano considera relevantes pueden diferir significativamente de los que los modelos de lenguaje citan efectivamente en sus respuestas.
2. **Separación entre benchmark y herramienta.** Los trabajos existentes producen resultados de investigación (benchmarks, rankings, métricas) pero no sistemas reutilizables que permitan a propietarios de sitios web auditar su propia visibilidad GEO,

ni mecanismos de recomendación que traduzcan las métricas en acciones de mejora concretas.

GEO-Audit aborda estas dos limitaciones: automatiza el descubrimiento de competidores vía Gemini con Google Search grounding y produce un sistema auditable con un agente de recomendaciones (GEOOptimizer).

Planificación y metodología

3.1— Metodología de desarrollo

El desarrollo de GEO-Audit ha requerido una metodología híbrida que combina elementos propios de la ingeniería del software con los procedimientos típicos de la investigación experimental. Esta dualidad responde a la naturaleza misma del proyecto: por un lado, se trata de la construcción de un sistema software complejo con múltiples módulos interconectados; por otro, el sistema constituye un instrumento de investigación cuyo propósito es producir métricas de visibilidad GEO y recomendaciones de optimización para un caso de estudio real.

3.1.1. Enfoque iterativo adaptado

La metodología adoptada ha sido un enfoque iterativo adaptado: el proyecto se ha organizado en fases secuenciales con entregables concretos, pero los aprendizajes de cada fase han retroalimentado las decisiones técnicas anteriores. Cada fase produce un artefacto verificable (vectorstore congelado, conjunto de métricas, scorecard de baseline, recomendaciones GEO, dashboard) que sirve de punto de partida para la siguiente.

La Figura 3.1 ilustra los seis subsistemas del pipeline y los artefactos que se transfieren entre ellos.

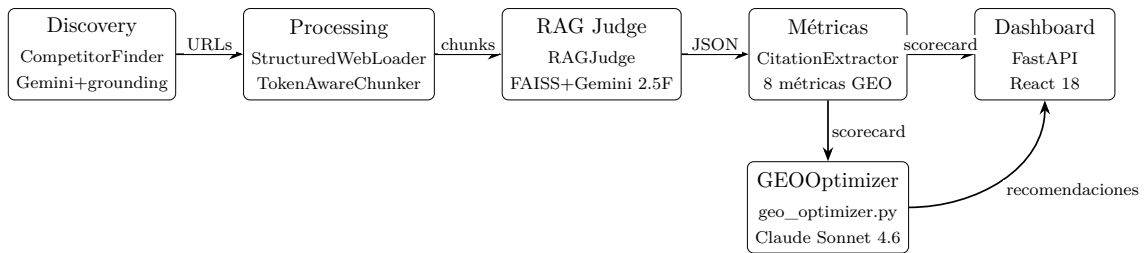


Figura 3.1: Diagrama de componentes del pipeline GEO-Audit.

3.1.2. Patrones de diseño

El diseño de GEO-Audit aplica dos patrones de diseño en los puntos donde la extensibilidad o el desacoplamiento resultan críticos para el control experimental: Adapter (Gamma et al., *Design Patterns*, 1994) y Registry (Fowler, *Patterns of Enterprise Application Architecture*, 2002).

Adapter (`src/rag/search_tool.py`). La función `create_search_tool()` envuelve un índice FAISS en la interfaz Tool de LangChain, adaptando una biblioteca de búsqueda vectorial al contrato que espera el agente RAG Judge. Este desacoplamiento permite sustituir el vectorstore subyacente sin modificar la lógica del agente.

Registry (`src/prompts/registry.py`). `PROMPT_REGISTRY` es un registro centralizado de prompts versionados semánticamente, cada uno con su propio changelog. Dado que cualquier cambio en el prompt del RAG Judge altera el comportamiento del simulador y, por tanto, las métricas resultantes, el patrón Registry garantiza que las modificaciones sean explícitas, trazables y reversibles.

3.1.3. Dimensión investigadora de la metodología

Además de su componente de ingeniería del software, el proyecto incorpora elementos propios de la metodología de investigación experimental:

- **Diseño con variables controladas.** El pipeline experimental (Capítulo 4) aísla la calidad del contenido del sitio objetivo como única variable independiente. Los competidores, las consultas, el modelo juez, los parámetros de recuperación y el vectorstore se congelan, de modo que cualquier cambio observado en las métricas GEO pueda atribuirse a modificaciones en el contenido del sitio objetivo.
- **Protocolo de consultas fundamentado.** Las 100 consultas experimentales se han diseñado a partir de la taxonomía de tipos de consulta de (Broder, 2002), adaptada al dominio específico del caso de estudio. Se distribuyen en tres categorías (informacionales, comparativas y navegacionales) con un sistema de rotación que equilibra cobertura y coste.
- **Evaluación en motores reales.** La evaluación en vivo contra Gemini, ChatGPT y Claude permite extraer métricas de visibilidad GEO en producción, complementando los resultados del simulador RAG con datos obtenidos directamente de los motores generativos sobre los que el sitio objetivo busca posicionarse.

3.2– Alcance del proyecto

El alcance del proyecto define el conjunto de trabajo necesario para entregar el sistema GEO-Audit con las características y funciones especificadas. Siguiendo las prácticas de la *Guía de los fundamentos para la dirección de proyectos* (PMBOK), se distinguen dos dimensiones complementarias: el alcance del proyecto, entendido como el trabajo a realizar para entregar el producto, y el alcance del producto, es decir, las características y funciones que definen al propio sistema.

A continuación se presentan, primero, el catálogo de requisitos de gestión que enmarca el desarrollo del trabajo, y a continuación la línea base del alcance, formada por el enunciado del alcance, la Estructura de Descomposición del Trabajo (EDT) y su diccionario.

3.2.1. Catálogo de requisitos de gestión

En esta sección se definen los requisitos de *gestión* del proyecto, que no deben confundirse con los requisitos del propio sistema (objetivos OE1–OE8 del Capítulo 1). Los requisitos de gestión se clasifican en cuatro familias siguiendo la taxonomía PMBOK: requisitos de negocio, de las soluciones, del proyecto y de calidad. Cada uno indica una prioridad (*Urgente, Alta, Media, Baja*) y el entregable de la EDT al que contribuye.

Requisitos de negocio

Describen las necesidades de alto nivel del trabajo a realizar.

RN-01: Elaboración y seguimiento de la planificación	
Descripción	Previo al inicio del proyecto debe elaborarse una planificación temporal, de alcance y de riesgos, así como mantener un seguimiento periódico para evitar desviaciones respecto a los objetivos académicos.
Prioridad	Urgente
Entregable EDT	2.1. Memoria: Planificación y seguimiento
RN-02: Redacción de la memoria del TFG	
Descripción	Durante y al finalizar el desarrollo, debe generarse una memoria que documente todos los aspectos relevantes del producto y del proceso, conforme al formato exigido por la ETSII de la Universidad de Sevilla.
Prioridad	Alta
Entregable EDT	3. Memoria
RN-03: Construcción del sistema GEO-Audit	
Descripción	El producto software a desarrollar debe consistir en un sistema completo de auditoría de visibilidad en motores generativos: pipeline experimental, pipeline live, agente de optimización y dashboard, aplicado al caso de estudio Programamos.es.
Prioridad	Alta
Entregable EDT	1. Producto software
RN-04: Defensa pública del trabajo	
Descripción	Debe prepararse una presentación que sintetice los aspectos esenciales del proyecto para su defensa ante el tribunal académico.
Prioridad	Media
Entregable EDT	4. Defensa

Requisitos de las soluciones

Describen las funciones y características del producto a alto nivel y se derivan directamente de los objetivos específicos OE1–OE8 enunciados en el Capítulo 1.

RS-01: Pipeline experimental basado en RAG (cubre OE2, OE3, OE4)	
Descripción	Debe existir un simulador RAG controlado, con vectorstore congelado de competidores y modelo juez determinista, capaz de aislar el contenido del sitio objetivo como única variable independiente.
Prioridad	Urgente
Entregable EDT	1.1. Pipeline experimental

RS-02: Marco de 8 métricas GEO (cubre OE1)	
Descripción	Debe implementarse un módulo de cálculo de métricas (Visibilidad, Share of Model, Ranking, PAWC, Citation Rate, Brand Mentions, Sentiment, Engine Coverage) fundamentado en la literatura académica.
Prioridad	Alta
Entregable EDT	1.1. Pipeline experimental
RS-03: Pipeline de evaluación en vivo (cubre OE5)	
Descripción	Debe implementarse un módulo que consulte directamente Gemini, ChatGPT y Claude con el banco de queries del experimento y extraiga las métricas GEO sobre las respuestas reales.
Prioridad	Alta
Entregable EDT	1.2. Pipeline live
RS-04: Agente GEOOptimizer (cubre OE7)	
Descripción	Debe existir un agente basado en LLM que, partiendo de los resultados del pipeline experimental y del contenido de los competidores, genere recomendaciones priorizadas y accionables ancladas a evidencia concreta.
Prioridad	Media
Entregable EDT	1.3. Agente GEOOptimizer
RS-05: Dashboard web (cubre OE8)	
Descripción	Debe desarrollarse una aplicación full-stack (FastAPI + React) que integre los resultados de los pipelines experimental y live, las métricas SEO y las recomendaciones del optimizador en una interfaz interactiva.
Prioridad	Media
Entregable EDT	1.4. Dashboard

Requisitos del proyecto

Describen procesos y condiciones necesarios para la correcta ejecución del proyecto.

RP-01: Cronograma de actividades	
Descripción	Debe establecerse un cronograma con la división temporal de fases y tareas, así como la estimación de esfuerzo asociada.
Prioridad	Urgente
Entregable EDT	2.1. Memoria: Planificación y seguimiento
RP-02: Reproducibilidad del experimento	
Descripción	Las ejecuciones experimentales deben ser reproducibles: vectorstore de competidores congelado, prompts y configuración versionados, modelo juez determinista (temperature = 0,0).
Prioridad	Alta
Entregable EDT	1.1. Pipeline experimental

RP-03: Control de versiones y trazabilidad

Descripción	Todo el código, configuración, prompts y datos generados deben mantenerse bajo control de versiones (Git), con estrategia de ramificación documentada y commits trazables.
Prioridad	Alta
Entregable EDT	1. Producto software

RP-04: Seguimiento por hito

Descripción	Para cada fase completada debe realizarse un seguimiento del esfuerzo real frente al estimado, así como una retrospectiva sobre las desviaciones detectadas.
Prioridad	Media
Entregable EDT	2.1. Memoria: Planificación y seguimiento

Requisitos de calidad

Describen las condiciones necesarias para validar el éxito del proyecto.

RC-01: Validación continua con tutores

Descripción	Cada fase importante debe ser validada en reunión con los tutores del trabajo para asegurar la alineación con los objetivos y evitar desvíos significativos.
Prioridad	Alta
Entregable EDT	Todos los paquetes

RC-02: Tests automatizados en integración continua

Descripción	Los módulos no generativos del backend y del frontend deben disponer de tests unitarios y de integración ejecutados automáticamente en GitHub Actions ante cada <i>pull request</i> contra la rama <code>main</code> .
Prioridad	Alta
Entregable EDT	1.4. Dashboard, 1.1. Pipeline experimental

RC-03: Documentación de decisiones (ADRs)

Descripción	Cada decisión arquitectónica relevante debe documentarse como <i>Architecture Decision Record</i> (ADR) en <code>docs/DECISIONS.md</code> , indicando contexto, decisión, justificación y alternativas descartadas.
Prioridad	Media
Entregable EDT	3. Memoria

3.2.2. Línea base del alcance

La línea base del alcance es la versión final acordada del enunciado del alcance junto con la EDT y su diccionario asociado.

Enunciado del alcance

El objetivo central del trabajo es diseñar, implementar y validar un sistema de auditoría automatizada (GEO-Audit) que mida la visibilidad de un sitio web en las respuestas generadas por motores de búsqueda basados en LLMs y produzca recomendaciones priorizadas para mejorarla, aplicándolo como caso de estudio a Programamos.es.

Para alcanzar este objetivo se construyen cuatro componentes integrados (pipeline experimental basado en RAG, pipeline de evaluación en vivo sobre motores reales, agente de optimización GEOOptimizer y dashboard web), se ejecuta un protocolo experimental con 100 consultas en un sistema de rotación, y se documenta todo el proceso en una memoria técnica acompañada de la presentación de defensa.

Los entregables principales son: (i) el repositorio de código y datos del sistema, (ii) la presente memoria del trabajo, y (iii) la presentación de defensa.

Estructura de Descomposición del Trabajo (EDT)

La Figura 3.2 muestra la jerarquía de paquetes de trabajo del proyecto.

```
TFG: SEO tecnico + AISO (caso Programamos.es)
|
+-- 1. Producto software (sistema GEO-Audit)
|   +-- 1.1. Pipeline experimental (RAG + 8 metricas GEO)
|   +-- 1.2. Pipeline live (Gemini + ChatGPT + Claude)
|   +-- 1.3. Agente GEOOptimizer
|   +-- 1.4. Dashboard web (FastAPI + React)
|
+-- 2. Investigacion
|   +-- 2.1. Estado del arte (revision sistematica)
|   +-- 2.2. Discovery de competidores (Gemini + grounding)
|   +-- 2.3. Protocolo experimental + ejecucion de runs
|
+-- 3. Memoria
|
+-- 4. Defensa
```

Figura 3.2: Estructura de Descomposición del Trabajo (EDT) del proyecto.

Diccionario de la EDT

Para cada paquete de trabajo se especifica su descripción, el rol funcional responsable (todos los roles los asume el alumno alternando *sombreros*) y las tareas asociadas. Los roles funcionales identificados son: *jefe de proyecto* (planificación, seguimiento, redacción de la memoria), *investigador / científico de datos* (revisión bibliográfica, diseño experimental, ejecución de runs), *desarrollador backend* (Python, FastAPI, pipeline RAG) y *desarrollador frontend* (React + TypeScript).

Tabla 3.1: Diccionario de la EDT (paquetes hoja).

Paquete	Descripción	Rol responsable	Tareas asociadas
1.1. Pipeline experimental	Simulador RAG controlado: scraping HTML, chunking, embeddings, vectorstore FAISS congelado, agente RAG Judge con tool calling y extractor de métricas GEO.	Investigador + dev backend	Fase 2 (pipeline core), Fase 3 (métricas), Fase 4 (experimentación)
1.2. Pipeline live	Módulo de consulta directa a Gemini, ChatGPT y Claude con el banco de queries y cálculo de las métricas GEO sobre respuestas reales.	Dev backend	Fase 4 (construcción) + CI semanal continuo desde S9
1.3. Agente GEOOptimizer	Agente basado en Claude Sonnet 4.6 que produce hasta 8 recomendaciones priorizadas a partir de los scorecards y del contenido de competidores citados.	Dev backend	Fase 6 (agente GEOOptimizer)
1.4. Dashboard	Aplicación full-stack (FastAPI + React 18 + TypeScript + Vite) que integra resultados de los pipelines experimental y live, métricas SEO y recomendaciones del optimizador.	Dev backend + dev frontend	Fase 5 (dashboard web)
2.1. Estado del arte	Revisión sistemática de la literatura sobre GEO, RAG y métricas de visibilidad, incluyendo la asistencia a webinars y <i>workshops</i> del sector.	Investigador	Fase 0 (estudio)
2.2. Discovery de competidores	Identificación automatizada de los sitios web competidores mediante consulta a Gemini con Google Search grounding y agregación ponderada por dominio.	Investigador + dev backend	Fase 4 (experimentación, ejecución única)
2.3. Protocolo experimental + runs	Diseño del banco de 100 queries con rotación, ejecución de los runs experimentales y generación de scorecards.	Investigador	Fase 4 (experimentación, N ejecuciones)
3. Memoria	Documento técnico que recoge todo el trabajo: contexto, estado del arte, planificación, diseño, implementación, resultados y conclusiones.	Jefe de proyecto	Transversal (todas las fases)
4. Defensa	Presentación que sintetiza el proyecto para la exposición ante el tribunal.	Jefe de proyecto	Fase 7 (cierre)

3.3— Planificación temporal

El desarrollo de GEO-Audit se ha estructurado en 8 fases distribuidas a lo largo de 15 semanas, con una carga de trabajo estimada de 300 horas.

La redacción de la memoria del TFG no se ha concentrado en una fase final, sino que ha sido un proceso transversal y progresivo a lo largo de todo el proyecto: cada fase ha producido borradores parciales del capítulo correspondiente, revisados e integrados de forma incremental.

3.3.1. Descripción de las fases

Fase 0: Estudio e investigación del estado del arte (Semanas 1–2). Revisión sistemática de la literatura sobre Generative Engine Optimization, sistemas RAG y métricas de visibilidad en motores generativos. Se analizaron los trabajos seminales (Aggarwal y cols., 2024; Kim y cols., 2026; Tan y cols., 2025) y se definió el alcance del sistema a construir. Esta fase incluyó además sesiones de formación específicas sobre LLMs, sistemas RAG y GEO impartidas por los tutores del trabajo, así como la asistencia a webinars y *workshops* especializados del sector, como el webinar de GEO

de Try Geometrics, lo que permitió contrastar la literatura académica con las prácticas emergentes en la industria. La fase proporcionó la fundamentación académica y profesional del marco de métricas y del protocolo experimental. Aunque el núcleo de la investigación se desarrolló de forma concentrada en estas dos semanas, la actividad de estudio tuvo una cola transversal que se prolongó hasta la Semana 6, con lecturas puntuales sobre chunking SAGEO Arena, taxonomía de queries y descubrimiento con *grounding* (decisiones que se materializaron en ADR-011, ADR-013 y ADR-010 respectivamente) y se cerró con la asistencia al webinar de Try Geometrics.

Fase 1: Setup e infraestructura (Semana 2). Esta fase constituye los cimientos técnicos del proyecto. Se reestructuró el repositorio desde un prototipo inicial en cuaderno Jupyter hacia una arquitectura modular con paquetes Python, se congelaron las dependencias del entorno, se creó la configuración centralizada del experimento y se implementó el registro de prompts versionado. En esta fase se programó además el workflow de integración continua `seo_audit.yml`, que desde su puesta en marcha recolecta diariamente métricas SEO (Lighthouse / PageSpeed Insights) del sitio objetivo y las persiste en `data/seo/`, acumulando una serie temporal estable durante el resto del proyecto sin requerir intervención manual. Los entregables de esta fase son la estructura de directorios definitiva, la configuración base del sistema y el pipeline diario de SEO en producción.

Fase 2: Pipeline core (Semanas 3–5). Implementación de los módulos centrales del pipeline de procesamiento y evaluación. Se desarrolló el procesador HTML-aware (`StructuredWebLoader`) capaz de extraer contenido principal eliminando ruido de navegación, el chunker basado en tokens con tiktoken (`TokenAwareChunker`), el RAG Judge con salida JSON estructurada, el extractor de citas (`CitationExtractor`) y la herramienta de búsqueda FAISS como tool de LangChain. Esta fase concentra el grueso del desarrollo de los módulos centrales del pipeline experimental.

Fase 3: Marco de métricas (Semana 6). Implementación del marco completo de 8 métricas GEO fundamentadas en la literatura académica: Visibilidad, Share of Model, Ranking, PAWC, Citation Rate, Brand Mentions, Sentiment y Engine Coverage. Se implementó también el scorecard agregado que sintetiza todas las métricas en un diagnóstico global, con identificadores de consulta (Q001–Q100) y agregados por categoría.

Fase 4: Experimentación (Semanas 7–11). La fase más extensa del proyecto en duración y horas estimadas, dedicada a la ejecución del protocolo experimental. Comprende la ejecución del discovery de competidores con Gemini y Google Search grounding, la congelación del vectorstore y la ejecución de la línea base (baseline). Cada ejecución experimental procesa 40 consultas (20 core + 20 rotativas) y genera un snapshot completo de métricas GEO, almacenado en `data/geo/experimental/`. Hacia el final de la fase se construyó además la infraestructura del pipeline de evaluación en vivo (`collect_geo_live.py` y el workflow `geo_live.yml`), que a partir de la Semana 9 consulta automáticamente cada domingo los motores generativos reales (Gemini, ChatGPT y Claude) con el banco de queries y persiste los scorecards semanales en `data/geo/live/`. En paralelo, el workflow de CI `seo_audit.yml` recolecta diariamente métricas SEO del sitio objetivo (PageSpeed Insights), acumulando una serie temporal en `data/seo/`.

Fase 5: Dashboard web (Semanas 10–12, en paralelo con Fase 4). Implementación del dashboard de visualización de resultados con arquitectura full-stack: backend FastAPI que expone las métricas del pipeline experimental y de evaluación en vivo a

través de una API REST, y frontend React 18 con TypeScript y Vite que presenta los datos en vistas interactivas (scorecards, gráficos de evolución temporal, comparativas entre runs). La interfaz se construye sobre shadcn/ui con Radix primitives y Tailwind CSS; la gestión de estado y datos utiliza Zustand y TanStack Query.

Fase 6: Agente GEOOptimizer (Semanas 12–13). Construcción del agente GEOOptimizer basado en Claude Sonnet 4.6, que analiza las queries en las que el sitio objetivo no es citado y el contenido de los competidores que sí son citados, y genera hasta 8 recomendaciones GEO priorizadas y organizadas por palancas de optimización (detalladas en el Capítulo 4), cada una con evidencia concreta, impacto estimado y esfuerzo de implementación. Esta fase se apoya en los datos que la infraestructura de evaluación en vivo, construida en la Fase 4, ha venido acumulando semanalmente desde la Semana 9, lo que permite al agente disponer de un histórico real de las respuestas de los motores generativos sobre el que razonar.

Fase 7: Pruebas, validación y cierre (Semanas 13–15). Fase final dedicada a la validación integral del sistema. Comprende la implementación de la suite de tests automatizados (pytest con cobertura para el backend FastAPI, Vitest y ESLint para el frontend React), la ejecución de pruebas de humo (*smoke tests*) sobre el pipeline experimental completo, la revisión final cruzada de la memoria y el etiquetado de la versión estable v1.0.0 del repositorio.

3.3.2. Cronograma y esfuerzo

La Tabla 3.2 presenta las fechas absolutas de cada fase del proyecto, asumiendo semanas de lunes a domingo y un inicio efectivo el 2 de febrero de 2026 (semana S1). La Tabla 3.3 muestra la distribución de las fases a lo largo de las 15 semanas de desarrollo. La Tabla 3.4 desglosa cada fase en sus actividades concretas con la estimación de esfuerzo asociada, y la Tabla 3.5 resume el esfuerzo agregado por fase.

Tabla 3.2: Fechas de inicio y fin de las fases del proyecto.

Fase	Inicio	Fin
0. Estudio e investigación del estado del arte	02/02/2026	15/02/2026
1. Setup e infraestructura	09/02/2026	15/02/2026
2. Pipeline core	16/02/2026	08/03/2026
3. Marco de métricas	09/03/2026	15/03/2026
4. Experimentación	16/03/2026	19/04/2026
5. Dashboard web	06/04/2026	26/04/2026
6. Agente GEOOptimizer	20/04/2026	03/05/2026
7. Pruebas, validación y cierre	27/04/2026	17/05/2026
<i>Defensa estimada</i>		junio 2026

A continuación, la Tabla 3.3 presenta la distribución temporal de las fases a lo largo de las 15 semanas. El símbolo ■ indica actividad principal en una semana dada y ○ actividad secundaria o de cierre. La fila *SEO CI* representa la operación diaria automatizada del workflow `seo_audit.yml`, programado durante la Fase 1, que corre en paralelo durante todo el proyecto sin requerir actividad adicional de desarrollo.

Tabla 3.3: Diagrama de Gantt del proyecto (15 semanas, feb–may 2026).

Fase	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
0. Estudio	■	■	○	○	○	○									
1. Setup		■													
2. Pipeline core			■	■	■										
3. Métricas						■									
4. Experimentación							■	■	■	■	■				
5. Dashboard										■	■	○			
6. GEOOptimizer												■	■		
7. Pruebas y cierre													■	■	■
SEO CI (diario)	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Live CI (semanal)									○	○	○	○	○	○	○

La Tabla 3.4 presenta el *backlog* detallado de actividades de cada fase, con la estimación de esfuerzo en horas. Los totales por fase coinciden con la estimación agregada de la Tabla 3.5.

Tabla 3.4: Backlog detallado de actividades por fase.

Fase	Actividad	Horas
0	Revisión sistemática de literatura GEO/RAG (Aggarwal, Wu, Chen, Tan)	10
	Estudio de RAG, embeddings y arquitecturas de motores generativos	6
	Webinars y <i>workshops</i> del sector (Try Geometrics, etc.)	4
	Definición de alcance y propuesta inicial del trabajo	5
1	Reestructuración del repositorio (notebook → paquetes Python)	8
	Configuración centralizada (<code>experiment_config</code> , <code>queries</code>)	5
	Registro de prompts versionado (<code>src/prompts/registry.py</code>)	3
	Workflow <code>seo_audit.yml</code> (PageSpeed diario) y entorno de CI	7
2	<code>StructuredWebLoader</code> (descarga, limpieza HTML, extracción contenido)	12
	<code>TokenAwareChunker</code> (tiktoken, 256/64 tokens)	8
	RAG Judge agente con <i>tool calling</i> y validación de salida JSON	18
	<code>CitationExtractor</code> y parsing/validación de schema JSON	12
	<code>create_search_tool()</code> (FAISS como herramienta LangChain)	5
3	Implementación de las 8 métricas GEO (Visibilidad, SoM, PAWC, etc.)	12
	Scorecard agregado v2 (Q001–Q100, agregados por categoría)	8
	Tests unitarios del extractor de métricas	5
4	Discovery de competidores (Gemini + Google Search grounding)	10
	Crawling y embedding de los competidores (vectorstore congelado)	12
	Ejecución de runs experimentales (~ 15 runs $\times \sim 2$ h cada uno)	30
	Análisis de scorecards, depuración y comparativa entre runs	10
	<code>collect_geo_live.py</code> (multi-engine: Gemini, ChatGPT, Claude)	10
	Workflow semanal <code>geo_live.yml</code> y persistencia en <code>data/geo/live/</code>	5
5	Backend FastAPI (routers, servicios, normalización scorecard v1/v2)	14
	Frontend React 18 + shadcn/ui + Zustand + TanStack Query	16
	Datos simulados (<code>mock-*.ts</code>) e integración E2E backend–frontend	5
6	Agente <code>GE0Optimizer</code> (Claude Sonnet 4.6, 6 palancas, prompt de optimización)	10
7	Tests automatizados (pytest backend, Vitest + ESLint frontend)	12
	<i>Smoke tests</i> sobre el pipeline experimental completo	5
	Redacción y revisión final cruzada de la memoria	25
	Preparación de la presentación de defensa	8
Total		300

Por último, la Tabla 3.5 resume la estimación de horas de trabajo por cada fase.

Tabla 3.5: Estimación de esfuerzo por fase.

Fase	Nombre	Semanas	Horas estimadas
0	Estudio e investigación del estado del arte	2	25
1	Setup e infraestructura	1	23
2	Pipeline core	3	55
3	Marco de métricas	1	25
4	Experimentación	5	77
5	Dashboard web (FastAPI + React)	2.5	35
6	Agente GEOOptimizer	2	10
7	Pruebas, validación y cierre	3	50
Total		15	300

3.4— Costes y recursos

Uno de los principios de diseño de GEO-Audit ha sido la minimización de costes, tanto por tratarse de un proyecto académico como para demostrar que la auditoría de visibilidad GEO es accesible sin inversiones significativas en infraestructura. La totalidad del proyecto se ha desarrollado y ejecutado con un **coste de infraestructura de 0 euros** y un coste total de APIs estimado inferior a 30 euros.

3.4.1. Infraestructura de cómputo

Toda la infraestructura de cómputo utilizada es gratuita:

- **Embeddings vía API de Google.** Inicialmente se planificó utilizar Kaggle con GPU para generar embeddings localmente con el modelo `intfloat/multilingual-e5-large`. Sin embargo, Kaggle no está diseñado como servidor computacional persistente: los túneles de red (`localtunnel`, `serveo`, `ngrok`) eran inestables y el flujo requería levantar manualmente el servidor remoto en cada ejecución, lo que dificultaba la integración con el pipeline local. Se migró a embeddings vía API de Google (`models/gemini-embedding-001` usando el SDK `google-genai`), dentro del free tier, eliminando la dependencia de GPU y simplificando el pipeline (ADR-016/ADR-017).
- **GitHub Actions.** La integración continua del proyecto se compone de tres *workflows*: `seo_audit.yml` (métricas SEO diarias), `geo_live.yml` (evaluación GEO semanal contra Gemini, ChatGPT y Claude) y `tests.yml` (suite de tests automatizados en cada *pull request* y *push* a `main`), ejecutados en runners gratuitos de Ubuntu proporcionados por GitHub para repositorios públicos. El detalle de cada uno se describe en la Sección 3.5.2.
- **Entorno de desarrollo.** El código se desarrolla y versiona en el repositorio Git del proyecto, sobre una máquina local convencional sin GPU ni hardware acelerador dedicado. Ninguna fase del desarrollo ha requerido infraestructura especializada en el entorno del desarrollador.
- **FAISS.** El vectorstore se almacena como archivos locales (`.faiss` + `.pkl`, típicamente 5–20 MB para ~100 documentos chunkeados) sin necesidad de servicios cloud de bases de datos vectoriales.

3.4.2. Costes de APIs

La Tabla 3.6 detalla los costes estimados de las APIs utilizadas a lo largo del proyecto.

Tabla 3.6: Desglose de costes de APIs.

Componente	Coste estimado	Notas
Discovery (Gemini + Google Search grounding)	0,00 €	Pruebas gratuitas de Google AI Studio
Embeddings (Gemini Embedding API)	0,00 €	Pruebas gratuitas de Google AI Studio
Ejecuciones experimentales (~15 runs × 40 queries)	0,00 €	Gemini 2.5 Flash; cubierto por Google AI Studio
Evaluación en vivo: Gemini	0,00 €	Pruebas gratuitas de Google AI Studio
Evaluación en vivo: ChatGPT (GPT-4o-mini)	~6,00 €	Modelo económico de OpenAI
Evaluación en vivo: Claude Haiku	~7,30 €	Modelo económico de Anthropic
GEOOptimizer (Claude Sonnet 4.6, análisis)	~1,40 €	Pocas ejecuciones de análisis de queries
GEOOptimizer (Claude Haiku 4.5, crítico)	~0,10 €	Una llamada por análisis, modelo económico
GEOOptimizer (Serper API, búsqueda en Google)	0,00 €	Free tier de 2 500 consultas, suficiente para los análisis ejecutados
PageSpeed Insights API	0,00 €	API gratuita de Google
Desarrollo y pruebas diversas	~4,60 €	Llamadas API durante desarrollo
Total estimado	~18-24 €	

El coste total del proyecto, incluyendo todas las APIs, es inferior a 30 €. Este coste reducido es posible gracias a varias decisiones de diseño deliberadas: la elección de Gemini 2.5 Flash como modelo juez por su relación coste-rendimiento (ADR-012), el uso de la API de embeddings de Google (`models/gemini-embedding-001`) dentro del free tier (ADR-016/ADR-017), el empleo de FAISS local en lugar de servicios de vectorstore cloud (ADR-005), y la utilización del free tier de Google Search grounding para el discovery (ADR-010). La migración del discovery desde Claude con web search (~0,42 €/ejecución) a Gemini con Google Search grounding (0,00 €/ejecución) fue, de hecho, motivada parcialmente por la reducción de costes. Los costes se reportan en euros aplicando un tipo de cambio aproximado de 1 USD ≈ 0,92 € sobre las tarifas oficiales de los proveedores.

3.4.3. Software y dependencias

La Tabla 3.7 recoge el stack tecnológico principal del proyecto. Todas las dependencias son software libre o disponen de licencias compatibles con uso académico.

Tabla 3.7: Principales dependencias del proyecto.

Categoría	Tecnología	Versión	Función
Lenguaje	Python	3.11	Lenguaje base del proyecto
Framework LLM	LangChain	0.3.x	Orquestación de pipelines RAG
Framework LLM	LangGraph	0.3.x	Ejecución del agente RAG Judge
SDK Google	google-genai	>=1, <2	Acceso a Gemini API y embeddings (SDK nuevo)
SDK Google	langchain-google-genai	>=2.1, <3	Integración LangChain-Gemini
SDK OpenAI	openai	>=1, <2	Acceso a GPT-4o-mini (evaluación en vivo)
SDK Anthropic	anthropic	>=0.45, <1	Acceso a Claude Haiku 4.5 (evaluación en vivo y pasada crítica del GEOOptimizer) y Claude Sonnet 4.6 (análisis del GEOOptimizer)
Búsqueda web	Serper API	-	Búsqueda en Google para el GEOOptimizer (free tier)
Vectorstore	faiss-cpu	>=1.9, <2	Almacenamiento y búsqueda vectorial
Tokenización	tiktoken	>=0.8, <1	Chunking basado en tokens
Scraping	BeautifulSoup4	>=4, <5	Procesamiento HTML
Backend API	FastAPI	>=0.110, <1	API REST del dashboard
Testing backend	pytest	>=8, <9	Tests del backend FastAPI
Frontend	React 18 + TypeScript + Vite	18.x / 5.x	Interfaz del dashboard
UI components	shadcn/ui + Tailwind CSS	-	Componentes y estilos del dashboard
Estado + datos	Zustand + TanStack Query	-	Gestión de estado y fetching en frontend
Datos	pandas	>=2, <3	Análisis de resultados experimentales
Visualización	matplotlib, seaborn	>=3, >=0.13	Gráficos y visualizaciones

3.5— Gestión del repositorio

3.5.1. Estrategia de ramificación

El repositorio sigue un modelo de ramificación simplificado adaptado a un proyecto individual:

- **main**: rama estable. Contiene exclusivamente código funcional y validado. Las integraciones se realizan mediante Pull Requests desde **dev** con revisión previa.
- **dev**: rama de desarrollo activo, donde se realizan los commits cotidianos.
- **kaggle**: rama auxiliar utilizada para la ejecución remota de los notebooks de generación de embeddings en la plataforma Kaggle, con el contenido ajustado a las particularidades de ese entorno.

3.5.2. Integración continua

El proyecto dispone de tres workflows de GitHub Actions:

- **seo_audit.yml**: ejecuta diariamente a las 08:00 UTC la recolección de métricas SEO mediante la API de PageSpeed Insights. Los resultados se almacenan como archivos JSON en **data/seo/** y se commitean automáticamente al repositorio.
- **geo_live.yml**: ejecuta semanalmente la evaluación en vivo de visibilidad GEO consultando los modelos **gemini-2.5-flash** (Google), **claude-haiku-4-5-20251001** (Anthropic) y **gpt-4o-mini** (OpenAI) con el protocolo de queries definido. Los resultados se almacenan en **data/geo/live/** y se commitean automáticamente.
- **tests.yml**: se dispara en cada *push* y Pull Request contra **main**. Ejecuta la batería de tests del backend con **pytest** (incluyendo cobertura mediante **pytest-cov** sobre los ocho módulos de tests, entre ellos **test_optimizer.py**) y, en el frontend, el linter ESLint, los tests unitarios con Vitest y la verificación del build de producción.

El workflow **tests.yml** verifica la corrección estática y la cobertura de los módulos no generativos. El pipeline experimental completo no se ejecuta en CI debido al coste de las APIs y a la naturaleza no determinista de los LLMs, que hace inviable escribir aserciones fijas sobre las salidas del modelo juez.

3.5.3. Versionado de configuración y prompts

Dos elementos del proyecto requieren un versionado especialmente cuidadoso porque sus cambios invalidan resultados anteriores:

- **Configuración del experimento** (**config/experiment_config.json**, versión 1.1.0). Contiene los parámetros del pipeline experimental: modelos, chunking (256 tokens / 64 overlap), retrieval (**top_k=5**), configuración del crawler y parámetros del RAG Judge. Cualquier cambio en este archivo entre ejecuciones experimentales debe documentarse y justificarse, ya que altera las condiciones del experimento.
- **Consultas experimentales** (**config/queries.json**, versión 2.0.0). Define las 100 consultas del protocolo experimental con su sistema de rotación. El formato v2 asigna un identificador único a cada consulta (Q001–Q100), su categoría (informacional, comparativa, navegacional), la pertenencia al set original de 15 consultas y la asignación a bloques de rotación (**core**, R1–R4).

- **Registro de prompts** (`src/prompts/registry.py`). Cada prompt del sistema tiene una versión semántica y un changelog que registra todas las modificaciones. Esto es esencial porque un cambio en el prompt del RAG Judge altera el comportamiento del simulador y, por tanto, las métricas resultantes.

3.5.4. Organización de datos

Los datos del proyecto se organizan en el directorio `data/` con la siguiente estructura:

- `data/seo/`: Series temporales de métricas SEO recolectadas diariamente por GitHub Actions. Cada archivo JSON contiene métricas de rendimiento, accesibilidad, mejores prácticas y SEO técnico del sitio objetivo en una fecha concreta.
- `data/frozen_vectorstore/`: Vectorstore FAISS congelado con los embeddings de los competidores. Este directorio es inmutable una vez completado el discovery y constituye la base del diseño experimental.
- `data/geo/experimental/run_*`: Resultados de cada ejecución experimental, incluyendo las respuestas generadas por el RAG Judge, las métricas GEO calculadas y los scorecards en formato v2 (con identificadores Q001–Q100 y agregados por categoría).
- `data/geo/live/`: Resultados de la evaluación en vivo generados periódicamente por `geo_live.yml`, con las métricas GEO calculadas a partir de respuestas reales de Gemini, ChatGPT y Claude.

3.6— Gestión de riesgos

La Tabla 3.8 identifica los principales riesgos del proyecto, evaluados en términos de probabilidad e impacto, junto con las estrategias de mitigación adoptadas.

3.6.1. Riesgo crítico: inmutabilidad del modelo de embeddings

Merece mención especial un riesgo de carácter estructural que no se gestiona mediante mitigación probabilística, sino mediante una restricción de diseño absoluta: el modelo de embeddings no puede cambiar entre ejecuciones experimentales. Si se sustituyera `models/gemini-embedding-001` por otro modelo (con distinta dimensión vectorial o métrica de similitud), todos los vectores del vectorstore congelado serían incompatibles con los nuevos embeddings, invalidando el diseño experimental. Esta restricción está documentada en los ADR-003 y ADR-014, y se aplica de forma programática en la configuración (`config/experiment_config.json`), donde el modelo de embeddings está fijado y no admite sobreescritura por variables de entorno.

3.6.2. Plan de contingencia general

Los costes operativos del proyecto son del orden de 18–24 € totales y el grueso del pipeline (modelo juez, embeddings y discovery) opera dentro del *free tier* de Google AI Studio. En el escenario improbable de discontinuación del free tier, el sostenimiento del proyecto sobre el tier de pago de Gemini sigue siendo asumible para el volumen del TFG, y el sistema admite además generar credenciales con una cuenta de Google distinta para reiniciar la cuota. El sistema sigue siendo modular: el modelo juez puede sustituirse editando `config/experiment_config.json` si fuese estrictamente necesario, pero un cambio del modelo de embeddings invalidaría el vectorstore congelado y exigiría reiniciar el experimento desde el descubrimiento de competidores.

Tabla 3.8: Matriz de riesgos del proyecto.

Riesgo	Probabilidad	Impacto	Estrategia de mitigación
Límites de tasa en la API de Gemini	Media	Medio	Retroceso exponencial implementado en el pipeline. Retardo de 2 segundos entre consultas consecutivas. El free tier (1500 búsquedas/día) es suficiente para las 40 consultas por ejecución experimental.
Límites de tasa en la API de embeddings de Google	Baja	Medio	El free tier de la Gemini Embedding API soporta volúmenes muy superiores a los necesarios para el proyecto. En caso de superarse, el pipeline puede pausar y reiniciar con backoff exponencial.
No determinismo en las salidas de LLMs	Alta	Medio	Temperatura fijada en 0.0 para el RAG Judge. La variabilidad residual se documenta entre runs y se asume como ruido del sistema.
Cambios en el contenido de sitios competidores	Baja	Bajo	Impacto nulo por diseño: el vectorstore de competidores se congela tras el discovery (ADR-005, ADR-006). Los competidores no se re-scrapean ni se re-embeben entre ejecuciones experimentales.
Cambios en precios o condiciones de APIs	Baja	Alto	El modelo juez (Gemini), los embeddings (Gemini API) y el discovery operan dentro del free tier. Solo la evaluación en vivo (ChatGPT, Claude) y el GEOOptimizer tienen coste, ambos acotados a menos de 18 € en total.
Cambios en el sitio objetivo (Programamos.es)	Media	Bajo	Cada ejecución experimental parte de un scraping fresco del sitio, por lo que los cambios se capturan automáticamente. La serie temporal de métricas documenta la evolución.

3.7– Seguimiento y cierre

A lo largo del proyecto se ha llevado un seguimiento del esfuerzo real invertido en cada fase y se han mantenido reuniones periódicas con los tutores tras cada hito relevante. La validación funcional se ha realizado mediante artefactos verificables (vectorstore congelado, scorecards, recomendaciones del optimizador y dashboard) y la validación metodológica mediante la trazabilidad de las decisiones arquitectónicas (ADRs) y el versionado de prompts y configuración.

3.7.1. Seguimiento temporal por fase

La Tabla 3.9 contrasta las horas estimadas en la planificación inicial con las horas efectivamente invertidas en cada fase.

Tabla 3.9: Seguimiento del esfuerzo: estimado vs. real por fase.

Fase	Nombre	Estimado (h)	Real (h)	Desviación
0	Estudio e investigación del estado del arte	25	40	+60 %
1	Setup e infraestructura	23	25	+9 %
2	Pipeline core	55	56	+2 %
3	Marco de métricas	25	25	0 %
4	Experimentación	77	77	0 %
5	Dashboard web	35	35	0 %
6	Agente GEOOptimizer	10	10	0 %
7	Pruebas, validación y cierre	50	50	0 %
Total		300	318	+6 %

3.7.2. Comentario de desviaciones

Las dos desviaciones materiales identificadas han sido las siguientes:

- **Extensión transversal de la investigación (Fase 0, +15 h, +60 %).** El núcleo del estado del arte se desarrolló de forma concentrada en las Semanas 1–2, pero la actividad de estudio se prolongó de manera transversal hasta la Semana 6 con lecturas puntuales que dieron lugar a tres decisiones clave: la alineación del chunking con SAGEO Arena (ADR-011), la expansión del banco de queries de 15 a 100 entradas con sistema de rotación (ADR-013) y la migración del descubrimiento a Gemini con *grounding* (ADR-010). Estas horas adicionales de investigación se computan íntegramente en Fase 0 en lugar de inflar las fases técnicas correspondientes; la implementación de cada decisión, en cambio, sí se contabiliza en su fase. La cola se cierra con la asistencia al webinar de Try Geometrics, último hito formal de la investigación antes de la dedicación plena al desarrollo.
- **Migración del modelo de embeddings (Fase 1, +2 h).** La planificación inicial contemplaba ejecutar embeddings con `intfloat/multilingual-e5-large` sobre Kaggle. Tras detectar la inestabilidad del entorno (*tunneling* no fiable y necesidad de levantar manualmente el servidor en cada ejecución), se decidió migrar a la API de Gemini (ADR-016/ADR-017), lo que añadió esfuerzo de implementación a Fase 1 pero eliminó dependencias de GPU y simplificó las fases posteriores.

Las fases 3, 4, 5, 6 y 7 se han ajustado al esfuerzo previsto: los aprendizajes absorbidos durante la cola transversal de la investigación permitieron tomar decisiones de diseño (chunking, queries, *grounding*) con la lectura ya hecha, evitando sobrecostes de exploración en las fases técnicas.

3.7.3. Cierre del proyecto

Al cierre del proyecto, el balance temporal y de validación es el siguiente:

- **Esfuerzo total real:** 318 h frente a las 300 h presupuestadas (+6 % de desviación). La desviación se considera asumible para un proyecto de carácter investigador con componentes experimentales y se justifica principalmente por la cola transversal de la investigación, que absorbió las horas de lectura asociadas a las decisiones arquitectónicas tomadas en las fases técnicas (todas documentadas como ADRs).
- **Validación del alcance:** el alcance comprometido en el Capítulo 1 (objetivos OE1–OE8) se ha cubierto íntegramente. Los entregables de la EDT (Sección 3.2.2) están completos: el repositorio de código y datos es público, la presente memoria documenta

el proceso y los resultados, y la presentación de defensa se elaborará en las semanas siguientes al cierre.

- **Validación de calidad:** los requisitos RC-01 (validación con tutores), RC-02 (tests automatizados en CI) y RC-03 (documentación de decisiones como ADRs) se han cumplido a lo largo de todo el proyecto: se han registrado 16 ADRs efectivamente implementados (numerados ADR-001 a ADR-017, sin el ADR-015 que quedó como diseño no construido) en `docs/DECISIONS.md`, que documentan formalmente las decisiones sobre *chunking*, *vectorstore*, modelo de embeddings, modo del juez RAG, rotación de consultas y separación *discovery/run*, entre otras.
- **Lecciones aprendidas:** las desviaciones registradas refuerzan tres principios para proyectos futuros de naturaleza similar: (i) priorizar infraestructura estable sobre infraestructura gratuita pero frágil (Kaggle → API gestionada); (ii) alinear desde el inicio las decisiones de ingeniería con los *benchmarks* de referencia del estado del arte (*chunking* 256/64); y (iii) invertir tiempo en versionar contratos de datos antes de propagar formatos por el sistema (formato v2 del scorecard).

Diseño del sistema

4.1— Visión general de la arquitectura

El sistema GEO-Audit se compone de cuatro componentes integrados: (1) un pipeline experimental basado en RAG que mide la visibilidad del sitio bajo condiciones controladas; (2) un módulo de evaluación en vivo que consulta motores generativos reales; (3) un agente de optimización (GEOOptimizer) que genera recomendaciones priorizadas a partir de los resultados; y (4) una aplicación web que visualiza las métricas acumuladas. Los dos primeros abordan la medición de visibilidad desde perspectivas complementarias: el experimental en un entorno controlado y reproducible, el live con datos obtenidos directamente de motores generativos reales.

4.1.1. Casos de uso del sistema

La Figura 4.1 resume la funcionalidad del sistema desde el punto de vista de sus actores. El actor principal es el auditor GEO (el investigador), que ejecuta las fases del pipeline y consume los resultados a través del dashboard. El segundo actor es el planificador de integración continua (GitHub Actions), que automatiza los dos casos de uso recurrentes: la evaluación semanal en motores reales y la recolección diaria de métricas SEO. El caso de uso de descubrimiento de competidores se ejecuta una única vez, y los runs experimentales lo incluyen de forma implícita al cargar el vectorstore congelado que produce.

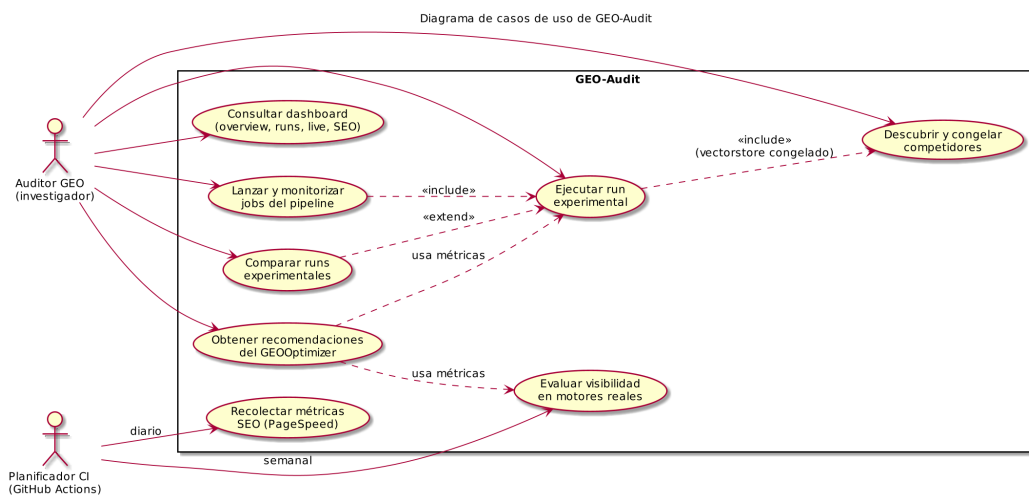


Figura 4.1: Diagrama de casos de uso de GEO-Audit.

4.1.2. Modo experimental

El modo experimental tiene como objetivo aislar la variable independiente, el contenido del sitio web objetivo (Programamos.es), manteniendo constantes todos los demás factores que podrían influir en los resultados. Para ello, se emplea un simulador RAG (*Retrieval-Augmented Generation*) que reproduce el comportamiento de un motor generativo en un entorno controlado.

Los componentes fijos del modo experimental son:

- **Modelo juez:** Gemini 2.5 Flash con `temperature=0.0`, que minimiza la variabilidad de las respuestas ante las mismas entradas. Dado que Gemini 2.5 Flash no expone el parámetro `seed`, persiste una variabilidad residual no eliminable (véase la Sección 6.6.2), por lo que la reproducibilidad es aproximada y no estricta.
- **Vectorstore congelado:** Un índice FAISS construido una única vez con los contenidos de los competidores, que permanece inmutable entre ejecuciones.
- **Conjunto de queries:** 40 consultas por ejecución (20 fijas + 20 rotativas), seleccionadas de un banco de 100.
- **Modelo de embeddings:** `models/gemini-embedding-001` via API de Google (`google-genai` SDK), fijo para todo el experimento.
- **Parámetros de chunking y retrieval:** 256 tokens con 64 de solapamiento, `top_k=5` con similitud coseno.

La única variable que cambia entre ejecuciones es el contenido del sitio objetivo, que se re-scrapea, re-chunkea y re-embedea antes de cada run. Esta rigidez metodológica permite atribuir cualquier variación en las métricas exclusivamente a los cambios realizados en el contenido web.

4.1.3. Modo live

El modo live evalúa la visibilidad del sitio objetivo directamente en motores generativos reales con acceso a la web. A diferencia del simulador, aquí nada está controlado: los motores actualizan sus índices, modifican sus algoritmos y pueden devolver resultados distintos ante la misma consulta en momentos diferentes. Su función no es sustituir al modo experimental sino complementarlo, comprobando si las mejoras observadas en el simulador se reflejan en los motores reales sobre los que el sitio objetivo busca visibilidad.

Los motores evaluados, junto con la herramienta oficial que cada proveedor expone para que el modelo consulte la web durante la generación de la respuesta, son:

- **Gemini:** `gemini-2.5-flash` con *Google Search grounding*.
- **ChatGPT:** `gpt-4o-mini` con la herramienta `web_search` de OpenAI.
- **Claude:** `claude-haiku-4-5` con la herramienta `web_search` de Anthropic.

Los tres motores se consultan con el mismo banco de queries del modo experimental, lo que permite comparar consulta a consulta la presencia del objetivo entre el simulador y la realidad. Los resultados se persisten en `data/geo/live/` mediante el workflow `geo_live.yml` de GitHub Actions, que ejecuta la evaluación semanalmente y produce una serie temporal independiente que incluye además la métrica *Engine Coverage*, no disponible en el modo experimental.

4.1.4. Comparativa de modos

La Tabla 4.1 sintetiza las diferencias fundamentales entre ambos modos operacionales.

Tabla 4.1: Comparativa entre el modo experimental y el modo live.

Dimensión	Modo experimental	Modo live
Motor evaluador	Simulador RAG (Gemini 2.5 Flash)	Motores reales con búsqueda web (Gemini, ChatGPT, Claude)
Índice de competidores	FAISS congelado tras discovery	Índices web reales, no controlados
Variable independiente	Solo el contenido del sitio objetivo	Todo puede cambiar entre ejecuciones
Coste del estudio	≈ 8 € en total (free tier de Google AI Studio prácticamente suficiente)	≈ 14 € en total (Gemini gratis; OpenAI y Anthropic con coste por uso)
Pregunta que responde	Causalidad: ¿el cambio en el contenido mejora la métrica?	Realidad: ¿los motores reales muestran esa mejora?

La complementariedad de ambos modos permite cerrar el ciclo de mejora del sitio objetivo. El modo experimental sirve para ensayar cambios en el contenido bajo condiciones controladas y observar su efecto sobre las métricas (“*si cambio X en mi web, ¿mejora la métrica Y?*”); el modo live permite después comprobar si esas mejoras se reflejan en los motores generativos reales (“*¿esa mejora se traduce en mayor visibilidad en Gemini, ChatGPT o Claude?*”). Cuando ambos modos apuntan en la misma dirección, la evidencia gana solidez; cuando difieren, el contraste aporta información útil sobre el grado en que el simulador anticipa el comportamiento de los motores reales.

4.1.5. Pipeline de métricas SEO

Además de los dos modos de evaluación GEO, el sistema mantiene un tercer pipeline que recolecta métricas de SEO técnico del sitio objetivo, complementarias al diagnóstico de visibilidad generativa. SEO y GEO operan en planos distintos (el primero sobre listas de resultados, el segundo sobre respuestas generadas en lenguaje natural), pero comparten señales (velocidad, accesibilidad, estructura semántica) que pueden influir indirectamente en cómo los motores generativos indexan y citan el contenido.

Los componentes son:

- **Recolector** (`collect_metrics/collect_seo.py`): consulta la API de Google PageSpeed Insights para Programamos.es y obtiene los cuatro grupos estándar de Lighthouse (rendimiento, accesibilidad, mejores prácticas y SEO técnico).
- **Persistencia** (`data/seo/`): cada ejecución produce un fichero JSON con marca temporal, generando una serie histórica sin requerir base de datos.
- **Automatización** (`workflow seo_audit.yml`): GitHub Actions ejecuta el recolector diariamente a las 08:00 UTC y commitea automáticamente los resultados al repositorio.

A diferencia de los pipelines GEO, este corre sin intervención manual desde la Fase 1 del proyecto. La serie temporal acumulada se expone en el dashboard a través del router `seo`, junto a las métricas GEO experimental y live, para ofrecer una visión integrada del estado de visibilidad del sitio.

4.1.6. Organización del código fuente

El sistema se organiza en módulos funcionales bajo el directorio `src/`, cada uno con una responsabilidad claramente delimitada:

Tabla 4.2: Organización del código fuente en módulos.

Módulo	Responsabilidad	Clases principales
<code>src/discovery/</code>	Descubrimiento de competidores via Gemini + Google grounding	<code>CompetitorFinder</code>
<code>src/processing/</code>	Scraping, limpieza HTML, chunking y embeddings	<code>StructuredWebLoader</code> , <code>SiteCrawler</code> , <code>TokenAwareChunker</code> , <code>create_embeddings()</code>
<code>src/rag/</code>	Simulador RAG Judge y extracción de métricas	<code>RAGJudge</code> , <code>CitationExtractor</code> , <code>create_search_tool()</code>
<code>src/optimizer/</code> <code>src/prompts/</code>	Agente de recomendaciones GEO Registro centralizado y versionado de prompts	<code>GEOOptimizer</code> <code>PROMPT_REGISTRY</code>
<code>config/</code>	Configuración del experimento y banco de queries	<code>experiment_config.json</code> , <code>queries.json</code>
<code>scripts/</code>	Orquestación del pipeline completo (ejecución local)	<code>run_discovery.py</code> , <code>run_experimental.py</code>
<code>collect_metrics/</code>	Recolección de métricas SEO y evaluación en vivo	<code>collect_seo.py</code> , <code>collect_geo_live.py</code>
<code>backend/</code>	API REST (FastAPI) del dashboard	Routers: <code>experimental</code> , <code>live</code> , <code>seo</code> , <code>overview</code> , <code>optimizer</code>
<code>frontend/</code>	Interfaz de visualización (React + TypeScript)	Pages: <code>Experimental</code> , <code>Live</code> , <code>Seo</code> , <code>Compare</code> , <code>Optimizer</code>

La configuración centralizada en `src/config.py` proporciona funciones de acceso (`load_experiment_config()`, `get_queries_for_run()`, `get_discovery_queries()`) que abstraen el formato de los ficheros JSON, de modo que los módulos del pipeline consumen siempre estructuras Python ya normalizadas y no dependen de los detalles concretos del banco de queries.

4.1.7. Dashboard de visualización

El dashboard de GEO-Audit es una aplicación web que integra los resultados producidos por los pipelines `experimental` y `live` en una interfaz interactiva. Se reparte en un backend que expone los datos como API REST y un frontend que los consume y visualiza.

Los componentes principales son:

- **Backend (FastAPI):** lee los ficheros JSON de `data/` (no usa base de datos para los datos de investigación, solo una SQLite ligera para el seguimiento de trabajos asíncronos) y los expone mediante routers especializados: `experimental` (scorecards y resultados crudos), `live` (evaluaciones contra motores reales), `seo` (métricas de PageSpeed), `overview` (panel agregado), `optimizer` (recomendaciones del agente GEOOptimizer) y `jobs` (cola de ejecuciones).
- **Frontend (React 18 + Vite):** interfaz tipada en TypeScript construida con `shadcn/ui` sobre Radix y Tailwind CSS; la gestión de estado se apoya en Zustand y

la obtención de datos en TanStack Query.

El dashboard no introduce nuevos cálculos sobre los datos: actúa como capa de presentación. Toda métrica visualizada proviene directamente de los artefactos generados por los pipelines, garantizando que cualquier consulta a la interfaz refleje exactamente los mismos resultados que producen los scorecards en disco.

4.1.8. Vista de componentes y despliegue

La Figura 4.2 sintetiza la arquitectura descrita en las secciones anteriores mediante un diagrama de componentes UML. El elemento central es el almacenamiento en ficheros (**data/**): los componentes del pipeline y los colectores de CI escriben en él, y el dashboard solo lee de él. Esta separación productor/consumidor a través del sistema de ficheros es la que permite que cada parte evolucione y se ejecute de forma independiente.

La Figura 4.3 muestra la vista de despliegue: el desarrollo, los pipelines y el dashboard se ejecutan en la estación de trabajo (devcontainer), y los tres workflows automatizados en runners de GitHub Actions, que persisten sus resultados commiteando al propio repositorio. Los servicios externos (APIs de IA, PageSpeed, SonarCloud y los sitios auditados, tanto el objetivo como los competidores que se scrapean en el discovery) se consumen siempre vía HTTPS.

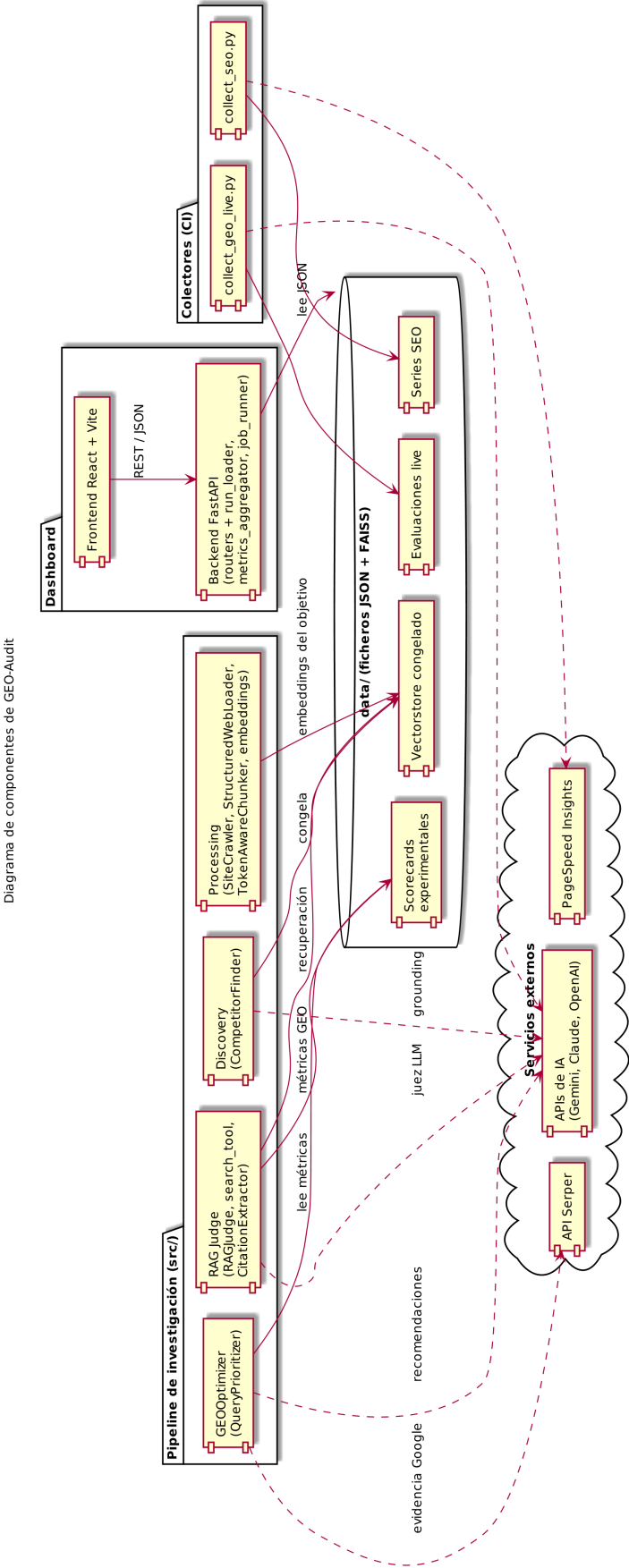


Figura 4.2: Diagrama de componentes UML de GEO-Audit.

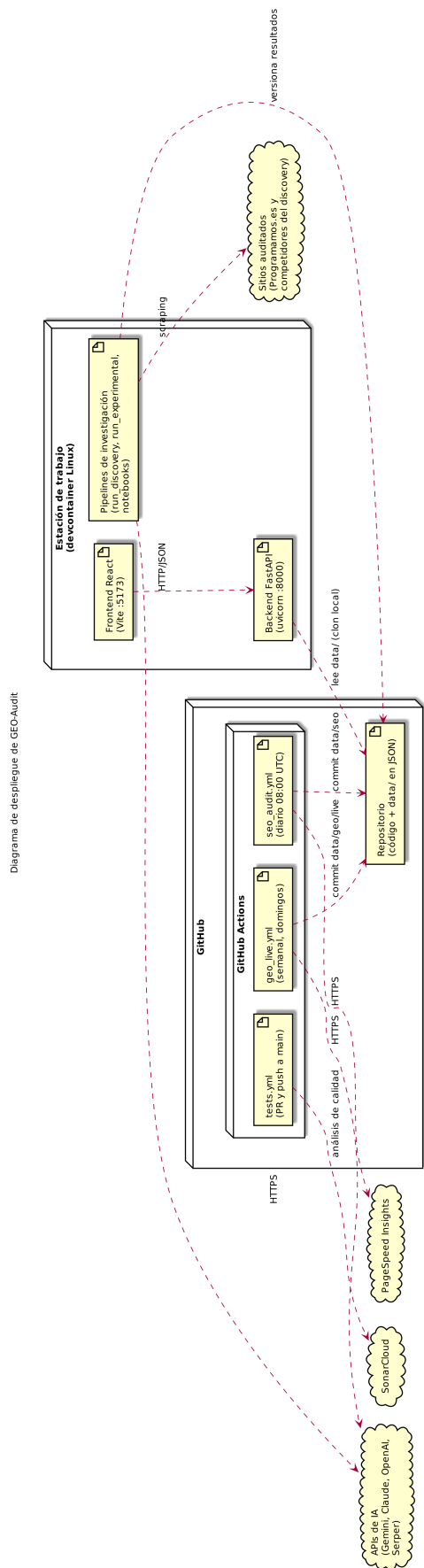


Figura 4.3: Diagrama de despliegue UML de GEO-Audit.

4.2– Diseño experimental

El diseño experimental de GEO-Audit sigue los principios de la experimentación controlada, adaptados al contexto de la evaluación de visibilidad en motores generativos. La estructura se inspira en la metodología de SAGEO Arena (Kim y cols., 2026), el benchmark de referencia para evaluación GEO, adaptada a un estudio de caso único sobre un sitio web real.

4.2.1. Variables del experimento

La identificación y control riguroso de las variables es fundamental para poder establecer relaciones causales entre las intervenciones realizadas en el contenido web y los cambios observados en las métricas de visibilidad. La Tabla 4.3 detalla la clasificación completa de variables.

Tabla 4.3: Variables del diseño experimental de GEO-Audit.

Tipo	Variable	Valor / Rango	Justificación
Independiente	Contenido web de Programamos.es	Puede modificarse entre runs	Única variable que se manipula deliberadamente
Dependientes	Visibilidad (V)	{0, 1}	Presencia binaria en la respuesta
	Share of Model (SoM)	[0, 100] %	Proporción de citas del objetivo
	Ranking PAWC	1..N (ordinal) [0, +∞)	Posición de la primera cita Contenido citado ponderado por posición
	Coverage	[0, 100] %	Cobertura por categoría de query
	Citation Rate	[0, 100] %	Ratio citas / oportunidades de retrieval
	Sentiment Engine Coverage	{POS, NEU, NEG} [0, 100] %	Tono de las menciones Cobertura entre motores (solo modo live)
Controladas	Modelo juez	Gemini 2.5 Flash, temp=0.0	Determinismo en las respuestas
	Queries	40 por run (20 core + 20 rotativas)	Conjunto fijo con rotación sistemática
	Modelo de embeddings	gemini-embedding-001 (API Google)	Mismo espacio vectorial entre runs; cambiarlo invalida el FAISS (ADR-003)
	Chunking Competidores	256 tokens / 64 solapamiento FAISS congelado tras discovery	Alineado con SAGEO Arena Solo el objetivo se re-embedea
	Retrieval	top_k=5, similitud coseno	Parámetros fijos en experiment_config.json
	Prompt del juez	v1.0.0 del registro versionado	Inmutable durante todo el estudio

La variable independiente, el contenido web del sitio objetivo, puede modificarse entre runs para observar el efecto sobre las métricas dependientes. El agente GEOOptimizer (Sección 4.5) genera recomendaciones priorizadas que orientan dichas modificaciones.

Las variables controladas se mantienen fijas mediante dos mecanismos: la congelación de artefactos (vectorstore de competidores, prompts versionados, modelo de embeddings declarado en configuración) y la parametrización centralizada en `experiment_config.json`. Cualquier modificación accidental de una variable controlada invalidaría la serie temporal, por lo que los parámetros relevantes (modelo de embeddings, chunking, `top_k`, modelo juez) se concentran en un único punto de configuración: cambiarlos exige editar deliberadamente

el JSON, lo que deja trazabilidad en el historial de Git. Además, algunas modificaciones quedan bloqueadas implícitamente por el propio diseño: por ejemplo, sustituir el modelo de embeddings por otro de distinta dimensionalidad hace incompatible el vectorstore congelado y obliga a regenerar la fase de discovery completa, tal como recoge ADR-003.

4.2.2. Separación Discovery vs. Runs experimentales

La arquitectura del sistema separa explícitamente dos fases con ciclos de vida distintos, formalizadas en ADR-006. Esta separación es la piedra angular del diseño experimental, ya que garantiza que el conjunto de competidores permanezca estable a lo largo de todo el estudio.

Fase de Discovery (ejecución única)

La fase de discovery se ejecuta una sola vez al inicio del estudio. Su propósito es descubrir qué fuentes web citan los motores generativos cuando se les formulan las consultas del dominio de estudio (programación educativa infantil en España), y construir un vectorstore representativo de esos competidores.

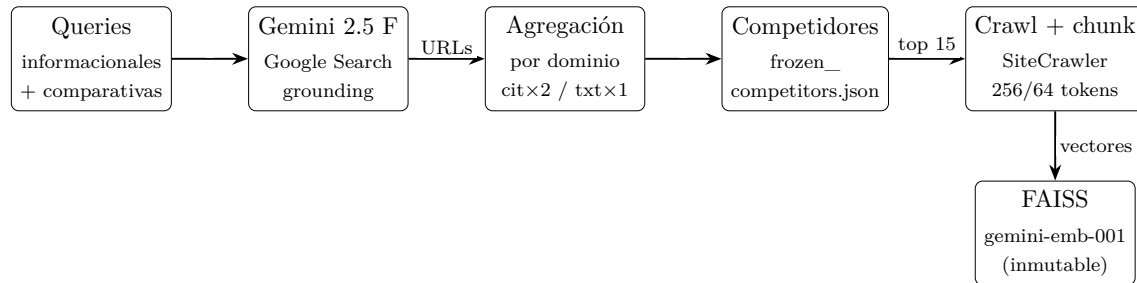


Figura 4.4: Pipeline de Discovery (ejecución única): identifica los competidores y construye el vectorstore FAISS que permanece congelado durante todo el estudio.

Las queries navegacionales se excluyen del discovery (`get_discovery_queries()` filtra la categoría `navegacional`) porque preguntan directamente por el sitio objetivo (por ejemplo, “*qué es Programamos*”), lo cual distorsionaría el ranking de competidores al primar fuentes que mencionan al propio objetivo en lugar de competidores reales.

Análogamente, en la fase de agregación por dominio se descartan los dominios infraestructurales y de información generalista que aparecen con frecuencia en cualquier consulta (Google, YouTube, Wikipedia, redes sociales, GitHub, Amazon, Medium, entre otros) por no ser competidores reales del sitio objetivo. Sin este filtrado, esos dominios coparían el top 15 y desplazarían a las plataformas educativas que sí compiten directamente con Programamos.es. La lista concreta está centralizada en `_EXCLUDED_DOMAINS` dentro de `src/discovery/competitor_finder.py`.

El peso diferenciado entre URLs de citación (`_CITATION_WEIGHT=2`) y URLs extraídas del texto (`_TEXT_WEIGHT=1`) refleja que las primeras provienen del mecanismo de grounding verificado de Google, mientras que las segundas pueden ser meras menciones de pasada.

Fase de Runs experimentales (ejecución N veces)

Cada run experimental sigue una secuencia determinista que solo re-procesa el contenido del sitio objetivo:

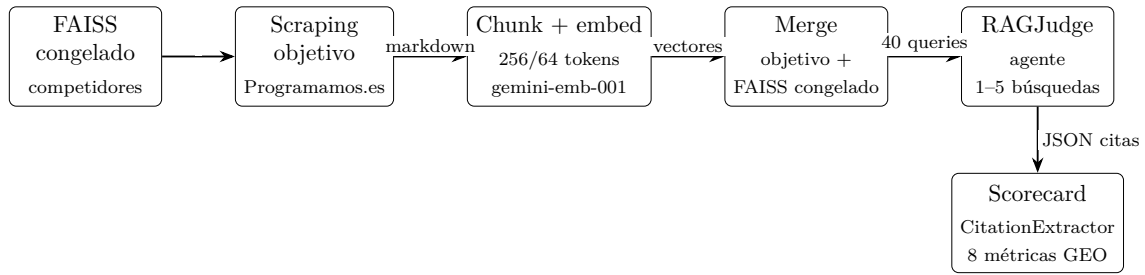


Figura 4.5: Pipeline de un run experimental (ejecución N veces): re-procesa únicamente el contenido del sitio objetivo y reutiliza el vectorstore de competidores congelado.

La separación física entre ambas fases se implementa en dos scripts de automatización bajo `scripts/` (`run_discovery.py` y `run_experimental.py`), que orquestan el pipeline completo desde la máquina local. Los notebooks equivalentes, bajo el directorio `notebooks/`, se mantienen como referencia exploratoria.

4.2.3. Sistema de queries con rotación

El banco de consultas constituye uno de los elementos más críticos del estudio: se utiliza tanto en el modo experimental como en el modo live, de modo que las métricas obtenidas en ambos pipelines son directamente comparables consulta a consulta. Un conjunto excesivamente reducido limitaría la representatividad del estudio; uno excesivamente amplio incrementaría el coste y tiempo de ejecución de cada run hasta hacerlo impracticable.

Composición del banco

El banco contiene 100 queries distribuidas en tres categorías según su intención:

Tabla 4.4: Composición del banco de consultas experimentales.

Categoría	Cantidad	Propósito	Ejemplo
Informacional	35	Consultas de conocimiento general sobre el dominio	<i>“Cómo enseñar programación a niños de primaria”</i>
Comparativa	35	Consultas que comparan opciones o piden recomendaciones	<i>“Mejores plataformas para aprender Scratch en español”</i>
Navegacional	30	Consultas que buscan un recurso o entidad específica	<i>“Programamos cursos de robótica educativa”</i>

Cada query tiene un identificador único (Q001–Q100), su categoría, y un flag `original_15` que indica si pertenece al conjunto original de 15 queries del prototipo.

Mecanismo de rotación

Las 100 queries se organizan en cinco bloques:

- CORE (Q001–Q020): Se ejecutan en todos los runs. Incluyen las 15 queries originales del prototipo más 5 adicionales, garantizando continuidad con la serie temporal existente y comparabilidad longitudinal directa entre cualquier par de runs.
- R1 (Q021–Q040): Bloque rotativo 1.
- R2 (Q041–Q060): Bloque rotativo 2.

- R3 (Q061–Q080): Bloque rotativo 3.
- R4 (Q081–Q100): Bloque rotativo 4.

Cada run ejecuta exactamente 40 queries: las 20 core más las 20 del bloque rotativo correspondiente. La asignación de bloque sigue un ciclo secuencial (R1, R2, R3, R4, R1, ...), de modo que cada cuatro runs se completa una evaluación de las 100 queries.

La función `get_queries_for_run(block)` en `src/config.py` implementa esta lógica:

```

1 def get_queries_for_run(block: Optional[str] = None) -> list[str]:
2     """Devuelve las queries para un run: core 20 + bloque rotativo."""
3     # Core queries (siempre)
4     core_ids = rotation["core"]
5     result = [queries_db[qid]["text"] for qid in core_ids]
6     # Bloque rotativo (opcional)
7     if block is not None:
8         block_ids = rotation[block]
9         result.extend([queries_db[qid]["text"] for qid in block_ids])
10    return result

```

Código 4.1: Selección de queries por bloque rotativo.

Justificación del diseño

La rotación resuelve tres problemas simultáneamente:

1. **Representatividad:** Las 100 queries cubren subtemas amplios del dominio (robótica, STEAM, inteligencia artificial, inclusividad, formación docente, herramientas específicas), evitando que el estudio se limite a un nicho temático.
2. **Comparabilidad longitudinal:** Las 20 queries core son las únicas sobre las que es posible comparar directamente cualquier par de runs. Las queries rotativas amplían la cobertura temática pero, al no aparecer en todos los runs, no permiten comparación longitudinal directa: su utilidad es la amplitud del diagnóstico en cada run, no el seguimiento temporal.
3. **Viabilidad económica y temporal:** Ejecutar 100 queries en cada run requeriría aproximadamente 2.5 horas (considerando los límites de tasa de la API de Gemini: 15 RPM en el tier gratuito), mientras que 40 queries se completan en menos de una hora.

4.3— Marco de métricas GEO

La medición de la visibilidad en motores generativos requiere un conjunto de métricas específicamente diseñadas para este contexto, distinto de las métricas SEO tradicionales. Mientras que el SEO clásico mide posiciones en listas de resultados (*rankings*), GEO debe medir la presencia, prominencia y calidad de las menciones dentro de respuestas generadas en lenguaje natural.

El marco de métricas de GEO-Audit integra y adapta propuestas de la literatura emergente en este campo, principalmente de Aggarwal y cols. (2024), Kim y cols. (2026) y Lüttgenau y cols. (2025).

4.3.1. Definición formal de las métricas

Métrica 1: Visibilidad $V(q)$

La visibilidad es la métrica más fundamental: indica si el sitio objetivo aparece citado en la respuesta generada para una consulta dada.

$$V(q) = \begin{cases} 1 & \text{si el sitio objetivo es citado en la respuesta a } q \\ 0 & \text{en caso contrario} \end{cases} \quad (4.1)$$

- **Rango:** $\{0, 1\}$ (binaria).
- **Interpretación:** Presencia o ausencia del sitio en la respuesta. Es la condición necesaria para que las demás métricas tengan sentido.
- **Referencia:** (Aggarwal y cols., 2024), definida como *impression* en el contexto GEO.

Métrica 2: Share of Model (SoM) $SoM(q)$

El Share of Model cuantifica la proporción de citas de la respuesta que corresponden al sitio objetivo, análogamente al *Share of Voice* en marketing digital.

$$SoM(q) = \frac{|\{c \in \text{citations}(q) : c.\text{url} \in \text{target}\}|}{|\text{citations}(q)|} \times 100 \quad (4.2)$$

- **Rango:** $[0, 100]$ (porcentaje).
- **Interpretación:** Un $SoM = 40\%$ significa que el 40% de las citas de la respuesta provienen del sitio objetivo. Valores altos indican dominancia informativa.
- **Referencia:** (Aggarwal y cols., 2024) como *Source Proportion*.

Métrica 3: Ranking $R(q)$

El ranking indica la posición ordinal de la primera cita del sitio objetivo dentro de la respuesta.

$$R(q) = \min\{i : \text{citation}_i.\text{url} \in \text{target}\} \quad (4.3)$$

- **Rango:** $1..N$ donde N es el número total de citas ($\perp$ si no aparece).
- **Interpretación:** Análogo al ranking SEO: posiciones tempranas reciben más atención del usuario. $R(q) = 1$ es la posición óptima.
- **Referencia:** Adaptación de la noción de *Source Position* usada en GEO-Bench (Aggarwal y cols., 2024).

Métrica 4: Position-Adjusted Word Count (PAWC) $PAWC(q)$

El PAWC combina la cantidad de contenido citado con la posición de cada cita, penalizando las citas que aparecen en posiciones tardías de la respuesta. Está inspirado en el DCG (*Discounted Cumulative Gain*) de la recuperación de información.

$$PAWC(q) = \sum_{\substack{i=1 \\ \text{citation}_i.\text{url} \in \text{target}}}^N \frac{w_i}{\log_2(i+1)} \quad (4.4)$$

donde w_i es el número de palabras de la cita (*quote*) en la posición i .

- **Rango:** $[0, +\infty)$ (valores mayores = mejor).
- **Interpretación:** Captura simultáneamente *cuánto* contenido del objetivo se cita y *dónde* aparece. Una cita extensa en primera posición contribuye mucho más que la misma cita en quinta posición.
- **Referencia:** Propuesta original de (Aggarwal y cols., 2024) en GEO-Bench.

Métrica 5: Coverage $Cov(c)$

La cobertura mide el porcentaje de consultas de una categoría en las que el sitio objetivo es visible.

$$Cov(c) = \frac{|\{q \in c : V(q) = 1\}|}{|c|} \times 100 \quad (4.5)$$

donde c es una categoría de queries (informacional, comparativa o navegacional).

- **Rango:** $[0, 100]$ (porcentaje, por categoría).
- **Interpretación:** Un $Cov(\text{informacional}) = 60\%$ indica que el sitio aparece en el 60% de las consultas informacionales. Permite identificar categorías débiles.
- **Referencia:** Métrica agregada derivada del marco GEO-Bench (Aggarwal y cols., 2024) aplicada por categoría de consulta.

Métrica 6: Citation Rate $CR(q)$

El Citation Rate mide la eficiencia con que el contenido recuperado del sitio objetivo se traduce en citas efectivas en la respuesta.

$$CR(q) = \frac{|\text{target cited}|}{|\text{target retrieved}|} \times 100 \quad (4.6)$$

donde *target cited* son las veces que el objetivo aparece en las citas de la respuesta, y *target retrieved* son las veces que chunks del objetivo fueron recuperados por el sistema de retrieval (tanto usados como no usados). Cuando el objetivo no fue recuperado en absoluto, la métrica no vale 0 sino **None** (no computable): $CR = 0\%$ significa que el contenido llegó al contexto del modelo y fue descartado al generar, mientras que **None** señala un problema previo, de recuperabilidad. La distinción importa porque cada caso exige una intervención distinta, y es la señal que explota el optimizador (Sección 4.5).

- **Rango:** $[0, 100]$ (porcentaje). **None** si el objetivo no fue recuperado.
- **Interpretación:** Un CR bajo indica que el contenido del objetivo se recupera pero no se cita, posiblemente porque no es lo suficientemente relevante, claro o citable para el modelo generativo.
- **Referencia:** (Kim y cols., 2026), como *Citation Efficiency* en SAGEO Arena.

La implementación distingue entre `sources_used` y `sources_available_but_unused` del JSON estructurado del juez, de modo que el denominador incluye también los chunks del objetivo que el agente recuperó pero finalmente descartó al redactar la respuesta.

Métrica 7: Sentiment $S(q)$

El sentimiento clasifica el tono con que el motor generativo menciona al sitio objetivo en su respuesta.

$$S(q) \in \{\text{POSITIVO}, \text{NEUTRO}, \text{NEGATIVO}\}$$

- **Rango:** Categórico (tres niveles).
- **Interpretación:** Un sitio puede ser visible pero mencionado negativamente (“*Prográmanos es una opción, aunque tiene menos contenido que Code.org*”). El sentimiento captura esta dimensión cualitativa.

- **Referencia:** Métrica complementaria propuesta en este trabajo para capturar la dimensión cualitativa de la presencia de marca en respuestas generativas, no recogida por las métricas de visibilidad estándar.

Métrica 8: Engine Coverage $EC(q)$

La cobertura entre motores mide en qué fracción de los motores generativos evaluados el sitio objetivo es visible para una consulta dada.

$$EC(q) = \frac{|\{e \in E : V_e(q) = 1\}|}{|E|} \times 100 \quad (4.7)$$

donde E es el conjunto de motores evaluados.

- **Rango:** $[0, 100]$ (porcentaje).
- **Interpretación:** Un $EC = 75\%$ indica presencia en 3 de 4 motores. Solo disponible en modo live, donde se evalúan múltiples motores reales.
- **Referencia:** Métrica específica del modo *live* de GEO-Audit, que extiende el marco GEO-Bench (Aggarwal y cols., 2024) al evaluar la visibilidad en múltiples motores generativos reales.

4.3.2. Jerarquía de métricas

Las ocho métricas no son independientes entre sí; forman una jerarquía donde las métricas básicas condicionan a las derivadas:

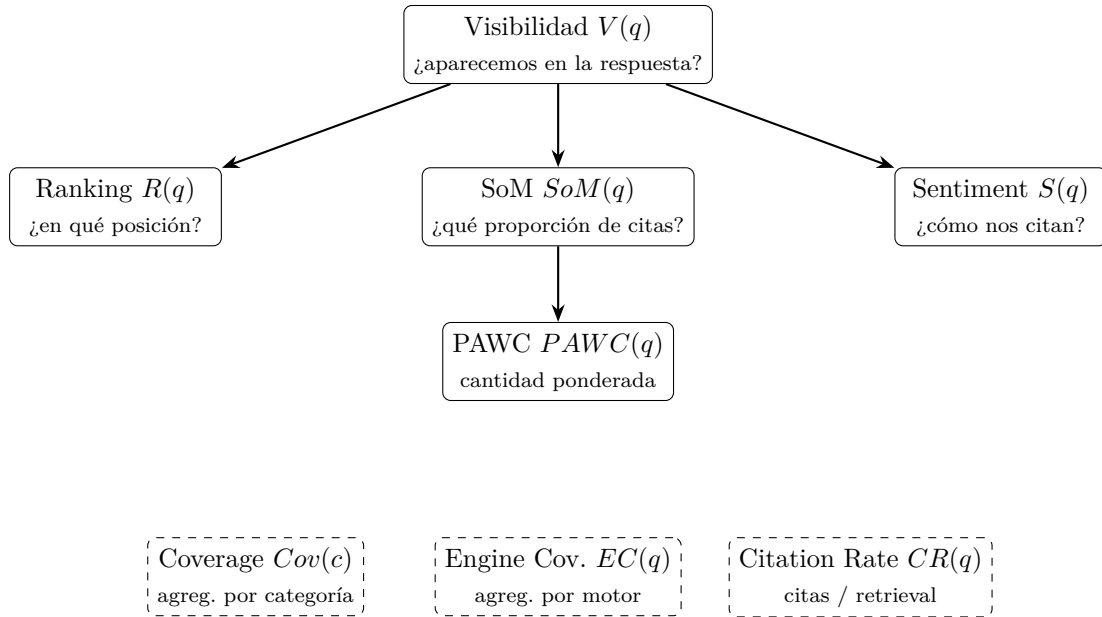


Figura 4.6: Jerarquía de las métricas GEO. Las flechas indican dependencia: si $V(q) = 0$, las métricas hijas carecen de valor informativo para esa consulta. Las cajas en línea discontinua son agregaciones que se calculan sobre $V(q)$ a distintos niveles (categoría, motor o eficiencia de retrieval).

Esta jerarquía implica que si $V(q) = 0$, las métricas de ranking, SoM, PAWC y sentimiento carecen de valor informativo para esa consulta. El Citation Rate, en cambio, puede ser informativo incluso cuando $V(q) = 0$: un $CR = 0\%$ con contenido recuperado indica que el modelo descartó activamente el contenido del objetivo.

4.4– Decisiones arquitectónicas

El diseño del sistema se ha guiado por un conjunto de decisiones arquitectónicas documentadas formalmente como ADRs (*Architecture Decision Records*) en `docs/DECISIONS.md`. Cada ADR registra el contexto, la decisión adoptada, su justificación y las alternativas descartadas. A continuación se resumen las diez decisiones más relevantes para el diseño del sistema.

4.4.1. ADR-003/ADR-016/ADR-017: Modelo de embeddings, evolución a Gemini API

Contexto. El prototipo inicial usaba `OpenAIEmbeddings (text-embedding-3-small)`, con coste recurrente. ADR-003 migró a `intfloat/multilingual-e5-large` ejecutado en GPU de Kaggle para eliminar ese coste. Sin embargo, Kaggle no está diseñado como servidor computacional persistente: los túneles de red (`localtunnel`) eran inestables, dificultando la integración con el pipeline local. ADR-016 sustituyó la GPU remota por `text-embedding-004` (768 dimensiones) via API. Posteriormente, la indisponibilidad práctica de ese modelo a través de `langchain-google-genai` motivó la migración definitiva al modelo de nueva generación.

Decisión. Adoptar `models/gemini-embedding-001` (3072 dimensiones) de Google como modelo de embeddings definitivo, accesible via el SDK `google-genai>=1` dentro del free tier de la API (ADR-017, que reemplaza a ADR-016).

Justificación. (1) Coste cero dentro del free tier, suficiente para el volumen del proyecto. (2) Elimina la dependencia de infraestructura GPU y los problemas de inestabilidad de Kaggle. (3) Soporte multilingüe nativo, alineado con el dominio en español del caso de estudio. (4) Integración directa con el SDK `google-genai>=1` ya utilizado para el discovery y el RAG Judge.

Restricción de diseño. El modelo de embeddings no puede cambiar entre ejecuciones experimentales: cambiar de modelo invalida todos los vectorstores existentes al generar vectores en un espacio dimensional incompatible. Esta restricción se aplica programáticamente en `config/experiment_config.json`.

Alternativa descartada. Mantener `multilingual-e5-large` en Kaggle. Rechazada por inestabilidad operacional. Mantener OpenAI Embeddings. Rechazada por coste recurrente.

4.4.2. ADR-004: Competidores descubiertos desde LLMs reales

Contexto. El prototipo incluía una lista fija de competidores seleccionados manualmente (`code.org`, `scratch.mit.edu`, etc.), introduciendo un sesgo de selección.

Decisión. Descubrir los competidores ejecutando las queries experimentales contra motores generativos reales con acceso a búsqueda web, extrayendo las URLs que estos citan.

Justificación. Los competidores descubiertos son los que realmente aparecen en las respuestas de motores generativos, no los que el investigador presupone. Esto elimina el sesgo de selección y hace que el benchmark refleje la realidad del ecosistema de visibilidad generativa.

Alternativa descartada. Lista manual de competidores. Rechazada por sesgo de selección. También se descartó Tavily (buscador web para agentes) porque busca en la web tradicional, no en motores generativos: lo que aparece en Google no es necesariamente lo que cita ChatGPT.

4.4.3. ADR-005: FAISS local para el vectorstore

Contexto. El vectorstore de competidores debe ser accesible desde cualquier entorno de ejecución (Kaggle, local, CI/CD).

Decisión. Utilizar FAISS local, almacenando el índice como ficheros (.faiss + .pkl, ~5-20 MB) en `data/frozen_vectorstore/`.

Justificación. Sin dependencias externas ni API keys adicionales. Reproducibilidad total: el mismo fichero produce idénticos resultados. El tamaño es insignificante para menos de 10.000 documentos. Accesible directamente al clonar el repositorio.

4.4.4. ADR-006: Separación discovery (una vez) vs. runs (N veces)

Contexto. El prototipo ejecutaba el descubrimiento de competidores en cada run, generando conjuntos de competidores distintos y violando el principio de control experimental.

Decisión. Dos pipelines independientes: discovery (una ejecución) y runs experimentales (N ejecuciones). La única variable que cambia entre runs es el contenido del sitio objetivo.

Justificación. Diseño experimental riguroso. Si los competidores cambiaran entre runs, sería imposible atribuir los cambios en las métricas exclusivamente a las intervenciones en el contenido web.

4.4.5. ADR-007: Procesamiento HTML-aware

Contexto. El prototipo usaba `WebBaseLoader` de LangChain, que extrae todo el texto de la página incluyendo navegación, pie de página, scripts y publicidad, contaminando los chunks con contenido irrelevante.

Decisión. Implementar `StructuredWebLoader`, un procesador HTML propio que: (1) descarga con retry y backoff exponencial; (2) elimina tags de ruido (<nav>, <footer>, <script>, <style>, <aside>); (3) extrae el contenido principal via <main> o <article>, con fallback al div de mayor tamaño; (4) convierte a markdown preservando la estructura jerárquica (h1-h4, listas, párrafos); y (5) extrae metadata (título, meta description, JSON-LD Schema.org).

Justificación. La calidad del RAG depende directamente de la calidad de los chunks. Un chunk que mezcla el menú de navegación con contenido sustantivo degrada la relevancia de la recuperación y, por tanto, la fidelidad de la simulación.

4.4.6. ADR-010: Discovery con Gemini + Google Search grounding

Contexto. ADR-009 empleaba Claude con `web_search` tool para el discovery, con un coste de ~0.45 USD por ejecución. Gemini 2.5 Flash ofrece Google Search grounding de forma gratuita.

Decisión. Migrar el discovery a Gemini 2.5 Flash con Google Search grounding, usando el SDK nuevo `google-genai>=1`.

Justificación. Coste cero frente a ~0.45 USD por ejecución. Google Search grounding utiliza el mismo índice que el buscador de Google, proporcionando URLs verificadas via `grounding_metadata`. Las URLs de citación (respaldando afirmaciones específicas) reciben peso doble frente a las URLs de búsqueda genérica.

Alternativa descartada. Mantener Claude con `web_search`. Rechazado por coste innecesario cuando Gemini ofrece funcionalidad equivalente de forma gratuita.

4.4.7. ADR-011: Chunking 256/64 alineado con SAGEO Arena

Contexto. ADR-001 establecía chunks de 1024 tokens, configuración adoptada por defecto en el prototipo inicial. Sin embargo, los motores generativos reales procesan fragmentos más cortos (~200-377 palabras según análisis de (BrightEdge Research, 2024)), y SAGEO Arena, el benchmark de referencia, emplea 256 tokens con 64 de solapamiento (Kim y cols., 2026).

Decisión. Reducir el chunking a 256 tokens con 64 de solapamiento para todos los perfiles.

Justificación. Alineación directa con el benchmark de referencia. Chunks más cortos proporcionan mayor granularidad en las citaciones, haciendo las métricas GEO más discriminativas. Con `top_k=5`, el contexto total ($5 \times 256 = 1280$ tokens) queda muy por debajo de la ventana de contexto del modelo juez (Gemini 2.5 Flash admite hasta 1 M tokens de entrada), por lo que el coste por consulta es despreciable.

Alternativa descartada. Mantener 1024 tokens. Rechazada por desalineación con el benchmark y menor granularidad métrica. Impacto: Invalidó el vectorstore congelado existente, requiriendo re-ejecución del discovery completo.

4.4.8. ADR-012: RAG Judge como agente con herramienta de búsqueda FAISS

Contexto. El RAG Judge original usaba GPT-4o con chunks pre-recuperados, lo que no reproduce fielmente el comportamiento de un motor generativo real, donde el modelo decide qué buscar y puede reformular las consultas.

Decisión. Migrar a Gemini 2.5 Flash y convertir el juez en un agente LangChain con una herramienta de búsqueda que encapsula el retriever FAISS. El LLM no sabe que busca en FAISS; desde su perspectiva, dispone de un buscador web. Puede realizar entre 1 y 5 búsquedas con queries que él mismo formula (`max_iterations=5`).

Justificación. (1) Coste ~10 veces menor que GPT-4o. (2) Pipeline más realista: el agente reformula queries como lo haría un motor generativo real. (3) El modo clásico (pre-retrieval) permanece disponible como fallback via configuración. (4) `temperature=0.0` maximiza la reproducibilidad.

La herramienta de búsqueda (`create_search_tool()`) formatea los resultados con URL, título y contenido, mimetizando lo que un motor generativo recibiría de un buscador web real.

4.4.9. ADR-013: 100 queries con sistema de rotación

Contexto. El conjunto original de 15 queries era insuficiente para análisis estadísticamente significativos y cubría pocos subtemas del dominio.

Decisión. Expandir a 100 queries (36 informativas + 33 comparativas + 31 navegacionales) con el sistema de rotación descrito en la Sección 4.2.3.

Justificación. Cobertura temática amplia (robótica, STEAM, IA, inclusividad, formación docente), comparabilidad longitudinal via las 20 core, y viabilidad práctica al limitar cada run a 40 queries.

4.5— Agente de optimización: GEOOptimizer

4.5.1. Motivación y posición en el sistema

El GEOOptimizer cierra el ciclo de auditoría: una vez calculadas las métricas GEO y conocida la posición del sitio respecto a sus competidores, el sistema genera recomendaciones accionables que orientan las mejoras de contenido. El agente está implementado en `src/optimizer/geo_optimizer.py` y se basa en Claude Sonnet 4.6 para el análisis y la redacción de las recomendaciones, con Claude Haiku 4.5 como modelo crítico en una pasada de revisión posterior.

A diferencia de los benchmarks existentes (GEO-Bench, SAGEO Arena) que solo miden visibilidad, GEO-Audit incorpora un agente que traduce las métricas en acciones concretas priorizadas.

4.5.2. Entradas del agente

El GEOOptimizer recibe tres tipos de entrada:

1. **Scorecard del run:** métricas agregadas (Visibility Rate, SoM, Ranking, Visibility Rate por categoría) que identifican el estado actual del sitio.
2. **Queries perdidas:** el subconjunto de consultas en las que el sitio no fue citado, que revelan las brechas de visibilidad.
3. **Fragmentos de competidores citados:** el contenido concreto que los competidores incluyen en esas queries y que el modelo juez sí citó, es decir, la evidencia de que estas estrategias funcionan. El agente se apoya además en la API de Serper para dos consultas auxiliares a Google: (i) identificar la URL del propio sitio objetivo que mejor responde a cada query (búsqueda con prefijo `site:programamos.es`), de modo que las recomendaciones aterricen en una página concreta; y (ii) cuando para una consulta no existe evidencia en el RAG experimental, obtener *snippets* de los competidores del set congelado del Discovery. En esta segunda búsqueda, la query se construye restringiéndola a los dominios de `top_competitors` (`frozen_competitors.json`) mediante `site:` concatenados con `OR`, de forma que Google priorice resultados de esos sitios; un filtro posterior sobre las URLs devueltas actúa como salvaguarda. Así solo pasan al agente fragmentos de competidores reconocidos por el experimento, no de fuentes genéricas externas.

4.5.3. Palancas de optimización

Las recomendaciones se organizan en seis palancas, cada una anclada a evidencia de la literatura:

4.5.4. Formato de salida

Para cada recomendación, el agente genera un objeto estructurado con los siguientes campos:

- **Palanca:** categoría de la intervención (una de las seis anteriores).
- **Título:** descripción concisa de la acción recomendada.
- **Evidencia:** fragmento concreto del contenido competidor que demuestra que la estrategia funciona en este dominio.
- **Impacto estimado:** High / Medium / Low.

Tabla 4.5: Palancas de optimización GEO del agente GEOOptimizer.

Palanca	Descripción	Impacto esperado
Citation Readiness	Datos cuantitativos, estadísticas, fragmentos autocontenidos	Hasta +40 % (Aggarwal y cols., 2024)
Authority Injection	Citas a fuentes autoritativas, instituciones, estudios	Moderado (combinación con citas/estadísticas >40 %) (Aggarwal y cols., 2024)
Low Perplexity	Texto fluido, estructura SVO, vocabulario controlado	Hasta +37 % (Aggarwal y cols., 2024)
Machine Scannability	HTML semántico, Schema.org, jerarquía de encabezados	Significativo (Tan y cols., 2025)
Semantic Clarity	Párrafos autocontenidos, topic sentences	Moderado (Aggarwal y cols., 2024)
Caption Injection	FAQ, encabezados descriptivos, listas estructuradas	No cuantificado en literatura

- **Esfuerzo:** estimación en horas de implementación.
- **Acción concreta:** descripción específica de qué añadir o modificar en el sitio objetivo.

El agente genera hasta 8 recomendaciones ordenadas por ratio impacto/esfuerzo. Adicionalmente, produce un prompt de optimización autocontenido que traduce las recomendaciones en instrucciones de implementación directamente ejecutables sin contexto adicional.

Implementación

Este capítulo describe en detalle la implementación del sistema GEO-Audit, recorriendo los componentes de software que conforman la arquitectura presentada en el Capítulo 4. Para cada módulo se presenta su motivación, las decisiones técnicas adoptadas y los fragmentos de código más relevantes del repositorio real del proyecto. El objetivo es proporcionar una visión completa de cómo se tradujeron los requisitos y el diseño arquitectónico en código funcional, destacando las técnicas novedosas empleadas y las restricciones que condicionaron las decisiones de implementación.

El núcleo del sistema (el pipeline de investigación, los scripts de automatización y el backend del dashboard) está implementado en Python 3.11 y combina tres capas de dependencias externas según el componente; la única excepción es el frontend del dashboard, una aplicación React escrita en TypeScript que se describe en la sección 5.10. El pipeline RAG experimental se apoya en el ecosistema LangChain (v0.3.x) para la orquestación del agente, FAISS para el almacenamiento vectorial, y el SDK `google-genai` (v1+) para la comunicación con Gemini 2.5 Flash y el modelo de embeddings `gemini-embedding-001`. El módulo de evaluación en vivo (`collect_metrics/collect_geo_live.py`) consulta directamente tres motores generativos mediante sus respectivos SDKs: `google-genai` para Gemini 2.5 Flash, el SDK `anthropic` para Claude Haiku 4.5, y el SDK `openai` para GPT-4o-mini. El optimizador de recomendaciones GEO (`src/optimizer/`) combina el SDK `anthropic` para generar las sugerencias de mejora mediante Claude con la API de Serper, que aporta resultados reales de Google (descubrimiento de URLs del sitio objetivo y *snippets* de competidores). El código fuente se organiza en los paquetes `src/processing`, `src/discovery`, `src/rag`, `src/optimizer`, `src/prompts` y `src/config`, además de los scripts de automatización en `scripts/` y los notebooks de referencia en `notebooks/`.

5.1– Pipeline de procesamiento de contenido web

El pipeline de procesamiento de contenido web constituye la capa fundamental sobre la que se construye todo el sistema de evaluación GEO. Su función es transformar páginas web arbitrarias en fragmentos de texto normalizados, enriquecidos con metadatos y representados como vectores densos en un espacio de embeddings. Este pipeline comprende cuatro componentes principales que se ejecutan secuencialmente: descarga y limpieza de HTML, rastreo completo de sitios web, fragmentación basada en tokens y generación de embeddings vía API.

5.1.1. StructuredWebLoader

El componente `StructuredWebLoader`, implementado en `src/processing/html_processor.py`, reemplaza el `WebBaseLoader` básico de `LangChain` con un procesador HTML consciente de la estructura semántica de las páginas web. Esta decisión de diseño, documentada en ADR-007, responde a un problema crítico observado durante el prototipado: el `WebBaseLoader` de `LangChain` extrae todo el texto de la página incluyendo elementos de navegación, pie de página, scripts y anuncios, contaminando los chunks con contenido irrelevante que degrada la calidad de la recuperación. Este saneamiento es necesario aunque el LLM podría ignorar ruido durante la generación: el ruido contamina los embeddings en la fase de indexado, antes de que el modelo lo vea, haciendo que FAISS devuelva chunks de navegación o publicidad en lugar de contenido relevante.

El proceso de extracción sigue cinco fases diferenciadas. En primer lugar, la descarga del contenido HTML se realiza con un mecanismo de reintentos con backoff exponencial (Listado G.1, Anexo G).

Este método incorpora una estrategia defensiva frente a errores SSL comunes en sitios educativos con certificados mal configurados: ante un primer fallo SSL, desactiva la verificación de certificados y reintenta, evitando que páginas legítimas pero con problemas de configuración TLS queden excluidas del corpus. El timeout de 10 segundos y un máximo de 3 intentos equilibran la robustez frente a la eficiencia temporal.

La segunda fase consiste en la eliminación de ruido HTML. Se definen dos listas: etiquetas completas a eliminar y selectores CSS para contenedores específicos:

```
1 _NOISE_TAGS = ["nav", "footer", "script", "style", "aside", "header", "noscript"]
2 _NOISE_SELECTORS = [
3     ".ad", ".advertisement", ".sidebar", ".social-share",
4     ".cookie-banner", ".popup", "#comments", ".menu", ".breadcrumb",
5 ]
```

Código 5.1: Listas de etiquetas y selectores de ruido en `StructuredWebLoader`.

La tercera fase extrae el contenido principal de la página, siguiendo una jerarquía de elementos semánticos de HTML5. El algoritmo busca primero etiquetas `<main>` o `<article>`, y si no las encuentra, recurre a una heurística basada en el `<div>` con mayor cantidad de texto:

```
1 def _extract_main_content(self, soup: BeautifulSoup) -> Tag:
2     """Find the main content area of the page."""
3     for semantic in ["main", "article"]:
4         for found in soup.find_all(semantic):
5             if found.get_text(strip=True):
6                 return found
7
8     # Fallback: div with the most paragraph text
9     divs = soup.find_all("div")
10    if divs:
11        return max(divs, key=lambda d: len(d.get_text(strip=True)))
12
13    return soup.body if soup.body else soup
```

Código 5.2: Extracción del contenido principal de la página HTML.

La cuarta fase convierte el árbol HTML limpio a formato Markdown, preservando la estructura de encabezados (h1-h4), listas, enlaces, bloques de cita y fragmentos de código. Esta conversión es fundamental porque los separadores jerárquicos del chunker posterior dependen de las marcas Markdown (`##`, `###`) para identificar límites semánticos entre secciones.

Finalmente, la quinta fase extrae metadatos estructurados: el título de la página (`<title>`), la meta descripción, las etiquetas Open Graph y los datos estructurados JSON-LD de Schema.org. Estos metadatos se almacenan junto al contenido en el objeto `Document` de LangChain y resultan esenciales para la atribución de fuentes en las fases posteriores del pipeline, ya que la URL de origen (`source_url`) y el título se utilizan en la herramienta de búsqueda del agente RAG para identificar las fuentes citadas.

5.1.2. SiteCrawler

El componente `SiteCrawler`, implementado en `src/processing/site_crawler.py`, orquesta el rastreo completo de sitios web, coordinando el descubrimiento de URLs, el filtrado y la priorización antes de invocar a `StructuredWebLoader` para cada página individual. Este componente resulta necesario porque el análisis de visibilidad GEO requiere representar el contenido completo de un sitio en el vectorstore, no únicamente páginas individuales.

El algoritmo de rastreo sigue ocho pasos secuenciales, implementados en el método principal `crawl_site` (Listado G.2, Anexo G).

El primer paso consulta el archivo `robots.txt` del sitio para respetar las directivas de rastreo del propietario, un requisito ético fundamental en web scraping académico. La clase auxiliar `RobotsTxtChecker` encapsula esta funcionalidad utilizando el módulo `urllib.robotparser` de la biblioteca estándar de Python y extrae adicionalmente las directivas `Sitemap`: que muchos sitios incluyen en su `robots.txt`.

El descubrimiento de URLs se realiza mediante el parseo de sitemaps XML, incluyendo soporte para archivos de índice de sitemaps (sitemap index) que referencian múltiples sitemaps hijos. La clase `SitemapParser` maneja ambos formatos y aplica un fallback sin namespace XML para compatibilidad con sitemaps que no declaran el namespace estándar de sitemaps.org.

Un aspecto diferenciador del crawler es la integración de URLs descubiertas por el proceso de discovery con Gemini (paso 3). Las URLs que Gemini cita en sus respuestas con grounding se incorporan al conjunto de URLs a rastrear con la etiqueta `source="discovered"`, lo que permite capturar páginas específicas que los motores generativos consideran relevantes incluso si no aparecen en el sitemap del sitio.

El mecanismo de fallback BFS (Breadth-First Search) se activa únicamente cuando no se encuentra ningún sitemap, ejecutando un rastreo en anchura limitado a una profundidad máxima de 2 niveles, siguiendo exclusivamente enlaces internos del mismo dominio.

La priorización de URLs soporta dos modos configurables:

- `pages_first`: ordena las páginas estáticas (sin patrón de fecha `/20xx/` en la URL) antes que las entradas de blog, priorizando las páginas institucionales que suelen contener información más estable y representativa del sitio.
- `discovered_urls_first`: prioriza las URLs descubiertas por Gemini, seguidas de las de sitemap, y finalmente las de BFS.

Para el sitio objetivo, que se scrapea sin límite de páginas, el orden de priorización es irrelevante. Para los competidores (limitados a 100 páginas) se utiliza `discovered_urls_first`, garantizando que las URLs que Gemini citó explícitamente tengan prioridad sobre las descubiertas por sitemap o BFS.

El límite de páginas (`max_pages`) se aplica de forma diferenciada: sin límite para el sitio objetivo (para capturar todo su contenido) y con un máximo configurable (100 por defecto) para los competidores, equilibrando la representatividad del corpus con el coste computacional del embedding.

5.1.3. TokenAwareChunker

El componente `TokenAwareChunker`, implementado en `src/processing/chunker.py`, reemplaza la fragmentación basada en caracteres del prototipo por una fragmentación basada en tokens, alineada con el benchmark SAGEO Arena (ADR-011). Esta decisión responde a una observación fundamental: los modelos de lenguaje procesan tokens, no caracteres, y un fragmento de 1000 caracteres puede contener entre 200 y 400 tokens dependiendo del idioma y la densidad léxica del texto.

La implementación reutiliza el `RecursiveCharacterTextSplitter` de LangChain pero sustituye la función de medición predeterminada (`len()`, que cuenta caracteres) por `_token_length()`, que utiliza el tokenizador `cl100k_base` de la biblioteca tiktoken. Este tokenizador, un codificador Byte Pair Encoding (BPE) con vocabulario de ~100 000 tokens, de ahí el nombre, es el empleado por la familia de modelos GPT-4 y resulta una aproximación razonable para estimar la longitud en tokens de textos que serán procesados por distintos LLMs.

Los separadores se definen en orden jerárquico descendente de especificidad:

```
1 _DEFAULT_SEPARATORS = ["\n## ", "\n### ", "\n\n", "\n", ". ", " "]
```

Código 5.3: Separadores jerárquicos del `TokenAwareChunker`.

Esta jerarquía garantiza que el chunker intente primero cortar en límites de secciones Markdown (encabezados de nivel 2 y 3), luego en separaciones de párrafos, a continuación en saltos de línea, después en límites oracionales y solo como último recurso en espacios entre palabras. De este modo, los chunks resultantes tienden a ser unidades semánticamente coherentes que respetan la estructura documental original.

La configuración de 256 tokens con 64 tokens de solapamiento (overlap) se alinea con el benchmark SAGEO Arena, la referencia de evaluación GEO más relevante en el momento de desarrollo. Como se argumentó en ADR-011, los motores generativos reales procesan ventanas de texto cortas (entre 200 y 377 palabras según análisis de BrightEdge Research (2024); cifra expresada en palabras, la unidad natural de los estudios de legibilidad: con `cl100k_base` los 256 tokens del chunker equivalen aproximadamente a 180-200 palabras en español, alineándose con el límite inferior de ese rango), y esta configuración con `top_k=5` produce un contexto total de aproximadamente 1280 tokens, bien dentro del presupuesto de contexto de cualquier LLM moderno.

Cada chunk generado se enriquece con tres metadatos adicionales: su índice ordinal dentro del documento original (`chunk_index`), el perfil de contenido utilizado (`content_type`) y el recuento real de tokens (`chunk_tokens`). Estos metadatos facilitan la trazabilidad y el análisis posterior de la calidad de la fragmentación.

5.1.4. Embeddings via API de Google

La generación de embeddings utiliza la API de Google (`models/gemini-embedding-001`) a través del SDK `google-genai`, eliminando cualquier dependencia de infraestructura GPU local. La clase `GoogleGenAIEmbeddings`, implementada en `src/processing/embeddings.py`, implementa la interfaz `Embeddings` de LangChain Core (Listado G.4, Anexo G).

El diseño incorpora tres decisiones relevantes. Primero, la diferenciación de `task_type` entre documentos (`RETRIEVAL_DOCUMENT`) y consultas (`RETRIEVAL_QUERY`): durante el fine-tuning, Google entrenó el modelo con dos objetivos distintos: con `RETRIEVAL_DOCUMENT` el modelo aprende a producir vectores que representen contenido que podría ser buscado (optimizado para ser recuperado); con `RETRIEVAL_QUERY` aprende a producir vectores que representen la intención de búsqueda (optimizado para encontrar documentos relevantes). Esta arquitectura bi-encoder asimétrica mejora la precisión de la similitud coseno en FAISS

porque las queries y los documentos tienen patrones lingüísticos distintos (una pregunta corta frente a un fragmento de texto largo). Segundo, el batching en lotes de 50 textos con una pausa de 1.5 segundos entre lotes mantiene el uso por debajo de la cuota del free tier del modelo de embeddings (3000 peticiones/minuto). Tercero, el backoff exponencial ante errores de red (hasta 5 reintentos, empezando en 5 segundos) tolera interrupciones transitorias de conectividad.

La función factoría `create_embeddings()` encapsula la instanciación y es el único punto de entrada al subsistema de embeddings desde el resto del pipeline. La única credencial requerida es `GOOGLE_API_KEY` en el fichero `.env`, la misma clave utilizada para el discovery y el RAG Judge, lo que simplifica la configuración del entorno respecto a la arquitectura anterior basada en servidor Kaggle.

5.2– Descubrimiento de competidores

El proceso de descubrimiento de competidores constituye la primera fase del pipeline experimental y tiene como objetivo identificar automáticamente qué sitios web son citados por los motores generativos de IA cuando se les formulan preguntas relacionadas con el dominio temático del sitio objetivo. A diferencia de un análisis SEO tradicional donde los competidores se identifican manualmente o mediante herramientas que analizan las páginas de resultados de los buscadores, en GEO los competidores son aquellos sitios que los modelos de lenguaje citan como fuentes en sus respuestas. De ahí la decisión de descubrirlos dinámicamente a partir de las citas reales de los motores generativos, en lugar de partir de una lista predefinida (ADR-004).

5.2.1. CompetitorFinder con Google Search grounding

La clase `CompetitorFinder`, implementada en `src/discovery/competitor_finder.py`, utiliza Gemini 2.5 Flash con Google Search grounding (ADR-010) para el descubrimiento de competidores. A diferencia de los enfoques que consultan chatbots sin acceso a web (cuyos resultados pueden ser alucinaciones), esta implementación aprovecha la capacidad de grounding de Gemini para ejecutar búsquedas reales en el índice de Google y anclar las respuestas a fuentes verificables.

El método central `_query_gemini` envuelve cada consulta en un prompt que instruye al modelo para incluir URLs explícitas y enviar la respuesta a través de la herramienta de búsqueda:

```

1 def _query_gemini(self, query, max_retries=5, initial_wait=30.0):
2     from google.genai import types
3
4     response = self._client.models.generate_content(
5         model=self._gemini_model,
6         contents=(
7             "Responde a la siguiente pregunta usando tu herramienta "
8             "de búsqueda. En tu respuesta DEBES incluir las URLs "
9             "completas de cada fuente que utilices, escritas tal cual "
10            "(ejemplo: https://scratch.mit.edu). "
11            "Al final de tu respuesta incluye una seccion "
12            "'Fuentes:' con la lista numerada de todas las URLs "
13            "consultadas. Prioriza fuentes en español.\n\n"
14            f"Pregunta: {query}"
15        ),
16        config=types.GenerateContentConfig(
17            tools=[types.Tool(google_search=types.GoogleSearch())]
18        ),
19    )

```

Código 5.4: Método `_query_gemini` de `CompetitorFinder` con Google Search grounding.

La instrucción de incluir URLs en el texto de la respuesta actúa como fallback: la fuente principal de URLs es el objeto `grounding_metadata` que Gemini adjunta a su respuesta, extraído programáticamente como se describe a continuación. La petición textual de URLs cubre los casos en que esos metadatos estén vacíos o incompletos.

El aspecto clave de la implementación reside en la extracción diferenciada de URLs a partir de los metadatos de grounding que Gemini incluye en su respuesta. El objeto `grounding_metadata` contiene dos estructuras complementarias (Listado G.5, Anexo G).

Los `grounding_chunks` representan todas las fuentes web que la herramienta de búsqueda encontró y proporcionó al modelo como contexto. Los `grounding_supports`, por el contrario, representan las fuentes que el modelo utilizó efectivamente para respaldar afirmaciones concretas en su respuesta. Esta distinción es análoga a la diferencia entre los resultados de una búsqueda y las fuentes realmente citadas en un trabajo académico.

Un detalle técnico importante es la resolución de URLs proxy. Google Search grounding devuelve las URLs a través de dominios intermedios de `vertexaisearch.cloud.google.com`, que actúan como proxy con redirección HTTP. El método `_resolve_proxy_url` resuelve estas URLs a su destino real mediante una petición HTTP HEAD sin seguimiento de redirecciones:

```
1 def _resolve_proxy_url(self, url: str) -> Optional[str]:
2     domain = self._get_domain(url)
3     if domain not in _GROUNDING_PROXY_DOMAINS:
4         return url
5
6     try:
7         resp = _requests.head(url, allow_redirects=False, timeout=5)
8         location = resp.headers.get("Location")
9         if location and location.startswith("http"):
10             return location
11     except Exception as exc:
12         logger.debug("Failed to resolve proxy URL %s: %s", url[:80], exc)
13     return None
```

Código 5.5: Resolución de URLs proxy de Google Search grounding.

El manejo de errores de rate limiting es crítico dado que el tier gratuito de Gemini 2.5 Flash permite 15 peticiones por minuto. El método implementa reintentos con backoff exponencial (espera inicial de 30 segundos, duplicándose hasta 5 reintentos), detectando errores 429 y `RESOURCE_EXHAUSTED` tanto en códigos HTTP como en mensajes de excepción.

5.2.2. Agregación y selección de competidores

Una vez recopiladas las URLs de todas las consultas de descubrimiento, el método `_aggregate_by_domain` implementa un sistema de puntuación ponderada por dominio:

```
1 def _aggregate_by_domain(self, per_query):
2     domains = defaultdict(
3         lambda: {"score": 0, "queries": [], "urls": set(), "citation_count": 0}
4     )
5
6     for query, data in per_query.items():
7         gemini_data = data.get("gemini", {})
8         citation_urls = set(gemini_data.get("citation_urls", []))
9         all_urls = gemini_data.get("urls_cited", [])
```

```

10
11     domain_best_weight = {}
12     for url in all_urls:
13         domain = self._get_domain(url)
14         weight = _CITATION_WEIGHT if url in citation_urls else _TEXT_WEIGHT
15         domain_best_weight[domain] = max(
16             domain_best_weight.get(domain, 0), weight
17         )
18
19     for domain, weight in domain_best_weight.items():
20         domains[domain]["score"] += weight

```

Código 5.6: Agregación de URLs competidoras por dominio con puntuación ponderada.

El sistema de puntuación asigna 2 puntos a las URLs que aparecen en **grounding_supports** (citation URLs, fuentes que respaldan afirmaciones concretas) y 1 punto a las que solo aparecen en el texto de la respuesta o en **grounding_chunks** sin respaldo específico. Un dominio solo puede puntuar una vez por consulta, aplicándose el peso máximo de entre todas sus URLs en esa consulta. Esta restricción evita que un dominio con múltiples páginas en una misma consulta infle artificialmente su puntuación. Por ejemplo, si el dominio X tiene tres páginas citadas en la misma consulta, su puntuación para esa consulta es 2 (el máximo entre las tres), no 6; pero si aparece como citation URL en cinco consultas distintas, acumula 10 puntos, superando a un dominio presente como mero chunk en diez consultas diferentes.

La selección final se ordena por puntuación descendente y se limita a los 15 dominios principales. Se excluyen dominios que no representan competidores reales: plataformas sociales (YouTube, Facebook, LinkedIn), sitios generalistas (Wikipedia, Amazon), y dominios de infraestructura del propio Google. El dominio objetivo (programamos.es) también se excluye, ya que su presencia en las respuestas de Gemini sería circular.

5.3— RAG Judge

El RAG Judge constituye el componente central del modo experimental: simula un motor generativo de IA que responde a preguntas utilizando fuentes web recuperadas de un índice vectorial. Su implementación como agente con herramienta de búsqueda, documentada en ADR-012, representa una simulación más realista que el enfoque clásico de pre-recuperación, ya que el modelo decide autónomamente qué buscar, puede reformular consultas y realizar múltiples búsquedas antes de sintetizar una respuesta.

5.3.1. Herramienta de búsqueda FAISS

La pieza clave que habilita el modo agente es la encapsulación del retriever FAISS como una herramienta de LangChain. La función `create_search_tool`, implementada en `src/rag/search_tool.py`, transforma un vectorstore FAISS en una herramienta invocable por el agente LLM (Listado G.6, Anexo G).

El diseño de esta herramienta es deliberadamente simple pero efectivo. El decorador `@tool` de LangChain registra la función con su *docstring* como descripción, que el agente LLM utiliza para decidir cuándo y cómo invocarla. Desde la perspectiva del agente, esta es una herramienta de “búsqueda en páginas web”; el agente desconoce que internamente busca en un índice FAISS local. Esta abstracción es precisamente lo que permite simular del mejor modo posible el comportamiento de un motor generativo: el LLM decide qué buscar, formula la consulta y recibe resultados formateados con URL, título y contenido, tal como ocurriría con una herramienta de búsqueda web real.

El formato de respuesta incluye la URL de origen, el título de la página y el contenido del chunk, separados por líneas horizontales. Este formato permite al agente LLM identificar y citar las fuentes correctamente en su respuesta JSON final.

5.3.2. Agente RAG con Gemini

La clase `RAGJudge`, implementada en `src/rag/judge.py`, orquesta la generación de respuestas mediante un agente de `LangChain` con el modelo Gemini 2.5 Flash. El método principal `generate_answer_with_agent` crea un agente *tool-calling* que puede invocar la herramienta de búsqueda entre 1 y 5 veces antes de producir una respuesta (Listado G.7, Anexo G).

El flujo de ejecución del agente es el siguiente. El `AgentExecutor` invoca al LLM con el prompt del sistema y la pregunta del usuario. El LLM analiza la pregunta y decide invocar la herramienta `search` con una consulta que él mismo formula (que puede ser la pregunta original o una reformulación). Los resultados de la búsqueda se incorporan al contexto del agente (*scratchpad*). El LLM puede entonces decidir realizar búsquedas adicionales (hasta el límite de 5 iteraciones) o generar la respuesta final en formato JSON.

La inicialización del LLM instancia `ChatGoogleGenerativeAI` con los parámetros del fichero de configuración:

```
1 def _init_llm(self, rag_config):
2     model = rag_config["model"]
3     from langchain_google_genai import ChatGoogleGenerativeAI
4     self.llm = ChatGoogleGenerativeAI(
5         model=model,
6         temperature=rag_config["temperature"],
7         max_output_tokens=rag_config["max_tokens"],
8     )
```

Código 5.7: Método `_init_llm` del `RAGJudge`.

El sistema de prompts utiliza un registro centralizado (`src/prompts/registry.py`) que mantiene las versiones y el *changelog* de cada prompt. El versionado permite correlacionar cambios en las métricas GEO con su causa real: si entre dos runs cambia el prompt y también el contenido del sitio, el *changelog* deja constancia de qué cambió primero, separando el efecto del contenido del efecto del prompt. El prompt del agente (`rag_judge_agent`) instruye al LLM para que se comporte como un “motor de búsqueda generativo”, establece las reglas de citación, el formato JSON esperado y el estilo de respuesta.

5.3.3. Validación y parseo de salida JSON

La extracción de JSON a partir de la salida del agente requiere un parseo robusto, ya que los LLMs pueden envolver el JSON en bloques de código Markdown o incluir texto adicional antes o después. El método `_extract_json` implementa tres estrategias de parseo en cascada (Listado G.8, Anexo G).

Primero intenta parsear el texto completo como JSON puro. Si falla, busca bloques delimitados por triple acento grave (`"'json ... '"`). Como último recurso, localiza la primera llave de apertura y la última de cierre, extrayendo el JSON intermedio. Esta cascada cubre la gran mayoría de formatos de salida observados empíricamente en Gemini 2.5 Flash.

La validación de esquema posterior verifica la presencia de las cuatro claves obligatorias (`answer`, `citations`, `sources_used`, `sources_available_but_unused`) y comprueba que cada citación contenga los campos requeridos (`index`, `url`, `quote`):

```
1 _REQUIRED_KEYS = ["answer", "citations", "sources_used",
```

```

2         "sources_available_but_unused"]
3 _REQUIRED_CITATION_KEYS = ["index", "url", "quote"]
4
5 @staticmethod
6 def _validate_schema(result: dict) -> None:
7     missing = [k for k in _REQUIRED_KEYS if k not in result]
8     if missing:
9         raise ValueError(f"Missing required keys: {missing}")
10
11     for citation in result.get("citations", []):
12         missing_c = [k for k in _REQUIRED_CITATION_KEYS if k not in citation]
13         if missing_c:
14             raise ValueError(f"Citation {citation} missing keys: {missing_c}")

```

Código 5.8: Validación de esquema JSON de la salida del RAGJudge.

Si la validación falla, el mecanismo de reintentos (hasta 3 adicionales) vuelve a ejecutar al agente completo, dando al LLM otra oportunidad de generar una salida conforme al esquema.

El modo agente es el único modo activo en el experimento, configurado mediante el parámetro `mode` con valor `agent` en `config/experiment_config.json`. Sus ventajas respecto a la alternativa de pre-recuperación son: el modelo formula sus propias búsquedas (puede reformularlas si los primeros resultados no son suficientes), puede realizar hasta 5 iteraciones, y decide autónomamente cuándo tiene suficiente información para responder.

5.4– Extracción de métricas GEO

La clase `CitationExtractor`, implementada en `src/rag/citation_extractor.py`, calcula las métricas de visibilidad GEO a partir de la salida JSON estructurada del RAG Judge. Esta clase no tiene dependencias externas más allá de la biblioteca estándar de Python, lo que la hace altamente portable y testeable.

5.4.1. Métricas calculadas

El método principal `extract_metrics` devuelve un diccionario con las siguientes métricas:

- **Visibility** (`is_visible`): booleano que indica si el sitio objetivo aparece en al menos una citación.
- **Share of Model** (`som`): porcentaje de citaciones que corresponden al sitio objetivo respecto al total de citaciones. Calculado como $(\text{target_citations} / \text{total_citations}) * 100$.
- **Ranking** (`first_citation_rank`): posición (1-indexada) de la primera citación del sitio objetivo en la respuesta.
- **PAWC** (`pawc`): Position-Adjusted Word Count, métrica que pondera el recuento de palabras de cada citación del sitio objetivo por su posición en la respuesta.
- **Citation Rate** (`citation_rate`): porcentaje de veces que el sitio fue citado respecto a las veces que fue recuperado.
- **Brand Mentions** (`brand_mentions`): menciones de la marca en el texto de la respuesta y en las citas.

5.4.2. Cálculo de PAWC

La métrica PAWC (Position-Adjusted Word Count) fue introducida por Aggarwal y cols. (2024) y combina la extensión de las citas con su posición en la respuesta. Las citas que aparecen antes tienen mayor peso, siguiendo una función logarítmica inspirada en la métrica DCG (Discounted Cumulative Gain) de la recuperación de información:

```

1 def _calculate_pawc(self, citations):
2     """Position-Adjusted Word Count for target citations.
3
4     PAWC(q) = sum w_i / log2(i + 1) for each target citation at position i.
5     """
6     sorted_cits = sorted(citations, key=lambda c: c.get("index", 0))
7     pawc = 0.0
8     for position, cit in enumerate(sorted_cits, start=1):
9         if self._url_matches_target(cit.get("url", "")):
10             word_count = len(cit.get("quote", "").split())
11             pawc += word_count / math.log2(position + 1)
12     return round(pawc, 2)

```

Código 5.9: Cálculo de PAWC en CitationExtractor.

Para una citación en la posición 1, el divisor es $\log_2(2) = 1,0$, por lo que el recuento de palabras se aplica íntegro. Para una citación en la posición 2, el divisor es $\log_2(3) = 1,585$, reduciendo el peso un 37 %. Para la posición 5, el divisor es $\log_2(6) = 2,585$, reduciendo el peso un 61 %. Este descuento logarítmico refleja la observación de que las citas en posiciones iniciales reciben mayor atención del usuario y tienen mayor impacto en la percepción de autoridad de la fuente. Por requerir el texto literal de cada cita, PAWC solo se calcula en el modo experimental, donde el RAG Judge devuelve ese fragmento exacto en el campo `quote`; los motores reales del modo live exponen las URLs citadas pero no el texto tomado de cada fuente, por lo que allí la métrica no es computable.

5.4.3. Coincidencia de URLs y Citation Rate

La coincidencia de URLs entre las citaciones del RAG Judge y el sitio objetivo requiere un algoritmo de normalización robusto que evite tanto falsos negativos como falsos positivos:

```

1 @staticmethod
2 def _normalize_url(url: str) -> str:
3     """Strip scheme, www. prefix, and trailing slash for comparison."""
4     parsed = urlparse(url)
5     host = parsed.netloc or parsed.path
6     host = re.sub(r"~www\.|", "", host, count=1)
7     path = parsed.path.rstrip("/") if parsed.netloc else ""
8     return (host + path).lower()
9
10 def _url_matches_target(self, url: str) -> bool:
11     """Check whether *url* belongs to the target domain."""
12     normalized = self._normalize_url(url)
13     if not normalized.startswith(self._normalized_target):
14         return False
15     remainder = normalized[len(self._normalized_target):]
16     return remainder == "" or remainder[0] in ("/", "?", "#")

```

Código 5.10: Normalización y coincidencia de URLs en CitationExtractor.

La normalización elimina el esquema (`https://`), el prefijo `www.` y la barra final, convirtiendo todo a minúsculas. Crucialmente, la comprobación de coincidencia verifica que

tras la cadena del dominio objetivo no haya caracteres adicionales que no sean separadores de ruta, consulta o fragmento. Esto previene falsos positivos como `programamos.es.evill.com`, que empieza con `programamos.es` pero pertenece a un dominio completamente diferente.

El Citation Rate se calcula como la proporción de veces que el sitio objetivo fue citado respecto a las veces que fue recuperado (presente en `sources_used` o `sources_available_but_unused`):

```

1 def _calculate_citation_rate(self, judge_output):
2     all_sources = judge_output.get("sources_used", []) + judge_output.get(
3         "sources_available_but_unused", []
4     )
5     target_retrieved = sum(
6         1 for u in all_sources if self._url_matches_target(u)
7     )
8     if target_retrieved == 0:
9         return None
10    target_cited = self._count_target_citations(
11        judge_output.get("citations", [])
12    )
13    return round((target_cited / target_retrieved) * 100, 2)

```

Código 5.11: Cálculo del Citation Rate en CitationExtractor.

Cuando el sitio objetivo no fue recuperado en absoluto, la función devuelve `None` en lugar de 0, distinguiendo entre “no fue encontrado” y “fue encontrado pero no citado”. Esta distinción orienta la optimización GEO, porque cada caso exige una intervención distinta: un resultado `None` (no recuperado) señala un problema de recuperabilidad: el contenido no es localizable para esa consulta y debe reestructurarse para acercarlo semánticamente a ella. En cambio, un Citation Rate de 0% (recuperado pero no citado) señala un problema de citabilidad: el contenido está disponible pero no resulta lo bastante convincente para ser citado. El agente GEOOptimizer explota precisamente esta señal: las consultas con *gap de recuperación* disparan recomendaciones de reescritura del contenido para aumentar su similitud con la consulta.

5.4.4. Detección de menciones de marca

La detección de menciones de marca busca el nombre del sitio objetivo (por ejemplo, “Programamos”) tanto en el texto de la respuesta como en las citas textuales, proporcionando una ventana de contexto de 50 caracteres alrededor de cada mención (Listado G.9, Anexo G).

Esta ventana de contexto de 50 caracteres alrededor de cada mención es la entrada del analizador de sentimiento, que clasifica el tono con que el motor presenta la marca como positivo, neutro o negativo. Dicha clasificación, la métrica *Sentiment*, solo se calcula en el modo de evaluación en vivo; en el modo experimental las menciones se detectan y se registran junto a su contexto, pero no se clasifican. La ventana facilita además una revisión cualitativa rápida del tono sin necesidad de consultar la respuesta completa.

5.5— Evaluación en vivo sobre motores reales

El pipeline experimental basado en RAG es un simulador controlado: mide la *dirección* del cambio en visibilidad bajo condiciones congeladas, pero no captura el comportamiento real de los motores generativos comerciales. El módulo de evaluación en vivo, implementado en `collect_metrics/collect_geo_live.py`, complementa al simulador consultando directamente a tres motores reales con búsqueda web activada y midiendo si el sitio objetivo es citado en sus respuestas. La clase central, `LiveEvaluator`, reproduce el banco de consultas del modo experimental sobre los motores que los usuarios emplean en la práctica.

5.5.1. Consulta a tres motores con búsqueda web

Cada motor se consulta mediante su propia herramienta nativa de búsqueda web, lo que garantiza que las respuestas estén ancladas a fuentes reales y no a conocimiento paramétrico del modelo:

- **Gemini 2.5 Flash** con Google Search grounding, reutilizando la misma extracción de `grounding_metadata` y resolución de URLs proxy descrita en la sección de descubrimiento.
- **Claude Haiku 4.5** (`claude-haiku-4-5`) con la herramienta `web_search_20250305`, extrayendo las URLs de los bloques `web_search_tool_result` de la respuesta.
- **GPT-4o-mini** a través de la *Responses API* de OpenAI con la herramienta `web_search_preview`, recuperando las URLs de las anotaciones `url_citation`.

El adaptador de cada motor convierte su respuesta nativa a un esquema JSON común, idéntico al que produce el RAG Judge (Listado G.10, Anexo G).

La decisión de diseño clave es esta normalización: al devolver los tres adaptadores el mismo esquema (`answer`, `citations`, `sources_used`, `sources_available_but_unused`), el módulo reutiliza sin modificaciones el mismo `CitationExtractor` del pipeline experimental, lo que garantiza que las métricas se calculan de forma idéntica en ambos modos y, por tanto, son comparables.

5.5.2. Métricas y operativa del modo live

No todas las métricas del modo experimental tienen sentido en el modo live. Como los motores reales no exponen el texto exacto de cada cita ni distinguen una etapa de recuperación previa, se descartan PAWC y Citation Rate. A cambio, se añaden dos métricas específicas: *Sentiment*, que clasifica el tono de las menciones de marca mediante un analizador basado en `gemini-2.0-flash` (positivo, neutro o negativo), y *Engine Coverage*, que mide en qué proporción de los motores evaluados resulta visible el sitio para cada consulta:

```

1 judge_output = method(q["text"]) # adaptador segun el motor
2 metrics = self._extractor.extract_metrics(judge_output)
3
4 # Metricas que no aplican en modo live (sin quote ni retrieval)
5 metrics.pop("pawc", None)
6 metrics.pop("citation_rate", None)
7
8 # Sentiment: solo si el sitio es visible y hay menciones de marca
9 if metrics.get("is_visible"):
10     metrics["sentiment"] = self._sentiment.classify(
11         metrics.get("brand_mentions", [])
12     )
13
14 engine_results[engine] = metrics
15
16 # Engine Coverage: motores que citan / motores que evaluaron la query
17 n_visible = sum(1 for e in engines_for_query
18                 if engine_results.get(e, {}).get("is_visible"))
19 engine_coverage = round(n_visible / len(engines_for_query) * 100, 2)

```

Código 5.12: Adaptación de métricas y cálculo de Engine Coverage por consulta.

La evaluación admite distintos volúmenes de consultas mediante un sistema de *tiers* (core: 20; light: 40; medium: 60; full: 100), configurable de forma independiente por motor para equilibrar coste y cobertura. Cada ejecución se identifica con la semana ISO (`run_id = LIVE-AAAA-Wnn`) y se persiste en `data/geo/live/`, generando una serie temporal. Además, el módulo compara cada ejecución con la de la semana anterior, clasificando cada consulta como ganada, perdida o estable por motor, lo que permite seguir la evolución de la visibilidad a lo largo del tiempo. Esta evaluación se ejecuta de forma desatendida mediante el workflow `geo_live.yml` de GitHub Actions descrito en el Capítulo 4.

5.6— Agente de optimización GEOOptimizer

El agente GEOOptimizer, implementado en `src/optimizer/geo_optimizer.py`, cierra el ciclo del sistema: a partir de las consultas en las que el sitio objetivo pierde visibilidad, genera recomendaciones de optimización priorizadas, ancladas a evidencia concreta de competidores reales, y un prompt de optimización autocontenido para aplicarlas sobre el contenido del sitio. Combina la API de Serper para recopilar evidencia, con Claude para el análisis y la redacción de las recomendaciones.

5.6.1. Recopilación de evidencia con Serper

Antes de invocar al modelo, el agente realiza dos búsquedas en Google en paralelo (`asyncio.gather`). La primera, restringida al dominio objetivo (`site:programamos.es`), localiza las páginas propias relevantes a cada consulta; comparando estas URLs con las que el RAG Judge recuperó, el agente detecta un gap de recuperación (páginas que aparecen en la búsqueda real pero que el simulador nunca recuperó). La segunda búsqueda obtiene fragmentos de competidores, pero restringida explícitamente a los dominios del conjunto congelado de competidores del descubrimiento (Listado G.11, Anexo G).

La restricción `site:` en la consulta y el filtro posterior `_url_is_known_competitor`, que compara el dominio contra la lista `top_competitors` del descubrimiento, garantizan que los fragmentos provienen de los competidores GEO reales del estudio (los que los motores generativos citan), y no de cualquier sitio bien posicionado en Google. Sin esta restricción, el agente se apoyaría en los sitios que ranken en el buscador (competidores SEO) y acabaría produciendo recomendaciones de SEO genérico en lugar de GEO.

5.6.2. Generación de recomendaciones con Claude

Con la evidencia recopilada, el agente construye dos bloques de contexto: `queries_context` (motivo de la pérdida, fragmentos de competidores y marca de gap de recuperación por consulta) y `pages_context` (el contenido actual de las páginas objetivo, con avisos automáticos de contenido desactualizado o de posible desajuste temático). Ambos se inyectan en el prompt versionado del registro (clave `geo_optimizer`) y se envían a Claude Sonnet 4.6, que se invoca mediante `tool use` para forzar una salida estructurada nativa sin necesidad de parsear texto libre:

```

1 def _call_claude(self, user_message):
2     """Análisis con Claude Sonnet via tool use (salida estructurada)."""
3     response = client.messages.create(
4         model=_CLAUDE_MODEL, # claude-sonnet-4-6
5         max_tokens=self._prompt_config.get("max_tokens", 8192),
6         system=self._prompt_config["system"],
7         messages=[{"role": "user", "content": user_message}],
8         tools=[_CLAUDE_TOOL], # schema geo_optimizer_output
9         tool_choice={"type": "tool", "name": "geo_optimizer_output"},
10    )

```

```

11     for block in response.content:
12         if block.type == "tool_use":
13             return block.input # dict conforme al schema

```

Código 5.13: Llamada a Claude con tool use para salida estructurada.

El esquema de la herramienta obliga a Claude a devolver como máximo ocho recomendaciones. Cada una especifica la palanca GEO aplicada, de un catálogo de seis (*citation readiness*, *authority injection*, *low perplexity*, *machine scannability*, *semantic clarity* y *caption injection*), la ubicación exacta del cambio, el motivo, la evidencia (que debe ser un fragmento literal del contexto), el impacto, las consultas objetivo y el esfuerzo estimado; junto a ellas, un prompt de optimización autocontenido para ejecutar los cambios.

5.6.3. Validación y crítica de las recomendaciones

La salida se valida contra un esquema Pydantic (`GE00OptimizerOutput`); si algún campo no encaja exactamente, una capa de normalización tolerante reconcilia los alias de nombres de campo y los valores de enumeración antes de descartar una recomendación. Finalmente, una pasada de crítica con Claude Haiku 4.5 revisa cada recomendación y descarta aquellas cuya evidencia esté claramente fabricada (afirmaciones sin fuente, datos numéricos inexistentes en el contexto). El versionado del prompt en el registro permite trazar la evolución del agente y correlacionar cambios en las recomendaciones con cambios en sus instrucciones.

5.7— Recolección automatizada de métricas SEO

Aunque el núcleo del proyecto se centra en métricas GEO, el sistema incluye un módulo de recolección automatizada de métricas SEO tradicionales que permite contextualizar los resultados de visibilidad generativa con el rendimiento del sitio en motores de búsqueda convencionales.

El script `collect_metrics/collect_seo.py` consulta la API de Google PageSpeed Insights para obtener métricas de rendimiento, SEO y accesibilidad del sitio objetivo en sus variantes móvil y escritorio:

```

1 def obtener_metricas(estrategia):
2     base_url = "https://www.googleapis.com/pagespeedonline/v5/runPagespeed"
3     params = (f"?url={URL_OBJETIVO}&strategy={estrategia}"
4               f"&category=performance&category=seo&category=accessibility"
5               f"&key={API_KEY}")
6     url_api = base_url + params
7
8     response = requests.get(url_api)
9     if response.status_code != 200:
10         print(f"Error en la API ({estrategia}): {response.text}")
11         return None
12     return response.json()

```

Código 5.14: Función de recolección de métricas PageSpeed Insights.

Las métricas extraídas incluyen las puntuaciones de las categorías Lighthouse (Performance, SEO, Accessibility en escala 0-100), el Largest Contentful Paint (LCP) y el Total Blocking Time (TBT) para la variante móvil. Los resultados se almacenan como archivos JSON individuales con marca temporal en `data/seo/metrics_{timestamp}.json`.

La automatización se realiza mediante un workflow de GitHub Actions definido en `.github/workflows/seo_audit.yml` (Listado G.12, Anexo G).

Este workflow se ejecuta diariamente a las 08:00 UTC y también puede dispararse manualmente desde la interfaz de GitHub Actions. Tras la ejecución del script, el workflow realiza un commit automático con los nuevos datos si se han producido cambios, creando así un registro histórico de la evolución de las métricas SEO del sitio objetivo a lo largo del tiempo.

5.8– Configuración y prompts

5.8.1. Sistema de configuración centralizado

El sistema de configuración se estructura en dos ficheros JSON y un módulo Python que proporciona acceso programático con compatibilidad hacia atrás.

El fichero `config/experiment_config.json` (versión 1.1.0) centraliza todos los parámetros del experimento (Listado G.13, Anexo G).

El fichero `config/queries.json` (versión 2.0.0) contiene las 100 consultas del experimento con un sistema de rotación documentado en ADR-013. Cada consulta tiene un identificador único (Q001-Q100), una categoría (informativa, comparativa o navegacional) y un flag `original_15` que indica si pertenece al conjunto original de 15 consultas del prototipo:

```
1 {
2   "version": "2.0.0",
3   "total": 100,
4   "rotation": {
5     "core": ["Q001", "Q002", ..., "Q020"],
6     "R1": ["Q021", ..., "Q040"],
7     "R2": ["Q041", ..., "Q060"],
8     "R3": ["Q061", ..., "Q080"],
9     "R4": ["Q081", ..., "Q100"]
10  },
11  "queries": {
12    "Q001": {
13      "text": "Que proyectos existen para enseñar programacion a niños en España?",
14      "category": "informativa",
15      "original_15": true
16    }
17  }
18 }
```

Código 5.15: Formato v2 del banco de consultas queries.json.

5.8.2. Cargador de configuración

El módulo `src/config.py` proporciona funciones de acceso a la configuración. El formato activo de consultas es la versión 2 (diccionario Q001-Q100 con categorías y bloques de rotación); el cargador mantiene soporte para el formato versión 1 (lista plana de strings por categoría) para compatibilidad con datos históricos de ejecuciones anteriores al cambio de formato (Listado G.14, Anexo G).

La función `get_discovery_queries` excluye las consultas navegacionales (aquellas que preguntan directamente por el sitio objetivo, como “¿Qué es Programamos.es?”) del proceso de descubrimiento, ya que distorsionarían el ranking al generar auto-referencias. La función `get_queries_for_run` devuelve las 20 consultas *core* más las 20 del bloque rotativo seleccionado, generando 40 consultas por ejecución.

5.8.3. Registro de prompts versionado

El módulo `src/prompts/registry.py` centraliza todos los prompts del sistema en un diccionario versionado con *changelog* (Listado G.15, Anexo G).

El prompt del agente RAG merece atención especial por su estructura tripartita. La sección PROCESO guía al agente a través de las fases de análisis, búsqueda y síntesis. La sección FORMATO DE RESPUESTA define el esquema JSON esperado con un ejemplo concreto. La sección REGLAS DE CITACIÓN establece seis normas que garantizan la trazabilidad entre afirmaciones y fuentes, requisito imprescindible para el cálculo de las métricas GEO. La instrucción de estilo (“Sé conciso e informativo, como un motor de búsqueda IA, usa lenguaje natural en español”) orienta al modelo hacia respuestas que simulen fielmente el tono de los motores generativos reales.

El registro incluye además el prompt del analizador de sentimiento, utilizado por el módulo de evaluación en vivo para clasificar el tono de las menciones de la marca en las respuestas de los motores reales.

5.9— Scripts de automatización

Los scripts `scripts/run_discovery.py` y `scripts/run_experimental.py` constituyen el mecanismo principal de ejecución del pipeline, orquestando todos los componentes desde la terminal local. Todas las operaciones computacionalmente intensivas son llamadas a APIs externas (Gemini para el RAG Judge, Google AI para embeddings) u operaciones en CPU (scraping, chunking, FAISS, cálculo de métricas), por lo que no se requiere infraestructura GPU.

5.9.1. Script de descubrimiento

El script de descubrimiento (`run_discovery.py`) orquesta seis fases secuenciales (Listado G.16, Anexo G).

El script experimental (`run_experimental.py`) carga el vectorstore congelado, reembebe únicamente el sitio objetivo, fusiona ambos vectorstores y ejecuta el RAG Judge para cada consulta (Listado G.17, Anexo G).

La operación `frozen_vs.merge_from(target_vs)` de FAISS es una operación en memoria que añade los vectores del sitio objetivo al índice congelado sin modificar los archivos originales en disco. Esto garantiza la inmutabilidad del vectorstore de competidores entre ejecuciones, cumpliendo la restricción experimental de que solo el contenido del sitio objetivo cambia entre runs.

Para acelerar ejecuciones sucesivas, el vectorstore del sitio objetivo se cachea en disco (`data/target_vectorstore_cache/`) y se reutiliza si su antigüedad no supera un umbral configurable (parámetro `-cache-ttl`, 24 horas por defecto): mientras el contenido del objetivo no cambie, el costoso paso de rastreo y *embedding* no se repite en cada run, sino que se carga el índice ya calculado.

El *scorecard* generado por cada ejecución incluye métricas agregadas (tasa de visibilidad, Share of Model promedio y número medio de citaciones por respuesta) y métricas por consulta individual, almacenándose en `data/geo/experimental/{run_id}/scorecard.json`.

5.10— Dashboard web

Sobre el pipeline de investigación se construye un dashboard web que da soporte al modo plataforma descrito en el Capítulo 4, visualizando los resultados de los tres pipelines (experimental, live y SEO) y las recomendaciones del optimizador en una interfaz interactiva. La arquitectura se divide en un backend de API y un frontend de página única.

El backend, implementado con FastAPI, expone una API REST que lee directamente los datos basados en ficheros que generan los scripts del pipeline (modo experimental) y los workflows de CI (evaluación en vivo y métricas SEO); no emplea ninguna base de datos externa para los datos de investigación (la única base SQLite, `backend/jobs.db`, se reserva para el seguimiento de trabajos asíncronos). Sus rutas se corresponden uno a uno con los componentes del sistema: `catalog` (runs y definiciones de métricas disponibles), `experimental`, `live` y `seo` (resultados de cada pipeline), `metrics` y `overview` (agregados y panel combinado), `optimizer` (priorización de consultas y recomendaciones GEO) y `jobs` (cola de ejecución de tareas). La autenticación es opcional mediante la variable de entorno `DASHBOARD_API_KEY` (cabecera `X-API-Key`), de modo que en desarrollo los endpoints quedan abiertos.

El frontend es una aplicación React 18 con Vite y TypeScript. La interfaz se construye con la biblioteca de componentes shadcn/ui (sobre primitivas Radix y Tailwind CSS), el estado global se gestiona con Zustand y la obtención de datos con TanStack Query. Cada página de la aplicación se corresponde con un router del backend, ofreciendo vistas de visión general, detalle y comparación de runs, métricas SEO y el panel del optimizador. Las Figuras 5.1, 5.2 y 5.3 muestran algunas de las vistas principales.

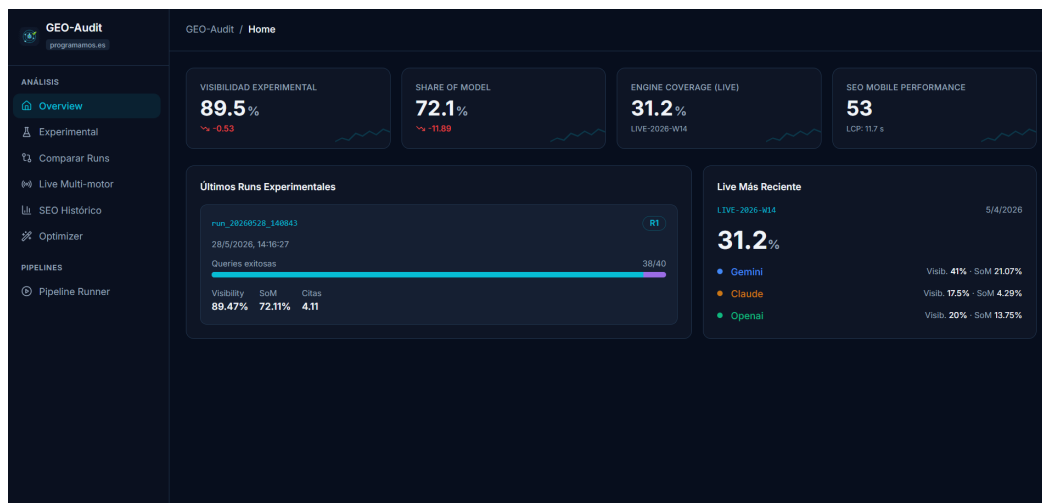


Figura 5.1: Vista de visión general del dashboard: instantánea combinada de los modos experimental, live y SEO.

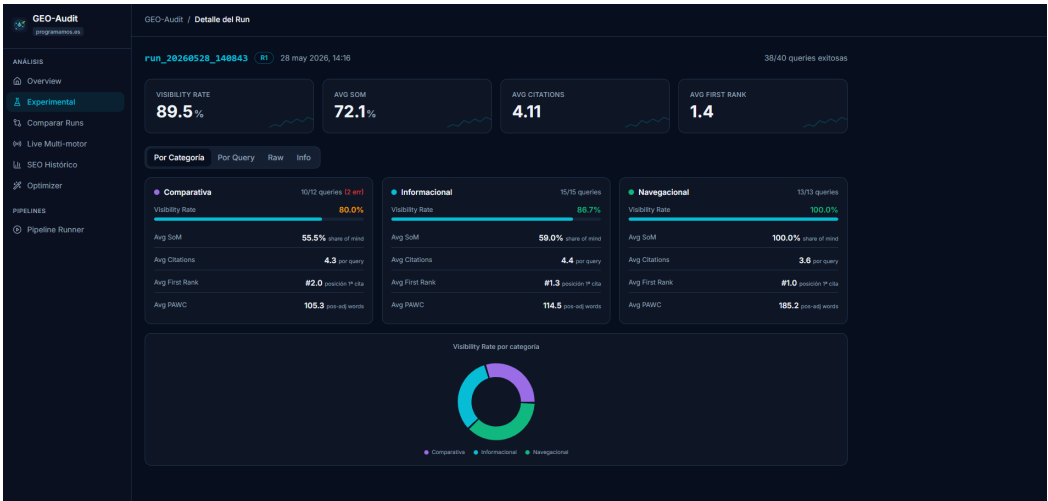


Figura 5.2: Detalle de una ejecución experimental: métricas GEO por consulta y agregados por categoría.

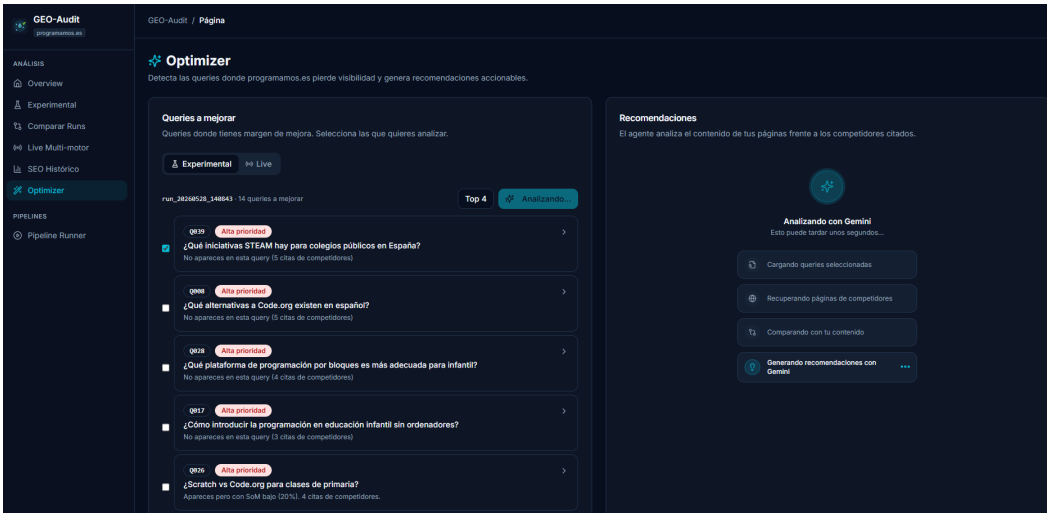


Figura 5.3: Panel del optimizador: consultas priorizadas y recomendaciones GEO generadas por el agente.

La Tabla 5.1 relaciona cada pantalla del dashboard con su función, los datos que consume (router del backend y ficheros de origen) y el objetivo específico del trabajo al que da soporte.

Tabla 5.1: Pantallas del dashboard: función, datos consumidos y objetivo cubierto.

Pantalla	Función	Datos usados	Objetivo
Visión general	Instantánea combinada: último run experimental, última semana live y SEO, con alertas	<code>/api/dashboard/overview</code> (scorecards, live y SEO más recientes)	OE8
Experimental	Listado de runs con sus agregados y evolución entre ejecuciones	<code>/api/runs/experimental</code> (<code>data/geo/experimental/run_*</code>)	OE2, OE8
Detalle de run	Métricas GEO consulta a consulta y agregados por categoría de un run	<code>/api/runs/experimental/{id}</code> (scorecard + resultados crudos)	OE2, OE4
Comparar	Diferencias de métricas entre dos runs; consultas ganadas y perdidas	<code>/api/runs/experimental/compare</code>	OE2
Live	Visibilidad por motor y por semana; cobertura entre motores	<code>/api/runs/live</code> (<code>data/geo/live/</code>)	OE5
SEO	Serie temporales de PageSpeed (móvil y escritorio)	<code>/api/seo</code> (<code>data/seo/</code>)	OE6
Optimizador	Priorización de consultas perdidas y recomendaciones del agente	<code>/api/optimizer</code> (scorecards + <code>src/optimizer/</code>)	OE7
Jobs	Lanzamiento y monitorización de ejecuciones del pipeline	<code>/api/jobs</code> (SQLite <code>jobs.db</code>)	OE8

5.11– Métricas del código

La Tabla 5.2 resume el tamaño del código fuente por componente, medido con la herramienta `cloc` (líneas de código efectivas, excluyendo blancos y comentarios). Se distingue el código propio del generado: los componentes de interfaz de `shadcn/ui` se instalan como código fuente en el repositorio (`frontend/src/components/ui/`), pero no se contabilizan como desarrollo propio y se excluyen del análisis de calidad.

Tabla 5.2: Tamaño del código fuente por componente (medido con `cloc`).

Componente	Ficheros	Líneas de código
Pipeline de investigación (<code>src/</code>)	13	2 612
Scripts y colectores (<code>scripts/</code> , <code>collect_metrics/</code>)	4	745
Backend del dashboard (<code>backend/app/</code>)	16	1 026
Frontend propio (<code>frontend/src/</code> , sin <code>ui/</code> ni tests)	37	3 689
Tests del backend (<code>backend/tests/</code>)	9	764
Tests del frontend (<code>frontend/src/test/</code>)	4	264
Tests del núcleo del pipeline (<code>tests/</code>)	5	204
Total de código propio	88	9 304
Componentes generados (<code>shadcn/ui</code>)	49	3 478

En cuanto a dependencias externas, el pipeline de investigación declara 20 paquetes en su `requirements.txt` (ecosistema LangChain, FAISS, SDKs de Google, Anthropic y OpenAI), el backend 14 (FastAPI, pydantic, pytest) y el frontend 51 dependencias de producción y 25 de desarrollo (React, Radix, Tailwind, TanStack Query, vitest). Las versiones están fijadas con rangos acotados para garantizar instalaciones reproducibles, con las restricciones de compatibilidad documentadas en el Capítulo 4 (en particular, el ecosistema LangChain fijado a la serie 0.3.x).

5.12– Matriz de trazabilidad

Como cierre del capítulo, la Tabla 5.3 traza cada objetivo específico (Capítulo 1) a través del diseño, la implementación, las pruebas y los resultados, permitiendo verificar que todos los objetivos tienen soporte en cada fase del trabajo.

Tabla 5.3: Matriz de trazabilidad: objetivos $\rightarrow$ diseño $\rightarrow$ implementación $\rightarrow$ pruebas $\rightarrow$ resultados.

Objetivo específico	Diseño	Implementación	Pruebas	Resultados
OE1. Marco de métricas GEO	§4.3	§5.4	§6.6	§7.1
OE2. Pipeline experimental RAG	§4.2	§5.1, §5.3	§6.6, §6.6.2	§7.1
OE3. Descubrimiento de competidores	§4.2.2	§5.2	§6.6.2	§7.1
OE4. Protocolo de 100 consultas	§4.2.3	Anexo B	§6.6.2	§7.1
OE5. Evaluación en vivo	§4.1.3	§5.5	§6.4	§7.2
OE6. Métricas SEO automatizadas	§4.1.5	§5.7	§6.4	§7.3
OE7. Agente GEOOptimizer	§4.5	§5.6	§6.2	§7.5
OE8. Dashboard web	§4.1.7	§5.10	§6.2, §6.3	Figs. 5.1–5.3

Pruebas y calidad

Este capítulo describe la estrategia de pruebas y aseguramiento de la calidad del sistema GEO-Audit. La validación se organiza en cinco niveles complementarios: una suite automatizada de pruebas del backend, pruebas del frontend, una cadena de integración continua, análisis de calidad del código (ESLint, métricas de complejidad y SonarCloud), y la validación funcional del núcleo del pipeline y del protocolo experimental. El objetivo es doble: garantizar que el software de la capa de dashboard se comporta según lo especificado, y asegurar que las métricas GEO calculadas por el núcleo son correctas y reproducibles.

6.1— Estrategia de pruebas

El sistema combina pruebas automatizadas con validación funcional, según la naturaleza de cada componente. La capa de dashboard (backend FastAPI y frontend React) dispone de una suite automatizada ejecutada de forma continua en cada cambio. El núcleo de cálculo de métricas (`src/`) se valida mediante casos unitarios sobre sus propiedades verificables (coincidencia de URLs, fórmula de PAWC, etc.) y mediante el protocolo experimental de reproducibilidad. La Tabla 6.1 resume los niveles.

Nivel	Componentes	Automatización
Backend	Routers FastAPI y servicios (<code>run_loader</code> , <code>metrics_aggregator</code> , <code>optimizer</code>)	pytest + coverage (CI)
Frontend	Componentes, reducers y utilidades de React	vitest (CI)
Calidad de código	Estilo y errores estáticos	ESLint + hook pre-commit
Núcleo del pipeline	<code>CitationExtractor</code> , chunker, configuración y prompts (<code>tests/</code>)	pytest + coverage (CI)
Experimental	Reproducibilidad y validación experimental $\leftrightarrow$ live	Protocolo definido

Tabla 6.1: Niveles de la estrategia de pruebas.

Atendiendo a la clasificación clásica por alcance, la suite automatizada (172 pruebas en total) se distribuye como muestra la Tabla 6.2. Las pruebas unitarias ejercitan funciones y servicios aislados; las de integración recorren la API completa (router, servicios y carga de datos) mediante peticiones HTTP reales contra la aplicación en memoria; las de interfaz renderizan componentes React en un DOM simulado; y el nivel de sistema lo cubren

la cadena de integración continua, que en cada cambio construye el frontend y ejecuta todas las suites sobre un entorno limpio, junto con la validación funcional del pipeline experimental descrita al final del capítulo.

Tipo	Qué cubre	Nº de pruebas
Unitarias (núcleo del pipeline)	<code>CitationExtractor</code> , chunker, cargador de configuración y registro de prompts (<code>tests/</code>)	38
Unitarias (backend)	Servicios internos: normalización de <i>scorecards</i> , agregación de métricas, priorización (<code>test_services</code>)	45
Unitarias (frontend)	Reducer de notificaciones y funciones de utilidad	20
Integración (backend)	Los 8 routers de la API vía <code>TestClient</code> , incluyendo autenticación y casos de error	43
Interfaz (frontend)	Componentes compartidos renderizados con <code>vitest</code> + <code>jsdom</code>	26
Sistema	CI completa (build + suites en cada cambio) y validación funcional del pipeline	—
Total automatizadas		172

Tabla 6.2: Distribución de las pruebas por tipo.

6.2— Pruebas automatizadas del backend

El backend cuenta con 88 pruebas distribuidas en ocho módulos, una por cada router más un módulo para los servicios internos. Todas se ejecutan sobre la aplicación FastAPI mediante el `TestClient` de Starlette, instanciado en un *fixture* compartido (`conftest.py`), de modo que cada prueba realiza peticiones HTTP reales contra la API en memoria sin necesidad de levantar un servidor. La suite no depende de los datos publicados en `data/`: el `conftest.py` apunta el directorio de datos a un conjunto congelado de *fixtures* (`backend/tests/data/`, con dos runs experimentales, una semana live y la serie SEO), lo que hace la suite determinista con independencia de qué runs se añadan o retiren del repositorio.

Módulo	Qué cubre
<code>test_catalog</code>	Catálogo de runs disponibles y definiciones de métricas.
<code>test_experimental</code>	Listado, detalle, resultados crudos y comparación de runs experimentales.
<code>test_live</code>	Listado, detalle, consulta individual, comparación y <i>alias</i> de la última ejecución live.
<code>test_metrics</code>	Agregación de métricas: medias por categoría, matriz de cobertura y métricas por motor.
<code>test_optimizer</code>	Priorización de consultas y endpoint de análisis (con los LLM simulados).
<code>test_overview</code>	Panel combinado (experimental + live + SEO).
<code>test_seo</code>	Métricas PageSpeed en variantes móvil y escritorio.
<code>test_services</code>	Servicios internos: normalización de <i>scorecards</i> v1/v2, coincidencia de URLs y casos límite.

Tabla 6.3: Módulos de pruebas del backend.

Una decisión metodológica relevante es que las pruebas del optimizador simulan (*mock*) tanto la priorización como el agente `GEOOptimizer`, evitando cualquier llamada real a las

APIs de Gemini, Anthropic u OpenAI. Esto hace que la suite sea determinista, gratuita y ejecutable en integración continua sin credenciales. La Tabla 6.4 recoge una selección de casos representativos.

Caso	Entrada	Resultado esperado
Autenticación por X-API-Key	Clave correcta / incorrecta	Acceso / rechazo
Análisis con cuerpo mal formado	<code>query_ids</code> no es lista	422 (validación)
Análisis sin consultas coincidentes	<code>query_id</code> inexistente	404
Análisis con demasiadas consultas	5 <code>query_ids</code> (>4)	422 (sin llamar al LLM)
Error de cuota del modelo	Excepción 429 simulada	429
Run experimental inexistente	ID desconocido	404
Falso positivo por dominio parecido	<code>programamos.es.evil.com</code>	No coincide con el objetivo
Priorización de consultas	Consultas con alta visibilidad	Excluidas (no requieren mejora)

Tabla 6.4: Casos representativos de las pruebas del backend.

La ejecución genera además un informe de cobertura (**coverage**) sobre el paquete `app`, que se conserva como artefacto de la integración continua. La Tabla 6.5 resume los porcentajes por módulo. La cobertura global es del 80 %, con todos los *routers* de la API por encima del 79 % (varios superan el 89 %) y los servicios principales (`run_loader`, `metrics_aggregator`, `live_loader`, `seo_loader`) entre el 80 % y el 95 %. El módulo `job_runner`, que orquesta los pipelines mediante subprocesos del sistema, queda en el 36 % al depender de procesos externos no instrumentados en la suite unitaria; cubrirlo requiere pruebas de integración con ejecuciones reales del pipeline, fuera del alcance de la verificación funcional automatizada.

Módulo	Sentencias	Cobertura
<code>app/deps.py</code>	8	100 %
<code>app/main.py</code>	21	100 %
<code>app/settings.py</code>	15	100 %
<code>routers/metrics.py</code>	29	97 %
<code>services/live_loader.py</code>	42	95 %
<code>routers/catalog.py</code>	46	91 %
<code>routers/optimizer.py</code>	44	91 %
<code>routers/seo.py</code>	23	91 %
<code>routers/experimental.py</code>	54	89 %
<code>routers/overview.py</code>	38	89 %
<code>services/seo_loader.py</code>	32	88 %
<code>services/metrics_aggregator.py</code>	159	84 %
<code>services/run_loader.py</code>	85	80 %
<code>routers/live.py</code>	48	79 %
<code>routers/jobs.py</code>	41	59 %
<code>services/job_runner.py</code>	98	36 %
Total	783	80 %

Tabla 6.5: Cobertura de pruebas del backend por módulo (medida con `coverage.py`).

6.3— Pruebas del frontend

El frontend se valida con vitest, cubriendo el renderizado de componentes compartidos, la lógica del *reducer* de notificaciones y las funciones de utilidad. A ello se suma el análisis estático con ESLint (configurado en `eslint.config.js`) y un hook de pre-commit gestionado con Husky que ejecuta `lint-staged` (`eslint -fix` sobre los ficheros `src/**/*.{ts,tsx}` en *staging*), evitando que llegue al repositorio código que no cumple las reglas de estilo.

6.4— Integración continua

El *workflow* `tests.yml` de GitHub Actions se ejecuta en cada *pull request* y en cada *push* a `main`, con cinco trabajos independientes:

- **Backend Tests:** instala dependencias y ejecuta `pytest` con cobertura (Python 3.11), publicando el informe como artefacto.
- **Frontend Lint:** ejecuta ESLint (Node 20).
- **Frontend Tests:** ejecuta la suite de vitest.
- **Frontend Build:** compila el *bundle* de producción, verificando que el proyecto construye sin errores.
- **SonarCloud Analysis:** regenera la cobertura del backend, del núcleo del pipeline y del frontend y publica el análisis de calidad en SonarCloud (Sección 6.5).

A diferencia de los *workflows* `seo_audit.yml` y `geo_live.yml`, descritos en los capítulos anteriores, que son pipelines de recolección de datos programados, `tests.yml` es exclusivamente de verificación: no produce datos, solo valida cada cambio del código.

6.5— Análisis de calidad del código

Más allá de las pruebas, la calidad del código se evalúa con tres herramientas complementarias: ESLint para el frontend, las métricas de complejidad de `radon` para el código Python y el análisis estático continuo de SonarCloud sobre todo el repositorio.

6.5.1. Métricas de complejidad y mantenibilidad

El análisis con `radon` sobre el código Python propio (pipeline, backend, colectores y scripts; 218 bloques entre clases, funciones y métodos) arroja una complejidad ciclomática media de 5,1, correspondiente al rango B de la escala de la herramienta (código simple y bien estructurado). El índice de mantenibilidad sitúa 41 de los 42 ficheros analizados en el rango A; el único fichero en rango B es `src/optimizer/geo_optimizer.py`, el módulo más extenso del agente optimizador. Cinco funciones superan el umbral de complejidad 20 y constituyen los puntos calientes del sistema: el bucle principal de `run_experimental.py` (31), el recorrido del árbol HTML en `StructuredWebLoader._walk` (30), la normalización de recomendaciones del optimizador (26), y las funciones de consulta a Gemini en el descubridor (24) y en el evaluador live (26). Son funciones de orquestación con muchas ramas de manejo de errores de APIs externas, identificadas como candidatas a refactorización en trabajo futuro.

En el frontend, la ejecución de ESLint sobre el código propio termina sin errores (tres avisos menores), y el hook de pre-commit garantiza que ningún cambio entra al repositorio sin pasar el linter.

6.5.2. Análisis continuo con SonarCloud

El repositorio está conectado a SonarCloud, que ejecuta un análisis estático completo en cada *push* mediante un job dedicado del workflow `tests.yml`: se generan los informes de cobertura del backend (`coverage.xml`), del núcleo del pipeline (`coverage-core.xml`) y del frontend (`lcov.info`) y el escáner los publica junto con las métricas de calidad (fiabilidad, seguridad, mantenibilidad, duplicación y deuda técnica). La configuración (`sonar-project.properties`) analiza el código propio y excluye los componentes de

interfaz generados por `shadcn/ui`, de modo que las métricas reflejan exclusivamente el código escrito para este trabajo.

El primer análisis detectó seis *bugs* (avisos de accesibilidad en el dashboard: elementos clicables sin alternativa de teclado), tres vulnerabilidades de análisis de flujo en el *crawler* y cuatro *security hotspots* (dos expresiones regulares con riesgo de *backtracking* polinómico y dos usos de `http://`). Tras revisarlos, se corrigieron: las tarjetas y filas clicables del frontend se hicieron accesibles por teclado (etiquetas asociadas y `role="button"` con manejo de `Enter`/espacio), las dos expresiones regulares se sustituyeron por operaciones de cadena equivalentes sin riesgo de *backtracking*, y el *crawler* incorporó saneado de URLs en los registros y una validación defensiva de esquema y dominio antes de cada petición del rastreo BFS. Los dos avisos restantes se documentaron como riesgo revisado: los usos de `http://` son un identificador de *namespace* XML exigido por la especificación de sitemaps y los orígenes CORS de desarrollo local, y la advertencia residual sobre el *crawler* es inherente a su función (rastrear URLs descubiertas del propio sitio auditado, ya validadas contra el dominio objetivo). La Tabla 6.6 resume el estado final tras esta iteración (análisis de la rama principal, junio de 2026, 8 202 líneas de código).

Dimensión	Resultado	Rating
Fiabilidad	0 <i>bugs</i>	A
Seguridad	0 vulnerabilidades abiertas, 0 <i>hotspots</i> pendientes	A
Mantenibilidad	169 <i>code smells</i> , deuda técnica ≈ 17 h	A
Duplicación	0,5 % de líneas duplicadas	—
Cobertura	67,6 %	—

Tabla 6.6: Resultados del análisis en SonarCloud tras la iteración de corrección (proyecto `angelmanuelherrero_TFG`).

La cobertura se mide sobre el código con lógica determinista comprobable, criterio declarado en la propia configuración del análisis: los módulos de orquestación de red y LLM (descubridor, juez RAG, *crawler*, *embeddings*, optimizador y colectores) se validan funcionalmente mediante el protocolo experimental, y las páginas del frontend, que son composición declarativa de interfaz, mediante el build, ESLint y las pruebas de los componentes compartidos. Dentro de ese ámbito, la cobertura por capas es del 80 % en el backend (Tabla 6.5) y, en el núcleo del pipeline, del 100 % en el `CitationExtractor`, 95 % en el chunker y 100 % en el registro de prompts.

6.6— Validación del núcleo y del experimento

6.6.1. Casos unitarios del `CitationExtractor`

El cálculo de métricas GEO se valida con una suite propia de 38 pruebas unitarias (`tests/`, ejecutada en integración continua con cobertura) cuyos casos pueden comprobarse a mano. Para la visibilidad, cero citas del objetivo producen `is_visible = False` y una o más, `True`. Para el Share of Model, dos citas del objetivo sobre cinco totales producen un 40 %, y el caso de cero citas totales devuelve 0 sin incurrir en división por cero. Para el ranking, la posición de la primera cita del objetivo se devuelve correctamente, o `None` si no aparece.

La coincidencia de URLs (`_url_matches_target`) se prueba con los casos límite que motivaron su diseño (Tabla 6.7): la comprobación de sufijo evita que dominios como `programamos.es.evil.com` se cuenten como objetivo.

URL de entrada (objetivo: programamos.es)	Resultado
https://programamos.es/talleres	Coincide
https://www.programamos.es/	Coincide (normaliza <code>www.</code>)
http://programamos.es	Coincide (normaliza el esquema)
https://programamos.es?ref=gemini	Coincide (<i>query string</i>)
https://programamos.es.evil.com	No coincide (sufijo seguro)
https://noprogramamos.es	No coincide

Tabla 6.7: Casos de coincidencia de URLs con el dominio objetivo.

El PAWC se contrasta con un cálculo manual. Dado un *output* con dos citas del objetivo (ocho palabras en la posición 2 y seis palabras en la posición 4), el valor esperado es $8/\log_2(3) + 6/\log_2(5) = 5,05 + 2,58 = 7,63$, que es exactamente lo que devuelve `_calculate_pawc` redondeado a dos decimales.

6.6.2. Reproducibilidad

Como el modo experimental es un estudio controlado, su reproducibilidad se sostiene en varios controles (Tabla 6.8). La limitación honesta es que Gemini 2.5 Flash no admite el parámetro `seed`, por lo que incluso con `temperature=0` persiste una variabilidad residual no eliminable.

Control	Implementación	Propósito
Temperatura 0	<code>rag_simulator.temperature: 0.0</code>	Minimizar variabilidad del juez
Configuración versionada	<code>experiment_config.json</code>	Trazabilidad de parámetros
Vectorstore congelado	FAISS local en <code>data/frozen_vectorstore/</code>	Recuperación constante
Prompts versionados	<code>src/prompts/registry.py</code> con <code>changelog</code>	Reproducir el prompt exacto

Tabla 6.8: Controles de reproducibilidad del modo experimental.

6.6.3. Validación experimental frente a evaluación en vivo

La contribución metodológica del trabajo es la validación dual: comprobar si una mejora detectada en el simulador RAG se traduce en mayor visibilidad en los motores reales. El procedimiento consiste en registrar un *baseline* con ambos modos, aplicar una intervención de contenido y re-medir, calculando la correlación (de Spearman) entre el cambio de Share of Model en el simulador y el cambio de visibilidad en los motores reales. Los resultados concretos de esta validación se presentan en el capítulo de resultados.

Resultados y discusión

Este capítulo presenta los resultados obtenidos por el sistema GEO-Audit sobre programamos.es en su estado actual (previo a cualquier intervención de contenido), a partir de los tres pipelines de medición: el simulador experimental basado en RAG, la evaluación en vivo sobre motores reales y la recolección de métricas SEO. Se incluye además una medición del coste de ejecución del pipeline experimental y una discusión sobre la relación entre los valores del simulador y los de los motores reales, que constituye la principal aportación metodológica del trabajo.

7.1– Resultados del modo experimental

Los resultados corresponden al run `run_20260528_140843`, que ejecutó las 40 consultas del bloque R1 (20 *core* más 20 del bloque de rotación) sobre el vectorstore con los competidores congelados.

7.1.1. Coste de ejecución

El coste de ejecución se reparte en dos fases. La primera, que comprende el rastreo, la fragmentación y el *embedding* del sitio objetivo, se ejecuta una sola vez y se cachea: procesar las 576 páginas del sitio (8754 fragmentos) tomó unos 28 minutos, tiempo dominado por el rastreo cortés de cada página (con pausa entre peticiones) y por el *embedding* de los fragmentos a través de la API de Google, enviados en lotes de 50 con pausas de 1,5 segundos para no superar la cuota. La segunda fase, las 40 consultas al RAG Judge, es la que se repite en cada run. Gracias a la caché del vectorstore del objetivo (Capítulo 5), el run definitivo se saltó por completo la primera fase y resolvió las 40 consultas en unos 8 minutos (de 14:08 a 14:16).

El RAG Judge está limitado por el tier gratuito de Gemini 2.5 Flash (15 peticiones por minuto): como el agente realiza varias llamadas por consulta, al superar ese límite reintenta con *backoff* exponencial. Conviene ser honesto en este punto: el tier gratuito no garantiza una ejecución sin errores. En este run, 2 de las 40 consultas (ambas comparativas) fallaron al agotar los reintentos; no es un error que se haya *evitado*, sino *reducido* respecto a ejecuciones en las que el límite se agota de forma sostenida y la tasa de fallo es mucho mayor. Eliminarlo por completo exigiría un tier de pago sin límite de peticiones, por lo que este límite es la principal fuente de incertidumbre operativa del modo experimental.

7.1.2. Métricas de visibilidad

Sobre las 38 consultas exitosas, el sitio alcanza una tasa de visibilidad del 89,5 % y un Share of Model medio del 72,1 % (Tabla 7.1).

Métrica	Valor
Consultas evaluadas	40 (<i>core</i> + R1)
Consultas exitosas / con error	38 / 2
Tasa de visibilidad	89,5 %
Share of Model medio	72,1 %
Citas medias por consulta	4,11

Tabla 7.1: Agregados del modo experimental (run `run_20260528_140843`, bloque R1).

El desglose por categoría (Tabla 7.2) reproduce el patrón observado en la evaluación en vivo. Las consultas navegacionales obtienen visibilidad y Share of Model del 100 %, pero esto es lo esperable: preguntan por el propio sitio, así que citarlo es casi una obviedad. El valor está en las otras dos categorías, donde los competidores congelados disputan una parte de las citas: la visibilidad sigue siendo alta (informacional 86,7 %, comparativa 80,0 %) pero el Share of Model baja a la franja del 55–59 %, indicando que cuando el objetivo aparece comparte protagonismo con los competidores.

Categoría	n	Visibilidad	SoM medio
Navegacional	13	100,0 %	100,0 %
Informacional	15	86,7 %	59,0 %
Comparativa	12	80,0 %	55,5 %

Tabla 7.2: Desglose por categoría del run experimental (bloque R1).

A nivel de consulta individual, 20 de las 38 exitosas alcanzan un Share of Model del 100 % (dominio total del objetivo), mientras que cuatro quedan invisibles (Share of Model del 0 %): dos comparativas (Q008, “¿Qué alternativas a Code.org existen en español?”; Q028) y dos informacionales (Q017, Q039). Las dos consultas con error (Q006, Q007) son ambas comparativas y fallaron por el límite de peticiones. El patrón es consistente con el modo live: el objetivo domina lo navegacional, compite en lo informacional y es más vulnerable en lo comparativo. Conviene notar, no obstante, que estos valores de Share of Model (72 % de media) son muy superiores a los del modo live; esa diferencia se analiza en la Sección 7.4.

7.2– Resultados del modo live

La evaluación en vivo lanza semanalmente el banco de consultas contra los tres motores reales con búsqueda web activada: `gemini-2.5-flash` con Google Search grounding, `gpt-4o-mini` con la herramienta `web_search` de OpenAI y `claude-haiku-4-5` con la de Anthropic (Sección 4.1.3); en adelante se abrevian como Gemini, OpenAI y Claude. Se dispone de cinco semanas consecutivas (LIVE-2026-W14 a W18, del 5 de abril al 3 de mayo de 2026). Conviene una precisión metodológica: cada motor se evalúa con un volumen distinto de consultas (Gemini con las 100, OpenAI con 60, Claude con 40), por lo que la comparación entre motores es indicativa y no estrictamente equivalente. Esta asimetría no es arbitraria, sino la respuesta al coste y a los límites de peticiones de cada proveedor: como cada consulta consume una llamada (de pago en OpenAI y Anthropic) y dispara una búsqueda web real con su latencia asociada, lanzar las 100 consultas contra los tres

motores cada semana sería costoso y lento. Los *tiers* (Gemini al completo por ser gratuito, OpenAI a 60 y Claude a 40) acotan el coste y el tiempo por motor; cada ejecución semanal procesa así unas 200 consultas-motor, con una pausa de 2 segundos entre llamadas para respetar los límites, y tarda del orden de 40 minutos (entre 36 y 43 en las ejecuciones observadas). Al igual que en el modo experimental, ese tiempo está dominado por los límites de peticiones de las APIs. La Tabla 7.3 resume los promedios de las cinco semanas y la Figura 7.1 muestra la evolución de la visibilidad.

Motor	Consultas	Visibilidad me- dia	SoM medio	Rank medio
Gemini 2.5 Flash	100	45,0 %	20,4 %	3,5
GPT-4o-mini	60	20,7 %	15,9 %	1,3
Claude Haiku 4.5	40	15,5 %	3,8 %	6,3

Tabla 7.3: Promedios de la evaluación en vivo sobre cinco semanas (W14–W18). Engine Coverage medio: 33,2 %.

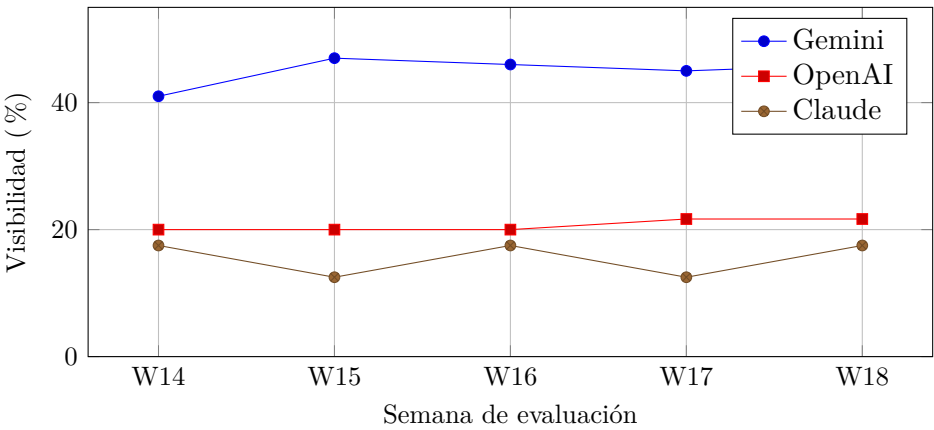


Figura 7.1: Evolución de la tasa de visibilidad de programamos.es por motor (W14–W18).

El orden entre motores es estable a lo largo de las cinco semanas: Gemini es el que más cita a programamos.es, seguido de OpenAI y, en último lugar, Claude. Gemini parte del 41 % en W14 y se consolida en torno al 45–47 % en las semanas siguientes; OpenAI se mantiene notablemente estable (20–22 %) y, pese a su menor visibilidad, sitúa la primera cita del objetivo en una posición muy temprana (rank medio 1,3), lo que sugiere que cuando lo cita lo hace de forma prominente. Claude es el motor con menor visibilidad (oscilando entre 12,5 y 17,5 %) y el Share of Model más bajo (en torno al 4 %). El Engine Coverage medio se mantiene estable entre el 31 y el 35 %. La consistencia del patrón a lo largo de cinco semanas refuerza la fiabilidad de estas observaciones frente a la variabilidad de una medición puntual.

7.2.1. Desglose por categoría de consulta

Agrupando las cinco semanas por categoría de consulta (Tabla 7.4) emerge el resultado más accionable de la evaluación en vivo: la visibilidad de programamos.es depende casi por completo del tipo de consulta. Los tres motores citan al sitio con frecuencia en las consultas navegacionales (Gemini 88 %, OpenAI 58 %, Claude 42 %), pero esto es lo esperable y aporta poca información: esas consultas preguntan explícitamente por el propio sitio (“¿En qué consiste el proyecto Programamos?”), de modo que citarlo es casi una obviedad y no mide su competitividad GEO real. El valor está en las otras dos categorías, donde el sitio

compite por captar a usuarios que todavía no lo conocen: en las informacionales y, sobre todo, en las comparativas, la visibilidad se hunde.

Categoría	Gemini	OpenAI	Claude
Navegacional	88 %	58 %	42 %
Informacional	29 %	4 %	4 %
Comparativa	21 %	0 %	2 %

Tabla 7.4: Tasa de visibilidad por categoría de consulta y motor (datos agrupados de W14–W18).

Gemini es el único motor que conserva visibilidad apreciable fuera de lo navegacional (29 % informacional, 21 % comparativa); OpenAI y Claude apenas citan al objetivo en esas categorías (4 % o menos). El patrón se traslada a consultas concretas. Entre las que citan a programamos.es de forma constante (las cinco semanas en Gemini) dominan las navegacionales (“¿En qué consiste el proyecto Programamos?”, “Recursos de Programamos para docentes”) junto a algunas informacionales sobre Scratch (“¿Qué recursos gratuitos hay para aprender Scratch?”). En el extremo opuesto, las consultas que *nunca* lo citan son comparativas genéricas: “¿Cuál es la mejor plataforma para que los niños aprendan a programar?”, “Diferencias entre Code.org, Scratch y otras plataformas” o “¿Qué alternativas a Code.org existen en español?”. El diagnóstico es claro: el sitio está bien posicionado para quien ya lo busca, pero pierde las consultas exploratorias y comparativas, donde el usuario descubre recursos nuevos, porque carece del contenido comparativo (tablas, rankings, “X frente a Y”) que sí ofrecen los competidores. Es precisamente el tipo de carencia que el agente optimizador está diseñado para detectar.

7.2.2. Tono de las menciones

El analizador de sentimiento clasificó el tono con que Gemini menciona la marca en las consultas donde resulta visible. De las 179 menciones clasificadas a lo largo de las cinco semanas, el 85 % fueron positivas y el 15 % neutras, sin ninguna mención negativa. Es decir, cuando los motores citan a programamos.es lo presentan de forma favorable: el reto del sitio es de *presencia*, no de *reputación*.

7.3– Resultados SEO

La recolección automatizada de métricas SEO (workflow diario sobre PageSpeed Insights) acumula 89 mediciones en una ventana de unos tres meses (del 4 de febrero al 5 de mayo de 2026). La Tabla 7.5 recoge la media de cada métrica junto a su rango, y la Figura 7.2 las compara visualmente. Las puntuaciones de SEO y accesibilidad se mantienen altas y muy estables; el punto débil es el rendimiento en móvil, con una media de 55 sobre 100 y una alta variabilidad (entre 33 y 73), asociada a un *Largest Contentful Paint* de unos 10–12 segundos, muy por encima del umbral recomendado. El rendimiento en escritorio es notablemente mejor (media 77).

Dispositivo	Rendimiento	SEO	Accesibilidad
Móvil	55,3 [33–73]	86,7	88,0
Escritorio	77,2 [44–86]	86,7	90,3

Tabla 7.5: Métricas SEO de PageSpeed Insights: media [mín–máx] de las 89 mediciones (feb–may 2026).

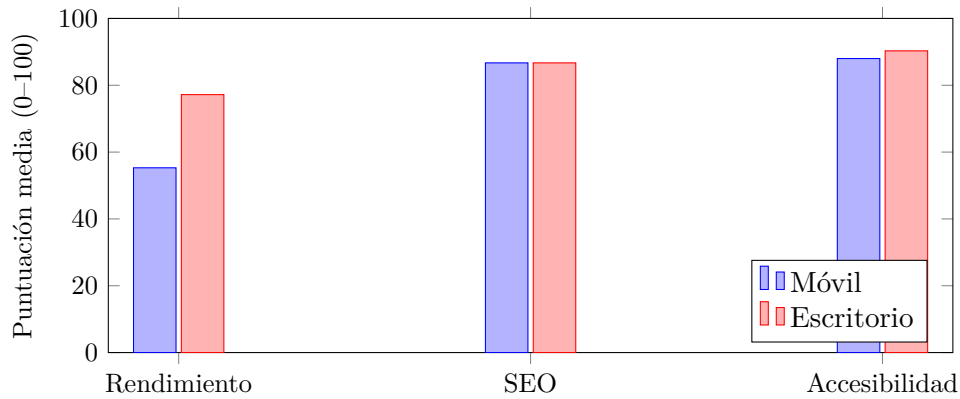


Figura 7.2: Puntuaciones medias de PageSpeed Insights por dispositivo (media de 89 mediciones).

7.4– Discusión: el simulador frente a la realidad

El contraste entre el modo experimental y el modo live es el resultado más relevante del trabajo. Comparando métricas homogéneas, la brecha es grande en ambas: la tasa de visibilidad del objetivo pasa del 89,5 % en el simulador a alrededor del 45 % en el mejor motor real (Gemini), y el Share of Model medio cae del 72 % al 20 %. Esta diferencia no es un defecto del sistema, sino una propiedad esperada y reconocida: el simulador trabaja sobre un vectorstore acotado de competidores congelados y no modela las señales externas que sí emplean los motores reales: autoridad de dominio, popularidad, presencia previa en el corpus de entrenamiento o mecanismos de recuperación propietarios.

La consecuencia metodológica es clara: el modo experimental no debe leerse en términos absolutos, sino como un indicador de la dirección del cambio cuando se modifica el contenido del sitio, en condiciones controladas y reproducibles. El modo live, por su parte, aporta la medida de visibilidad real, a costa de mayor variabilidad y menor control. Ambos son complementarios: el experimental aísla el efecto del contenido; el live confirma si ese efecto se traslada a los motores que usan los usuarios.

7.5– Evaluación del agente optimizador

Esta sección evalúa cualitativamente las recomendaciones del agente GEOOptimizer según tres criterios: (i) que cada recomendación esté anclada a evidencia real de un competidor del estudio y no a consejos SEO genéricos; (ii) que la palanca GEO propuesta sea pertinente al déficit detectado; y (iii) que la sugerencia sea concreta y accionable sobre una URL del sitio.

El agente se ejecutó sobre cuatro de las consultas de mayor prioridad de la evaluación en vivo (Q002, Q004, Q005 y Q016, todas con 0 % de visibilidad en los tres motores), que son precisamente consultas informacionales y comparativas, el punto débil identificado en las secciones anteriores. Produjo ocho recomendaciones priorizadas (Tabla 7.6); el informe completo, con el detalle y la justificación de cada recomendación, se reproduce en el Anexo F.

#	Palanca	URL objetivo	Impacto
1	Claridad semántica	/que-es-el-pensamiento-computacional/	Alto
2	Claridad semántica	/aprender-a-programar-sin-ordenador/	Alto
3	Citation readiness	/que-es-el-pensamiento-computacional/	Alto
4	Citation readiness	/aprender-a-programar-sin-ordenador/	Alto
5	Authority injection	/quienes-somos/	Alto
6	Claridad semántica	/vibot-the-robot/...	Medio
7	Machine scannability	/llms.txt (archivo nuevo)	Medio
8	Citation readiness	/quienes-somos/	Medio

Tabla 7.6: Recomendaciones generadas por el GEOOptimizer para las cuatro consultas perdidas analizadas.

Anclaje a evidencia. La mayoría de recomendaciones cita un fragmento literal de un competidor real recuperado por el sistema (Acebott, Codelearn, CodeMonkey) y se vincula a consultas concretas; por ejemplo, la recomendación 4 se apoya en un fragmento de competidor (“CodeMonkey ... Millones de estudiantes”) para justificar la ausencia de datos citables en el sitio. La excepción es la recomendación del archivo `llms.txt`, cuya “evidencia” es genérica: se trata de una buena práctica de *discovery*, no de una carencia anclada a un competidor.

Pertinencia de la palanca. El mapeo es coherente: las cuatro recomendaciones de *claridad semántica* responden a gaps de recuperación (páginas que aparecen en Google pero que el RAG no recupera), el déficit dominante observado en los modos experimental y live; las de *citation readiness* atacan la falta de datos con fuente, y la de *authority injection* refuerza `/quienes-somos/` para las consultas sobre organizaciones. La recomendación más discutible es la de `llms.txt` (*machine scannability*): su eficacia sobre la citación no está demostrada y queda más cerca del SEO técnico que del GEO de contenido. Los niveles de impacto de la Tabla 7.6 son, en todo caso, estimaciones heurísticas del agente y no mediciones empíricas sobre el caso de estudio.

Concreción. Es el punto más fuerte: cada recomendación indica la URL exacta, la ubicación precisa (primeros 256 tokens, segundo párrafo, etc.) y un ejemplo de reescritura listo para aplicar. El agente muestra además criterio propio: marca riesgos de *topic mismatch* (recomendaciones 5 y 6) y llega a sugerir crear una página-hub en lugar de forzar la optimización de una página inadecuada.

Limitaciones. La evaluación es cualitativa y sobre una única ejecución de cuatro consultas, por lo que es ilustrativa y no estadística. Además, los ejemplos de reescritura incluyen afirmaciones factuales concretas (nombres de cofundadores, colaboraciones institucionales, cifras de adopción) que el agente propone pero que requieren verificación humana antes de publicarse; el propio sistema lo reconoce al imponer en sus instrucciones no inventar datos. Por último, el impacto real de aplicar estas recomendaciones, es decir, si elevan o no la visibilidad, solo podría confirmarse con el ciclo completo de intervención y re-medicación descrito en el trabajo futuro.

Conclusiones y trabajo futuro

8.1— Conclusiones

Este trabajo ha construido y validado GEO-Audit, un sistema completo que cierra el ciclo de medir, diagnosticar y recomendar sobre la visibilidad de un sitio web en motores generativos: un pipeline experimental RAG con condiciones congeladas, una evaluación semanal automatizada sobre tres motores comerciales reales, la recolección diaria de métricas SEO, un agente de optimización anclado a evidencia y un dashboard que integra todos los resultados. El sistema completo se ha desarrollado con un coste de infraestructura de 0 euros y menos de 30 euros en APIs, con 172 pruebas automatizadas y calidad verificada de forma continua (rating A en fiabilidad, seguridad y mantenibilidad en SonarCloud).

Los hallazgos principales pueden resumirse en tres. Primero, la validación dual permite calibrar qué mide cada instrumento: la visibilidad del objetivo es del 89,5 % en el entorno controlado frente a un entorno al 45 % en el mejor motor real, y el Share of Model pasa del 72 % al 20 %, una diferencia esperable dado que el simulador aísla deliberadamente el contenido y prescinde de las señales de autoridad que operan en producción. Lejos de invalidar el simulador, esta cuantificación delimita el papel de cada componente: el entorno controlado anticipa la *dirección* de un cambio de contenido en condiciones reproducibles, y la evaluación en vivo calibra su magnitud real. Segundo, la visibilidad real de Programamos.es depende casi por completo del tipo de consulta. Que los motores lo citen cuando el usuario ya lo busca era previsible (88 % en consultas navegacionales en Gemini); el hallazgo relevante es la magnitud de la caída en las consultas informacionales y comparativas, precisamente aquellas en las que un usuario descubre recursos nuevos (4 % o menos en OpenAI y Claude). El orden entre motores es estable (Gemini cita más que OpenAI, y este más que Claude) y, cuando el sitio aparece, el tono es favorable (85 % de menciones positivas y ninguna negativa), de modo que el reto del sitio es de presencia, no de reputación. Tercero, ese diagnóstico es accionable: el agente optimizador lo tradujo en ocho recomendaciones priorizadas y ancladas a fragmentos reales de los competidores que ganan esas consultas, en lugar de consejos SEO genéricos.

La lectura global es que resulta viable montar, con medios accesibles, un observatorio GEO riguroso para un sitio web real: las métricas definidas, el control experimental y la evaluación en vivo bastan para fundamentar decisiones de contenido informadas en un campo aún sin herramientas de referencia. La cuantificación causal del impacto de las recomendaciones, aplicarlas y volver a medir, queda como la continuación natural del trabajo.

8.2— Cumplimiento de objetivos

El objetivo general del trabajo era construir un sistema para medir, diagnosticar y orientar la mejora de la visibilidad de programamos.es en los motores generativos de IA. La Tabla 8.1 resume el grado de cumplimiento de los objetivos específicos definidos en el Capítulo 1.

Obj.	Objetivo específico	Estado
OE1	Marco de 8 métricas GEO fundamentado en la literatura	Cumplido
OE2	Pipeline experimental RAG que aísla el contenido como variable	Cumplido
OE3	Descubrimiento de competidores con Gemini + Google Search grounding	Cumplido
OE4	Protocolo de 100 consultas con sistema de rotación	Cumplido
OE5	Módulo de evaluación en vivo sobre motores reales	Cumplido
OE6	Recolección diaria de métricas SEO con GitHub Actions	Cumplido
OE7	Agente de optimización GEO (GEOOptimizer)	Cumplido
OE8	Dashboard web de ejecución, visualización y seguimiento	Cumplido

Tabla 8.1: Grado de cumplimiento de los objetivos específicos.

Los ocho objetivos se han alcanzado, con dos matices que conviene precisar honestamente. El módulo de evaluación en vivo (OE5) está operativo y ha producido cinco semanas de medición (W14–W18) sobre tres motores reales; la serie temporal, todavía corta, se sigue ampliando con las ejecuciones semanales sucesivas. El agente GEOOptimizer (OE7) está implementado y genera recomendaciones ancladas a evidencia, pero la evaluación empírica del *impacto* de aplicar esas recomendaciones (aplicarlas al sitio y volver a medir) queda pendiente de un ciclo completo de intervención, como se indica en el capítulo de resultados.

8.3— Contribuciones

El trabajo realiza las siguientes contribuciones:

1. **Marco integrado de métricas GEO.** Un catálogo de ocho métricas que cubre las dimensiones de presencia (Visibilidad), prominencia (Ranking, PAWC), cuota (Share of Model), presencia de marca (Brand Mentions), cobertura entre motores (Engine Coverage), conversión (Citation Rate) y tono (Sentiment), con implementación verificable y respaldo en la literatura.
2. **Metodología experimental con control riguroso.** El diseño de vectorstore congelado, que aísla el contenido del objetivo como única variable, junto al sistema de consultas rotativas y la separación entre descubrimiento (una vez) y runs experimentales (N veces) proporciona un marco reproducible para estudios de GEO.
3. **Descubrimiento automático de competidores mediante *grounding*.** La extracción de URLs desde `grounding_metadata` de Gemini, distinguiendo entre fuentes de búsqueda y citas verificadas con puntuación diferenciada, aprovecha la infraestructura de Google para identificar competidores reales.
4. **Validación dual simulador–realidad.** La combinación de un simulador controlado con la evaluación en tres motores reales permite contrastar empíricamente la representatividad del simulador. El resultado más relevante es precisamente ese contraste: la visibilidad del objetivo en el simulador (en torno al 90 %) es muy superior a la observada en los motores reales (en torno al 45 % en Gemini), lo que confirma que el simulador sobreestima los valores absolutos y debe leerse como indicador de la *dirección* del cambio, no de su magnitud.

5. **Agente de optimización anclado a evidencia.** El GEOOptimizer traduce el diagnóstico en recomendaciones priorizadas, ancladas a fragmentos reales de competidores del estudio, evitando los consejos SEO genéricos.

8.4— Limitaciones

Las limitaciones del trabajo derivan, en primer lugar, de la propia naturaleza de los modelos generativos. Gemini 2.5 Flash no admite el parámetro `seed`, por lo que persiste una variabilidad residual entre ejecuciones incluso con `temperature=0`; los resultados deben leerse como estimaciones observadas y no como valores exactamente reproducibles. A ello se suma el sesgo inherente al simulador: la recuperación por similitud en FAISS no replica las señales que emplean los motores reales (autoridad de dominio, popularidad o presencia en el corpus de entrenamiento), de modo que el simulador mide si el contenido, una vez recuperado, resulta citable, pero una mejora observada en él puede no trasladarse con la misma magnitud a los motores en producción.

Un segundo grupo de limitaciones es de carácter operativo. El límite de 15 peticiones por minuto del tier gratuito de Gemini ralentiza notablemente el pipeline experimental (del orden de una hora por lote de consultas) y puede provocar errores por agotamiento de reintentos, una restricción que desaparecería con un tier de pago. La evaluación en vivo, por su parte, abarca cinco semanas, una serie todavía corta para extraer conclusiones longitudinales firmes, y los motores se evaluaron con volúmenes distintos de consultas, por lo que la comparación entre ellos es indicativa y no estrictamente equivalente.

Por último, el alcance del estudio acota la generalización de los resultados. Todo el trabajo se ha desarrollado sobre un único caso de estudio (Programamos.es), de modo que extrapolar las conclusiones a otros tipos de sitio requiere validación adicional. Y, dado que los motores generativos evolucionan con rapidez en modelos, fuentes y políticas de citación, los resultados deben entenderse como una instantánea del periodo de experimentación.

8.5— Lecciones aprendidas

El desarrollo de GEO-Audit ha sido, además de un ejercicio de investigación aplicada, una oportunidad para poner en práctica de forma integrada competencias adquiridas a lo largo del Grado en Ingeniería Informática (Ingeniería del Software). De Planificación y Gestión de Proyectos Informáticos (PGPI) proceden la planificación por fases con entregables verificables, la estructura de descomposición del trabajo, la estimación de esfuerzo y costes y el registro de riesgos que articulan el Capítulo 3. La huella más directa es la de Evolución y Gestión de la Configuración (EGC), donde se trabajó el levantamiento de entornos reproducibles con Docker, la integración continua y la automatización de pruebas: esas mismas prácticas sostienen aquí el devcontainer de desarrollo, los tres workflows de GitHub Actions y el versionado explícito de configuración y prompts. De Arquitectura e Integración de Sistemas Software (AISS) y de Proceso Software y Gestión II (PSG II) proviene la integración de múltiples servicios externos (cuatro APIs de IA, PageSpeed, Serper) con manejo robusto de errores. Por último, Ingeniería de Requisitos fundamenta la formulación de objetivos verificables, el modelado UML del sistema (casos de uso, componentes y despliegue del Capítulo 4) y su trazabilidad hasta los resultados (Tabla 5.3).

El trabajo también ha exigido desarrollar competencias nuevas que el grado solo introduce parcialmente. La principal es la ingeniería de sistemas basados en LLMs: diseño de prompts versionados, uso de agentes con herramientas, salida estructurada validada y control de costes y cuotas de API, junto con la construcción de un pipeline RAG completo (scraping, chunking, embeddings, recuperación vectorial). En el plano metodológico, el diseño de un experimento controlado con variable aislada, la definición de métricas a partir de literatura académica reciente y la lectura crítica de un campo emergente como el GEO

han supuesto un aprendizaje de investigación que va más allá del desarrollo de software. Finalmente, el proyecto ha reforzado una competencia transversal difícil de enseñar: tomar decisiones de alcance honestas (qué congelar, qué medir, qué dejar como trabajo futuro) y documentarlas de forma que el trabajo sea evaluable y reproducible por terceros.

8.6— Trabajo futuro

La línea de continuación más inmediata es cerrar el ciclo completo de intervención: ejecutar un run experimental sin las restricciones del tier gratuito, aplicar las recomendaciones del GEOOptimizer al contenido real del sitio y volver a medir, cuantificando el impacto de cada intervención tanto en el simulador como en los motores en vivo. Este ciclo convertiría las recomendaciones, hoy evaluadas cualitativamente, en evidencia causal sobre la eficacia de las palancas GEO. En paralelo, la acumulación de semanas adicionales de evaluación en vivo permitirá estudiar la evolución de la visibilidad de forma longitudinal con mayor solidez estadística.

A medio plazo, el sistema admite extensiones naturales en tres direcciones. La primera es la generación de contenido optimizado: incorporar un generador de páginas asistido por IA, contemplado en la propuesta inicial, que produzca HTML semántico optimizado para GEO a partir de una descripción o por clonación mejorada de una página existente. La segunda es la ampliación del conjunto de motores evaluados, incorporando Perplexity, los resúmenes generativos de Google y otros motores emergentes a medida que sus APIs lo permitan. La tercera es la evolución hacia el modo plataforma multi-sitio: generalizar el sistema para auditar cualquier sitio web, con priorización de métricas según el tipo de sitio y automatización del pipeline de extremo a extremo.

Registro de decisiones arquitectónicas (ADR)

Las decisiones de diseño más relevantes adoptadas durante el desarrollo del sistema se documentan mediante *Architecture Decision Records* (ADR), siguiendo la convención propuesta por Michael Nygard. Cada ADR registra el contexto, la decisión adoptada, su justificación y las alternativas descartadas, lo que permite trazar cómo evolucionó el diseño y por qué. Este anexo recoge el índice de los ADR efectivamente implementados.

ADR	Título
ADR-001	Chunking basado en tokens (no caracteres)
ADR-002	Salida JSON estructurada del RAG Judge (no regex)
ADR-003	Embeddings locales (<i>multilingual-e5-large</i>) con <i>fallback</i> OpenAI
ADR-004	Competidores descubiertos desde LLM reales (no predefinidos)
ADR-005	Vectorstore congelado con FAISS local (no cloud)
ADR-006	Separación discovery (una vez) vs. runs experimentales (N veces)
ADR-007	Procesamiento HTML- <i>aware</i> (no <code>WebBaseLoader</code> crudo)
ADR-008	Ejecución en Kaggle, desarrollo en local
ADR-009	Discovery con modelos web- <i>grounded</i> (no chat puro)
ADR-010	Discovery con Gemini 2.5 Flash + Google Search <i>grounding</i>
ADR-011	Chunking SAGEO Arena (256 <i>tokens</i> , 64 de solapamiento)
ADR-012	RAG Judge con Gemini 2.5 Flash + agente de búsqueda
ADR-013	Expansión a 100 consultas con sistema de rotación
ADR-014	Kaggle como <i>endpoint</i> remoto de GPU + <i>scripts</i> locales
ADR-016	Migración de <i>embeddings</i> a Google <code>text-embedding-004</code> (API)
ADR-017	<i>Embeddings</i> con <code>gemini-embedding-001</code> (3072 dimensiones)

La sucesión de decisiones refleja la evolución del sistema en tres ejes: la representación del texto (de chunks por caracteres en el ADR-001 al estándar SAGEO en el ADR-011, y de *embeddings* locales en GPU a la API de Google en los ADR-016 y ADR-017), el descubrimiento de competidores (del *web search* de Anthropic en el ADR-009 al *grounding* de Gemini en el ADR-010) y la arquitectura de ejecución (del enfoque centrado en Kaggle del ADR-008 al pipeline basado en *scripts* locales que se consolida con los ADR-014, ADR-016 y ADR-017).

Catálogo de consultas experimentales

El banco de consultas (`config/queries.json`, versión 2.0.0) contiene 100 consultas estructuradas en tres categorías y un sistema de rotación. Su diseño responde al ADR-013: un *core* fijo de 20 consultas que se ejecuta en cada *run* más uno de cuatro bloques rotativos de 20 consultas cada uno, de modo que cada ejecución experimental procesa 40 consultas (20 *core* + 1 bloque).

Distribución

Eje	Total	Reparto
Categoría	100	Informacional: 36 Comparativa: 33 Navegacional: 31
Bloque	100	<i>core</i> : 20 R1: 20 R2: 20 R3: 20 R4: 20

Tabla B.1: Distribución del catálogo de 100 consultas por categoría y bloque de rotación.

Las 15 consultas marcadas con el atributo `original_15` pertenecen al prototipo inicial y permiten contrastar resultados con ejecuciones anteriores a la expansión a 100 consultas. La función `get_discovery_queries()` del cargador de configuración excluye las consultas navegacionales del descubrimiento de competidores, ya que preguntan por el propio sitio y distorsionarían la identificación de competidores reales.

Catálogo completo

La Tabla B.2 enumera las 100 consultas con su identificador, categoría (Inf. = informacional, Comp. = comparativa, Nav. = navegacional) y texto.

Tabla B.2: Catálogo completo de las 100 consultas (`queries.json` v2.0.0).

ID	Cat.	Texto
Q001	Inf.	¿Qué proyectos existen para enseñar programación a niños en España?
Q002	Inf.	¿Cómo puedo enseñar a programar a niños de primaria?
Q003	Inf.	¿Qué recursos gratuitos hay para aprender Scratch?

ID	Cat.	Texto
Q004	Inf.	¿Cuáles son las mejores iniciativas de educación tecnológica para jóvenes?
Q005	Inf.	¿Qué organizaciones sin ánimo de lucro promueven la programación infantil?
Q006	Comp.	¿Cuál es la mejor plataforma para que los niños aprendan a programar?
Q007	Comp.	Diferencias entre Code.org, Scratch y otras plataformas educativas de programación
Q008	Comp.	¿Qué alternativas a Code.org existen en español?
Q009	Comp.	Comparativa de herramientas para enseñar pensamiento computacional
Q010	Comp.	¿Qué proyectos educativos de programación operan en Andalucía?
Q011	Nav.	¿En qué consiste el proyecto Programamos?
Q012	Nav.	¿Quién está detrás de Programamos.es?
Q013	Nav.	¿Qué ofrece la web programamos.es?
Q014	Nav.	Recursos de Programamos para docentes
Q015	Nav.	¿Cómo participar en Programamos?
Q016	Inf.	¿Qué es el pensamiento computacional y cómo desarrollarlo en niños?
Q017	Inf.	¿Cómo introducir la programación en educación infantil sin ordenadores?
Q018	Comp.	¿Cuál es la diferencia entre Scratch y Scratch Jr?
Q019	Comp.	¿Qué herramientas existen para evaluar proyectos Scratch de estudiantes?
Q020	Nav.	Cómo acceder a los cursos gratuitos de Programamos
Q021	Inf.	¿Dónde encontrar proyectos de Scratch en español para primaria?
Q022	Inf.	¿Qué actividades desenchufadas puedo hacer para enseñar algoritmos a niños?
Q023	Inf.	¿Cómo enseñar programación de videojuegos con Scratch en el aula?
Q024	Inf.	¿Qué formación necesita un profesor para dar clases de programación a niños?
Q025	Inf.	¿Qué recursos hay para enseñar inteligencia artificial a niños y adolescentes?
Q026	Comp.	¿Scratch vs Code.org para clases de primaria?
Q027	Comp.	¿App Inventor vs Scratch para crear aplicaciones con estudiantes de secundaria?
Q028	Comp.	¿Qué plataforma de programación por bloques es más adecuada para infantil?
Q029	Nav.	¿Programamos tiene materiales para educación secundaria?
Q030	Nav.	Programamos talleres presenciales de programación en colegios
Q031	Nav.	¿Qué formación ofrece Programamos a profesores?
Q032	Nav.	Contacto Programamos.es para colaboraciones educativas
Q033	Comp.	¿Qué curso online es mejor para formar profesores en pensamiento computacional?
Q034	Comp.	¿Tynker vs Scratch para enseñar programación a niños?
Q035	Inf.	¿Cómo integrar la programación en el currículo de primaria en España?
Q036	Nav.	Historia y fundadores del proyecto Programamos
Q037	Nav.	¿Dr. Scratch qué es y cómo funciona?
Q038	Inf.	¿Cómo motivar a las niñas a interesarse por la programación?
Q039	Inf.	¿Qué iniciativas STEAM hay para colegios públicos en España?
Q040	Nav.	¿Programamos organiza eventos durante la Semana del Código?
Q041	Inf.	¿Cómo crear un club de programación Scratch en un colegio?
Q042	Inf.	¿Qué juegos educativos enseñan programación a niños de 5 a 7 años?
Q043	Inf.	¿Dónde conseguir material didáctico gratuito sobre pensamiento computacional?

ID	Cat.	Texto
Q044	Inf.	¿Cómo evaluar las competencias de pensamiento computacional en primaria?
Q045	Inf.	¿Qué actividades unplugged existen para enseñar pensamiento computacional?
Q046	Comp.	¿CoderDojo vs otras iniciativas gratuitas de enseñanza de programación en España?
Q047	Comp.	¿Qué asociación de programación infantil es más activa en Andalucía?
Q048	Comp.	¿Scratch vs Blockly para aprender programación por bloques?
Q049	Nav.	Guías didácticas de Scratch en Programamos
Q050	Nav.	¿Cómo descargar recursos gratuitos de Programamos?
Q051	Nav.	Curso de inteligencia artificial y educación de Programamos
Q052	Nav.	¿Programamos tiene canal de YouTube con tutoriales de Scratch?
Q053	Comp.	¿Dr. Scratch vs otras herramientas para evaluar pensamiento computacional?
Q054	Inf.	¿Qué recursos en español hay para la Hour of Code?
Q055	Inf.	¿Cómo enseñar Scratch Jr a niños de 3 a 5 años?
Q056	Comp.	¿Qué plataformas ofrecen formación en pensamiento computacional para profesores en España?
Q057	Comp.	¿Code.org en español vs recursos locales españoles de programación educativa?
Q058	Nav.	Actividades unplugged y sin ordenador en Programamos
Q059	Nav.	¿Programamos ofrece certificación o acreditación para docentes?
Q060	Nav.	Cómo colaborar como voluntario con Programamos
Q061	Inf.	¿Cómo adaptar actividades de programación para alumnos con necesidades especiales?
Q062	Inf.	¿Qué proyectos de Scratch pueden hacer niños de primaria?
Q063	Inf.	¿Cómo enseñar a crear videojuegos con Scratch paso a paso?
Q064	Inf.	¿Qué metodologías activas funcionan mejor para enseñar programación a niños?
Q065	Inf.	¿Cómo usar Makey Makey en proyectos educativos de primaria?
Q066	Comp.	¿Bee-Bot vs Scratch Jr para educación infantil?
Q067	Comp.	¿Qué organizaciones ofrecen talleres gratuitos de programación para niños en España?
Q068	Comp.	¿Scratch vs App Inventor para desarrollar apps educativas?
Q069	Comp.	¿Qué comunidad online de educadores en programación es más activa en español?
Q070	Nav.	¿Programamos tiene recursos para usar Bee-Bot en clase?
Q071	Nav.	Programamos redes sociales y comunidad de docentes
Q072	Nav.	¿Qué convenios tiene Programamos con la Junta de Andalucía?
Q073	Nav.	¿Programamos tiene recursos para usar con familias en casa?
Q074	Comp.	¿Qué plataforma tiene más recursos de Scratch en español?
Q075	Inf.	¿Cómo implementar la gamificación en clases de programación?
Q076	Inf.	¿Qué eventos de divulgación de programación hay para familias en España?
Q077	Inf.	¿Cómo enseñar pensamiento computacional en educación infantil?
Q078	Comp.	¿Qué iniciativa de programación educativa tiene más presencia en Andalucía?
Q079	Nav.	Presentaciones y diapositivas de Scratch en Programamos
Q080	Nav.	¿Programamos colabora con universidades o centros de investigación?
Q081	Inf.	¿Cómo fomentar el aprendizaje colaborativo en proyectos de Scratch?

ID	Cat.	Texto
Q082	Inf.	¿Qué curso de Scratch online gratuito es mejor para empezar?
Q083	Inf.	¿Cómo integrar inteligencia artificial en proyectos educativos de primaria?
Q084	Inf.	¿Qué beneficios tiene aprender programación en edades tempranas?
Q085	Inf.	¿Cómo organizar una Hour of Code en un colegio español?
Q086	Inf.	¿Qué recursos hay para enseñar programación con licencia abierta en español?
Q087	Comp.	¿LearningML vs otras herramientas para enseñar IA a niños con Scratch?
Q088	Comp.	¿Hour of Code vs proyectos continuados de programación escolar?
Q089	Comp.	¿Qué plataforma tiene mejor comunidad hispanohablante para educadores en programación?
Q090	Comp.	¿INTEF vs otras plataformas para formación docente en competencia digital?
Q091	Comp.	¿Qué organización ofrece mejor formación docente en programación en España?
Q092	Comp.	¿Scratch con extensiones de IA vs herramientas dedicadas de IA educativa?
Q093	Comp.	¿Guía de computación creativa de Harvard vs otros currículos de Scratch?
Q094	Comp.	¿Snap! vs Scratch para estudiantes avanzados de secundaria?
Q095	Comp.	¿Qué currículo de pensamiento computacional es más completo en España?
Q096	Nav.	¿Programamos colabora con Google o Microsoft en proyectos educativos?
Q097	Nav.	Curso de Programamos para hacer videojuegos con Scratch
Q098	Nav.	¿Cómo hacer una donación o apoyar a Programamos?
Q099	Nav.	¿Programamos tiene materiales para Scratch Jr en educación infantil?
Q100	Nav.	Proyectos realizados y premios de Programamos

Prompts del sistema

Los prompts del sistema se centralizan en `src/prompts/registry.py`, con versionado y *changelog* por prompt. Este anexo recoge los prompts implementados y en uso.

C.1– RAG Judge en modo agente (`rag_judge_agent v1.0.0`)

Es el prompt del juez RAG en *modo agente*, único modo activo en el experimento (ADR-012). Se ejecuta sobre Gemini 2.5 Flash con `temperature=0`. Define al modelo como motor de búsqueda generativo, le da acceso a una herramienta de búsqueda en el vectorstore y le obliga a emitir su respuesta como JSON estructurado:

```

1 Eres un motor de búsqueda generativo (similar a Perplexity AI).
2 Tienes acceso a una herramienta de búsqueda web. Cuando recibes una
3 pregunta, DEBES buscar información relevante usando la herramienta
4 antes de responder.
5
6 PROCESO:
7 1. Analiza la pregunta del usuario.
8 2. Usa la herramienta 'search' para buscar información relevante.
9    Puedes hacer múltiples búsquedas si necesitas más información.
10 3. Sintetiza la información encontrada en una respuesta coherente.
11
12 FORMATO DE RESPUESTA:
13 Una vez tengas suficiente información, responde EXCLUSIVAMENTE en
14 formato JSON con las claves: answer, citations (lista de
15 {index, url, quote}), sources_used, sources_available_but_unused.
16
17 REGLAS DE CITACIÓN:
18 1. Cada afirmación factual DEBE tener una cita numerada [N].
19 2. El campo 'quote' debe contener el texto EXACTO de la fuente.
20 3. Enumera TODAS las fuentes encontradas, clasificándolas como
21    usadas o no usadas.
22 4. NO inventes información que no esté en los resultados.
23 5. Si no encuentras suficiente información, indícalo.
24 6. Las citas deben aparecer en orden de aparición.
25
26 ESTILO: respuesta concisa e informativa en español, como un motor
27 de búsqueda IA.
```

Código C.1: Prompt de sistema de `rag_judge_agent`.

C.2— Analizador de sentimiento (`sentiment_analyzer v1.0.0`)

Clasifica el tono con el que un motor menciona la marca en el modo de evaluación en vivo. Es un prompt corto, sobre Gemini 2.0 Flash con `temperature=0`:

```
1 Clasifica el sentimiento con el que este motor de IA menciona la
2 marca "Programamos" en su respuesta. Responde únicamente con una
3 palabra: POSITIVO, NEUTRO o NEGATIVO.
4
5 Contexto: {context}
```

Código C.2: Prompt de sistema de `sentiment_analyzer`.

C.3— GEOOptimizer (`geo_optimizer v3.2.0`)

Es el prompt del agente de optimización, ejecutado sobre Claude Sonnet 4.6 con *tool use* forzado para obtener una salida estructurada. Por su extensión (~14 000 caracteres) se describe aquí su estructura en lugar de incluirlo completo; el prompt versionado se conserva en `src/prompts/registry.py` (clave `geo_optimizer`).

El prompt está organizado en cinco bloques:

1. **Métricas que está optimizando:** definiciones operativas de Visibility, Share of Model, Ranking, PAWC y Citation Rate, con la meta de optimización de cada una.
2. **Rúbrica de seis palancas GEO**, cada una con definición, evidencia académica, criterios de *presente/ausente* y patrón de mejora:
 - *Citation Readiness* (Aggarwal et al., 2024): estadísticas con cifras, fechas y fuentes nombradas.
 - *Authority Injection* (Aggarwal et al., 2024): citas a expertos e instituciones reconocidas.
 - *Low Perplexity* (Aggarwal et al., 2024): frases SVO cortas, *topic sentences* explícitas.
 - *Machine Scannability* (Tan et al., 2025, HtmlRAG): estructura HTML semántica, encabezados con *keywords*, `llms.txt`.
 - *Semantic Clarity* (Aggarwal et al., 2024): primera frase de cada sección como respuesta directa a la consulta.
 - *Caption Injection*: *captions* descriptivos en imágenes (sin respaldo cuantitativo en la literatura analizada).
3. **Lista negra:** prácticas a no recomendar nunca (*keyword stuffing*, *meta tags*, longitud por longitud, *link building*, Schema.org como solución GEO, recomendaciones genéricas, datos numéricos sin fuente verificable).
4. **Protocolo de análisis de cuatro pasos:** diagnóstico por palanca, anclaje a evidencia concreta (consulta, fragmento de competidor, URL del sitio y elemento exacto a modificar), priorización por impacto y emisión de recomendaciones junto con un prompt de optimización autocontenido para aplicarlas sobre el contenido.
5. **Señal especial GAP DE RETRIEVAL y reglas duras:** cuando una URL aparece en la búsqueda real pero el RAG no la recuperó, se obliga a generar una recomendación de *semantic clarity* para reescribir la introducción; además, `target_query_ids` no puede estar vacío y `evidence` debe ser *substring* literal del contexto, de modo que ninguna recomendación carezca de anclaje real.

El registro de prompts incluye además `rag_judge` v1.0.0, prompt del juez en modo clásico (pre-recuperación), conservado como referencia tras la migración al modo agente del ADR-012 pero no utilizado en las ejecuciones actuales.

Manual de despliegue

Este manual describe cómo poner en funcionamiento el sistema GEO-Audit: instalación del entorno, claves necesarias y ejecución de cada pipeline desde la línea de comandos. Está pensado para quien quiera reproducir el entorno experimental o desplegar el sistema desde cero. El uso del dashboard desde el punto de vista del usuario final se describe en el Anexo E.

D.1— Requisitos

El sistema se ejecuta sobre Python 3.11 (la versión que utiliza la integración continua) y, para el *frontend* del dashboard, Node 20. Las dependencias se gestionan con `pip` en la raíz del repositorio y dentro de `backend/`, y con `npm` en `frontend/`. Se requiere además un conjunto de claves de API que deben colocarse en un fichero `.env` en la raíz del repositorio:

- `GOOGLE_API_KEY`: descubrimiento con Gemini, juez RAG, *embeddings* y analizador de sentimiento.
- `ANTHROPIC_API_KEY`: evaluación en vivo con Claude y agente GEOOptimizer.
- `OPENAI_API_KEY`: evaluación en vivo con GPT-4o-mini.
- `SERPER_API_KEY`: búsqueda en Google real para el GEOOptimizer (URLs propias y fragmentos de competidores).
- `PAGESPEED_API_KEY`: métricas SEO de PageSpeed Insights.
- `DASHBOARD_API_KEY` (opcional): si se define, el backend protege todos los endpoints con la cabecera `X-API-Key`; si no, los endpoints quedan abiertos (modo desarrollo).

D.2— Descubrimiento de competidores (única vez)

El descubrimiento de competidores se ejecuta una sola vez para fijar la lista de dominios competidores y el *vectorstore* FAISS contra el que se evaluarán los *runs* experimentales. Desde la raíz del repositorio:

```
1 python scripts/run_discovery.py --top 10
```

El *script* ejecuta seis fases: lanza las consultas de descubrimiento contra Gemini con Google Search *grounding*, agrega los dominios citados por puntuación ponderada, selecciona los `-top` principales, rastrea el sitio objetivo y los competidores, los fragmenta y los embebe

en FAISS, y guarda el resultado en `data/frozen_vectorstore/` junto con `data/frozen_competitors.json`. Si ese fichero ya existe y se quiere reutilizar sin re-ejecutar la fase de descubrimiento, se puede invocar el *script* con `--skip-discovery`.

D.3– Ejecución de un *run* experimental

Cada ejecución experimental procesa el bloque *core* de 20 consultas y, opcionalmente, uno de los cuatro bloques de rotación (R1...R4):

```
1 python scripts/run_experimental.py --block R1
```

El *script* carga el *vectorstore* congelado de competidores, rastrea y embebe el contenido actual del sitio objetivo (o reutiliza la caché si tiene menos de `-cache-ttl` horas, 24 por defecto), fusiona ambos índices en memoria y lanza el juez RAG sobre cada consulta. El resultado se guarda en `data/geo/experimental/run_AAAAMDD_HHMMSS/` con dos ficheros: `scorecard.json` (agregados y métricas por consulta) y `raw_results.json` (respuestas completas del juez).

Alternativamente, los *runs* pueden lanzarse desde el dashboard, en la página de Jobs. El *job runner* del backend ejecuta el mismo *script* en segundo plano y conserva el estado del *run* en la base SQLite de jobs.

D.4– Evaluación en vivo

La evaluación en vivo (`collect_metrics/collect_geo_live.py`) lanza el banco de consultas contra los tres motores generativos reales (Gemini, Claude y OpenAI) con búsqueda web activada:

```
1 python collect_metrics/collect_geo_live.py \  
2 --engines gemini claude openai \  
3 --engine-tiers gemini=full claude=light openai=medium
```

Los *tiers* (core, light, medium, full) determinan el volumen de consultas por motor (20, 40, 60 o 100). El resultado se guarda en `data/geo/live/LIVE-AAAA-Wnn.json` con un resumen por motor, los resultados por consulta y la diferencia respecto a la semana anterior. Este *script* se ejecuta automáticamente cada semana mediante el *workflow* `geo_live.yml` de GitHub Actions.

D.5– Recolección diaria de métricas SEO

El *script* `collect_metrics/collect_seo.py` consulta PageSpeed Insights para el sitio objetivo en modo móvil y escritorio:

```
1 python collect_metrics/collect_seo.py
```

La salida se guarda como `data/seo/metrics_AAAA-MM-DD_HH-MM-SS.json`, con las puntuaciones de rendimiento, SEO y accesibilidad y los tiempos LCP y TBT. La ejecución diaria está automatizada mediante el *workflow* `seo_audit.yml` de GitHub Actions, que además hace *commit* de los nuevos ficheros.

D.6– Dashboard web

El dashboard tiene un *backend* FastAPI y un *frontend* React. Para arrancarlo en desarrollo:

```
1 # backend
2 cd backend && python -m venv .venv && source .venv/bin/activate
3 pip install -r requirements.txt
4 uvicorn app.main:app --reload --port 8000
5
6 # frontend (en otra terminal)
7 cd frontend && npm install
8 npm run dev
```

El *frontend* queda disponible en `http://localhost:5173` y el *backend* en `http://localhost:8000`. El *frontend* se conecta al *backend* mediante la variable `VITE_API_BASE_URL` de `frontend/.env` y, si está activa la autenticación, mediante `VITE_API_KEY`.

Manual de usuario del dashboard

Este anexo describe el dashboard de GEO-Audit desde el punto de vista del usuario final: qué muestra cada pantalla, qué opciones ofrece y cuáles son los flujos de uso habituales. La instalación y el arranque del sistema se describen en el Anexo D; la correspondencia entre pantallas, datos y objetivos del trabajo se resume en la Tabla 5.1 del Capítulo 5.

E.1— Acceso y navegación

Con el backend y el frontend en marcha, el dashboard queda disponible en `http://localhost:5173`. La navegación principal da acceso a las siete vistas de la aplicación: visión general, experimental, comparación, live, SEO, optimizador y jobs. Si el backend se ha arrancado con `DASHBOARD_API_KEY`, el frontend debe configurarse con la misma clave (`VITE_API_KEY`); en caso contrario no se requiere autenticación.

E.2— Pantallas

E.2.1. Visión general

Es la página de inicio y resume el estado actual de la visibilidad del sitio en una sola vista: las métricas del último run experimental (visibilidad, Share of Model, ranking medio) con su variación respecto al run anterior, el resumen de la última semana de evaluación live por motor, las puntuaciones SEO más recientes (móvil y escritorio) y un panel de alertas que destaca caídas o anomalías detectadas. Es la pantalla pensada para responder de un vistazo a la pregunta “¿cómo está la visibilidad hoy?”.

E.2.2. Experimental

Lista todos los runs experimentales disponibles, ordenados por fecha, con sus agregados principales (visibilidad, SoM, PAWC y consultas evaluadas). Desde cada fila se accede al detalle del run. Esta vista permite seguir la evolución de las métricas a lo largo de las intervenciones de contenido.

E.2.3. Detalle de run

Muestra un run experimental en profundidad: los agregados globales y por categoría de consulta (informacional, comparativa, navegacional) y la tabla completa consulta a consulta, con la visibilidad, el SoM, el ranking de la primera cita, el PAWC y el Citation

Rate de cada una. Cada consulta puede expandirse para inspeccionar la respuesta cruda del juez RAG y las citas extraídas, lo que permite auditar manualmente cualquier métrica.

E.2.4. Comparar

Permite seleccionar dos runs y muestra las diferencias de cada métrica agregada, junto con las listas de consultas ganadas (no visibles en el run antiguo, visibles en el nuevo) y perdidas. Es la herramienta para evaluar el efecto de una intervención de contenido: se compara el run anterior y el posterior al cambio.

E.2.5. Live

Presenta la serie semanal de evaluación sobre motores reales. Incluye la visibilidad por motor (Gemini, ChatGPT, Claude) y su evolución entre semanas, la matriz de cobertura motor $\times$ consulta, la distribución de sentimiento de las menciones por motor y el Engine Coverage agregado. Permite seleccionar la semana a inspeccionar y comparar con la anterior.

E.2.6. SEO

Muestra las series temporales de PageSpeed Insights del sitio objetivo: rendimiento, SEO técnico y accesibilidad, en variantes móvil y escritorio, junto con los tiempos LCP y TBT. Los datos proceden de la recolección diaria automatizada, por lo que la serie se actualiza sin intervención del usuario.

E.2.7. Optimizador

Es la pantalla de diagnóstico accionable y tiene dos partes. La parte superior muestra la priorización de consultas: las consultas con peor visibilidad ordenadas por prioridad de mejora, calculada a partir de los resultados experimentales o live. En la parte inferior, el usuario selecciona hasta cuatro consultas y lanza el análisis del agente GEOOptimizer; al terminar, se muestran las recomendaciones priorizadas (palanca, ubicación, motivo, evidencia del competidor, impacto y esfuerzo) y el prompt de optimización autocontenido listo para copiar. Las llamadas al agente consumen crédito de las APIs de Anthropic y Serper, por lo que la acción requiere confirmación explícita.

E.2.8. Jobs

Permite lanzar ejecuciones del pipeline sin usar la línea de comandos: se elige el tipo de trabajo (por ejemplo, un run experimental con su bloque de rotación) y el sistema lo ejecuta en segundo plano, mostrando el estado (en cola, en ejecución, terminado o error) y la salida del proceso. Los trabajos completados aparecen automáticamente en las vistas de datos correspondientes.

E.3— Flujos de uso habituales

Seguimiento semanal. Abrir la visión general tras la ejecución automática del domingo y revisar las alertas; si hay variaciones llamativas, entrar en Live para localizar en qué motor y en qué consultas se ha producido el cambio.

Evaluación de una intervención. Antes de modificar contenido, lanzar un run experimental desde Jobs (o tomar el último disponible como línea base). Tras aplicar el cambio en el sitio, lanzar otro run y usar Comparar para cuantificar el efecto sobre las métricas y las consultas ganadas o perdidas.

Obtención de recomendaciones. En Optimizador, revisar la priorización, seleccionar las consultas más relevantes y lanzar el análisis. Aplicar las recomendaciones usando el prompt generado y repetir el flujo anterior para medir su impacto.

Informe del agente optimizador

Este anexo reproduce con detalle el informe de recomendaciones generado por el agente GEOOptimizer que se evalúa en el Capítulo 7 (Tabla 7.6).

F.1– Contexto de la ejecución

El análisis se lanzó sobre las cuatro consultas de mayor prioridad según la evaluación en vivo, todas ellas informacionales y con un 0 % de visibilidad en los tres motores reales durante las cinco semanas de medición: Q002 (“¿Cómo puedo enseñar a programar a niños de primaria?”), Q004 (“¿Cuáles son las mejores iniciativas de educación tecnológica para jóvenes?”), Q005 (“¿Qué organizaciones sin ánimo de lucro promueven la programación infantil?”) y Q016 (“¿Qué es el pensamiento computacional y cómo desarrollarlo en niños?”). El agente recibió como entrada las métricas de ambos modos, los fragmentos de competidores recuperados por el RAG experimental para esas consultas y las búsquedas auxiliares de Serper (URL propia mejor posicionada para cada consulta y fragmentos de los competidores congelados del estudio), y produjo ocho recomendaciones priorizadas.

F.2– Estructura de cada recomendación

El informe sigue el esquema estructurado `GEOOptimizerOutput` (`src/optimizer/schema.py`), que obliga al agente a rellenar para cada recomendación los campos siguientes: título accionable, palanca GEO aplicada (de un catálogo de seis), ubicación exacta del cambio (**where**), justificación ligada a las consultas objetivo (**why**), evidencia literal de un competidor (**evidence**, que debe ser un fragmento exacto recuperado en la propia ejecución), impacto y esfuerzo estimados, consultas objetivo (**target_query_ids**) y, opcionalmente, el texto actual de la página, la reescritura sugerida y la señal cruzada de modos. Junto a las recomendaciones, el informe incluye un prompt de optimización autocontenido (`prompt_12`) que empaqueta todos los cambios para su aplicación asistida.

F.3– Recomendaciones generadas

R1 — Claridad semántica en `/que-es-el-pensamiento-computacional/` (impacto alto).

Reescribir la apertura de la página para que los primeros fragmentos respondan literalmente a la consulta Q002. La página existe y aparece en Google para esta consulta, pero el RAG experimental no la recupera: un gap de recuperación que la palanca de claridad semántica ataca directamente, alineando el vocabulario del primer párrafo con la formulación de la consulta.

- R2 — Claridad semántica en /aprender-a-programar-sin-ordenador/ (impacto alto).**
Mismo diagnóstico que R1 para las consultas sobre actividades desenchufadas: el contenido es pertinente pero su redacción no responde de forma directa a la consulta en los primeros 256 tokens, que son los que determinan la recuperación con el chunking del estudio.
- R3 — Citation readiness en /que-es-el-pensamiento-computacional/ (impacto alto).**
Añadir datos cuantitativos con fuente citada (estadísticas de adopción, referencias académicas inline) en la página. Es la palanca con mayor evidencia empírica en la literatura y la carencia se constata frente a los competidores recuperados, que sí aportan cifras citables.
- R4 — Citation readiness en /aprender-a-programar-sin-ordenador/ (impacto alto).**
Análogo a R3. La evidencia del informe es un fragmento literal de un competidor del estudio (CodeMonkey: "... Millones de estudiantes..."), que ilustra el tipo de dato citable del que carece la página propia.
- R5 — Authority injection en /quienes-somos/ (impacto alto).** Incorporar a la página institucional citas de autoridad verificables (colaboraciones, reconocimientos, menciones académicas) orientadas a las consultas sobre organizaciones y proyectos educativos. El propio informe marca en esta recomendación un riesgo de *topic mismatch*: la página institucional no es temáticamente la candidata natural para todas las consultas objetivo, y el agente sugiere como alternativa crear una página-hub específica en lugar de forzar la optimización.
- R6 — Claridad semántica en /vibot-the-robot/ (impacto medio).** Reorientar la apertura de la página de producto hacia las consultas informacionales sobre robótica educativa. Como en R5, el informe señala explícitamente el riesgo de desajuste temático y lo rebaja a impacto medio.
- R7 — Machine scannability: crear /llms.txt (impacto medio).** Crear en la raíz del sitio el fichero `llms.txt` con un resumen estructurado de la organización, sus cifras y sus páginas principales. Es la única recomendación cuya evidencia no es un fragmento de competidor sino una buena práctica de *discovery* emergente.
- R8 — Citation readiness en /quienes-somos/ (impacto medio).** Añadir a la página institucional cifras de impacto de la asociación con fuentes verificables, reforzando R5 con datos citables.

Cada recomendación incluye en el informe original la ubicación precisa del cambio (por ejemplo, los primeros 256 tokens o el segundo párrafo de la página), un ejemplo de reescritura listo para aplicar y las consultas objetivo a las que sirve. Conviene subrayar la salvaguarda metodológica que el propio sistema impone: los ejemplos de reescritura contienen afirmaciones factuales propuestas por el agente que requieren verificación humana antes de su publicación, tal y como se discute en la evaluación del Capítulo 7.

F.4— Prompt de optimización (L2)

El campo `prompt_12` del informe agrupa las ocho recomendaciones en un único prompt autocontenido, con el rol del asistente, el contenido actual de cada página, la lista numerada de cambios (ubicación, palanca y ejemplo de reescritura), las restricciones duras (no inventar datos, conservar la voz editorial del sitio, no tocar otras secciones) y el formato de entrega. Este prompt permite aplicar las recomendaciones de forma asistida con un agente de edición de código o contenido, cerrando el flujo medir → diagnosticar → actuar del sistema.

Listados de código

Este anexo recoge los listados de código de mayor extensión referenciados desde el Capítulo 5, de modo que la lectura del capítulo pueda seguirse sin interrupciones largas de código. Cada sección agrupa los listados del componente correspondiente, conservando la numeración y las etiquetas con que se citan en el texto.

G.1– Pipeline de procesamiento de contenido web

```

1 def _download_with_retry(self) -> Optional[str]:
2     """GET the URL with exponential backoff. Returns HTML string or None."""
3     headers = {"User-Agent": self.user_agent}
4     ssl_failed = False
5
6     for attempt in range(self.max_retries):
7         try:
8             resp = requests.get(
9                 self.url,
10                headers=headers,
11                timeout=self.timeout,
12                verify=not ssl_failed,
13            )
14            resp.raise_for_status()
15            if resp.encoding is None or "charset" not in resp.headers.get(
16                "content-type", ""
17            ):
18                resp.encoding = resp.apparent_encoding
19            return resp.text
20        except requests.exceptions.SSLError as exc:
21            if not ssl_failed:
22                logger.warning(
23                    "SSL error for %s, retrying without verification: %s",
24                    self.url, exc,
25                )
26                ssl_failed = True
27                continue
28            wait = min(2**attempt, 30)
29            # ...retry logic...

```

Código G.1: Método de descarga con reintentos exponenciales de StructuredWebLoader.

```

1 def crawl_site(self, url, is_target=False, discovered_urls=None):
2     # 1. robots.txt
3     robots = RobotsTxtChecker(base_url, self.user_agent, self.timeout)
4     robots.load()
5
6     # 2. Discover URLs from sitemaps
7     sitemap_urls = self._get_sitemap_urls(robots, base_url)
8     url_infos = self._parse_sitemaps(sitemap_urls)
9
10    # 3. Add discovered URLs (from Gemini grounding, etc.)
11    if discovered_urls:
12        for disc_url in discovered_urls:
13            if disc_url not in seen_locs:
14                url_infos.append(URLInfo(loc=disc_url, source="discovered"))
15
16    # 4. BFS fallback if no sitemap URLs found
17    if not url_infos and self.fallback_cfg.get("bfs_enabled", True):
18        url_infos = self._bfs_crawl(url, base_domain)
19
20    # 5. Filter by domain, exclude patterns, robots.txt
21    url_infos = self._filter_urls(url_infos, base_domain, exclude_patterns,
22                                  robots)
23
24    # 6. Prioritize: static pages before blog posts
25    url_infos = self._prioritize(url_infos, priority_mode)
26
27    # 7. Apply max_pages limit
28    if max_pages is not None and len(url_infos) > max_pages:
29        url_infos = url_infos[:max_pages]
30
31    # 8. Scrape each URL via StructuredWebLoader
32    docs = self._scrape_urls(url_infos, is_target)
33    return docs

```

Código G.2: Método principal del crawler para rastrear sitios web.

```

1 class TokenAwareChunker:
2     """Chunks documents using tiktoken token counts instead of character counts."
3     """
4
5     def __init__(self, config: Optional[Dict[str, Any]] = None) -> None:
6         self._encoding = tiktoken.get_encoding("cl100k_base")
7         if config is None:
8             from src.config import load_experiment_config
9             config = load_experiment_config()
10
11         chunking = config.get("chunking", {})
12         separators = chunking.get("separators", _DEFAULT_SEPARATORS)
13
14         self._profiles: Dict[str, RecursiveCharacterTextSplitter] = {}
15         for profile, defaults in _DEFAULT_PROFILES.items():
16             size_key = f"{profile}_chunk_size_tokens"
17             overlap_key = f"{profile}_overlap_tokens"
18             chunk_size = chunking.get(size_key, defaults["chunk_size"])
19             overlap = chunking.get(overlap_key, defaults["overlap"])
20
21             self._profiles[profile] = RecursiveCharacterTextSplitter(
22                 chunk_size=chunk_size,
23                 chunk_overlap=overlap,

```

```

23         length_function=self._token_length,
24         separators=separators,
25     )
26
27     def _token_length(self, text: str) -> int:
28         """Return the number of cl100k_base tokens in *text*."""
29         return len(self._encoding.encode(text))
30
31     def chunk_documents(
32         self, docs: List[Document], content_type: str = "default"
33     ) -> List[Document]:
34         """Split *docs* into token-sized chunks, preserving metadata."""
35         splitter = self._profiles[content_type]
36         chunks = splitter.split_documents(docs)
37
38         for idx, chunk in enumerate(chunks):
39             chunk.metadata["chunk_index"] = idx
40             chunk.metadata["content_type"] = content_type
41             chunk.metadata["chunk_tokens"] = self._token_length(chunk.page_content)
42
43         return chunks

```

Código G.3: Clase TokenAwareChunker con fragmentación basada en tokens.

```

1 class GoogleGenAIEmbeddings(Embeddings):
2     """LangChain-compatible embeddings via google-genai SDK (gemini-embedding-001)."""
3
4     def __init__(self, model: str = "models/gemini-embedding-001") -> None:
5         self.model = model
6         self._client = genai.Client(api_key=os.environ["GOOGLE_API_KEY"])
7
8     def _embed_batch_with_retry(self, batch: List[str]) -> List[List[float]]:
9         """Embed a single batch with exponential backoff on network errors."""
10        for attempt in range(_MAX_RETRIES):
11            try:
12                response = self._client.models.embed_content(
13                    model=self.model,
14                    contents=batch,
15                    config=types.EmbedContentConfig(task_type="RETRIEVAL_DOCUMENT")
16                )
17                return [e.values for e in response.embeddings]
18            except (httpx.RemoteProtocolError, httpx.ReadError, httpx.ConnectError) as e:
19                delay = _RETRY_BASE_DELAY * (2 ** attempt)
20                logger.warning("Network error (attempt %d/%d): retrying in %.0fs",
21                               attempt + 1, _MAX_RETRIES, delay)
22                time.sleep(delay)
23
24        def embed_documents(self, texts: List[str]) -> List[List[float]]:
25            all_embeddings: List[List[float]] = []
26            for i in range(0, len(texts), _BATCH_SIZE):
27                batch = texts[i : i + _BATCH_SIZE]
28                all_embeddings.extend(self._embed_batch_with_retry(batch))
29            if i + _BATCH_SIZE < len(texts):
30                time.sleep(_BATCH_DELAY) # stay under quota (3000 req/min)

```

```

31     return all_embeddings
32
33     def embed_query(self, text: str) -> List[float]:
34         response = self._client.models.embed_content(
35             model=self.model,
36             contents=text,
37             config=types.EmbedContentConfig(task_type="RETRIEVAL_QUERY"),
38         )
39         return response.embeddings[0].values
40
41
42     def create_embeddings(config: Optional[Dict[str, Any]] = None) ->
43         GoogleGenAIEmbeddings:
44         """Create embeddings via Google gemini-embedding-001 API."""
45         return GoogleGenAIEmbeddings()

```

Código G.4: Clase GoogleGenAIEmbeddings para generación de vectores vía API de Google.

G.2– Descubrimiento de competidores

```

1  # Extract URLs from grounding metadata
2  grounding = getattr(response.candidates[0], "grounding_metadata", None)
3  if grounding:
4      # search_urls: all sources from grounding chunks
5      chunks = getattr(grounding, "grounding_chunks", None) or []
6      for chunk in chunks:
7          web = getattr(chunk, "web", None)
8          if web and getattr(web, "uri", None):
9              resolved = self._resolve_proxy_url(web.uri)
10             if resolved:
11                 search_urls.append(resolved)
12
13     # citation_urls: only those backing specific claims (weight=2)
14     supports = getattr(grounding, "grounding_supports", None) or []
15     cited_indices = set()
16     for support in supports:
17         indices = (
18             getattr(support, "grounding_chunk_indices", None) or []
19         )
20         for idx in indices:
21             cited_indices.add(idx)

```

Código G.5: Extracción de URLs de búsqueda y citación desde grounding_metadata.

G.3– RAG Judge

```

1  def create_search_tool(vectorstore: "FAISS", top_k: int = 5):
2      """Return a LangChain tool that searches the given FAISS vectorstore."""
3      retriever = vectorstore.as_retriever(search_kwargs={"k": top_k})
4
5      @tool
6      def search(query: str) -> str:
7          """Busca en el índice de paginas web. Devuelve fragmentos relevantes
8              con sus URLs y títulos."""

```

```

9     docs = retriever.invoke(query)
10
11     if not docs:
12         return "No se encontraron resultados relevantes."
13
14     sections = []
15     for i, doc in enumerate(docs, 1):
16         url = doc.metadata.get("source_url", "Desconocida")
17         title = doc.metadata.get("title", "")
18         header = f"[Resultado {i}] {url}"
19         if title:
20             header += f"\nTitulo: {title}"
21         header += f"\n\n{doc.page_content}"
22         sections.append(header)
23
24     return "\n\n---\n\n".join(sections)
25
26     return search

```

Código G.6: Herramienta de búsqueda FAISS encapsulada como tool de LangChain.

```

1 def generate_answer_with_agent(self, question, vectorstore, max_retries=3):
2     from langchain.agents import AgentExecutor, create_tool_calling_agent
3
4     top_k = self._config.get("retrieval", {}).get("top_k", 5)
5     search_tool = create_search_tool(vectorstore, top_k=top_k)
6
7     agent = create_tool_calling_agent(
8         llm=self.llm, # Gemini 2.5 Flash, temperature=0
9         tools=[search_tool],
10        prompt=self._agent_prompt,
11    )
12    executor = AgentExecutor(
13        agent=agent,
14        tools=[search_tool],
15        max_iterations=5,
16        verbose=True,
17    )
18
19    for attempt in range(max_retries + 1):
20        try:
21            result = executor.invoke({"input": question})
22            parsed = self._extract_json(result["output"])
23            self._validate_schema(parsed)
24            return parsed
25
26        except json.JSONDecodeError as exc:
27            logger.warning("Agent JSON parse error (attempt %d/%d): %s",
28                           attempt + 1, max_retries + 1, exc)
29            if attempt == max_retries:
30                raise

```

Código G.7: Método generate_answer_with_agent del agente RAGJudge.

```

1 @staticmethod
2 def _extract_json(text: str) -> dict:
3     """Extract JSON from agent output, handling markdown code blocks."""
4     text = text.strip()

```

```

5
6     # Try direct parse first
7     try:
8         return json.loads(text)
9     except json.JSONDecodeError:
10        pass
11
12    # Try extracting from '''json ... ''' blocks
13    match = re.search(r"'''(?:json)?\s*\n?(.*?)\n?'''", text, re.DOTALL)
14    if match:
15        return json.loads(match.group(1).strip())
16
17    # Try finding first { ... last }
18    start = text.find("{")
19    end = text.rfind("}")
20    if start != -1 and end != -1 and end > start:
21        return json.loads(text[start : end + 1])
22
23    raise json.JSONDecodeError("No JSON found in agent output", text, 0)

```

Código G.8: Extracción robusta de JSON de la salida del agente mediante tres estrategias.

G.4– Extracción de métricas GEO

```

1 def _detect_brand_mentions(self, answer, citations):
2     mentions = []
3     pattern = re.compile(re.escape(self.target_brand), re.IGNORECASE)
4
5     for match in pattern.finditer(answer):
6         start = max(0, match.start() - 50)
7         end = min(len(answer), match.end() + 50)
8         mentions.append({
9             "source": "answer",
10            "position": match.start(),
11            "context": answer[start:end],
12        })
13
14    for citation in citations:
15        quote = citation.get("quote", "")
16        for match in pattern.finditer(quote):
17            start = max(0, match.start() - 50)
18            end = min(len(quote), match.end() + 50)
19            mentions.append({
20                "source": f"citation_{citation.get('index', '?')}",
21                "position": match.start(),
22                "context": quote[start:end],
23            })
24
25    return mentions

```

Código G.9: Detección de menciones de marca en la respuesta y citas.

G.5– Evaluación en vivo sobre motores reales

```

1 def _query_claude(self, query: str) -> Dict[str, Any]:
2     response = self._anthropic_client.messages.create(
3         model=self._claude_model, # claude-haiku-4-5
4         max_tokens=1500,
5         tools=[{"type": "web_search_20250305", "name": "web_search"}],
6         messages=[{"role": "user", "content": query}],
7     )
8
9     answer = ""
10    search_urls: List[str] = []
11    for block in response.content:
12        if block.type == "text":
13            answer += block.text
14        elif block.type == "web_search_tool_result":
15            for result in block.content or []:
16                url = getattr(result, "url", None)
17                if url and url not in search_urls:
18                    search_urls.append(url)
19
20    citations = [{"index": i + 1, "url": u, "quote": ""}
21                for i, u in enumerate(search_urls)]
22    return {
23        "answer": answer,
24        "citations": citations,
25        "sources_used": search_urls,
26        "sources_available_but_unused": [],
27    }

```

Código G.10: Adaptador de Claude Haiku con búsqueda web en LiveEvaluator.

G.6— Agente de optimización GEOOptimizer

```

1 def _search_competitor_snippets(self, query_text):
2     """Busca en Google restringiendo a los competidores congelados."""
3     site_clause = " OR ".join(
4         f"site:{d}" for d in self._competitor_domains_list
5     )
6     scoped_query = f"{query_text} ({site_clause})"
7     r = requests.post(
8         "https://google.serper.dev/search",
9         headers={"X-API-KEY": api_key, "Content-Type": "application/json"},
10        json={"q": scoped_query, "num": 10, "gl": "es", "hl": "es"},
11        timeout=10,
12    )
13    results = []
14    for item in r.json().get("organic", []):
15        url = item.get("link", "")
16        if TARGET_DOMAIN in url: # descartar el propio sitio
17            continue
18        if not self._url_is_known_competitor(url): # solo competidores reales
19            continue
20        results.append({"url": url, "snippet": item.get("snippet", "")})
21        if len(results) >= 3:
22            break
23    return results

```


Código G.11: Búsqueda de fragmentos restringida a los competidores congelados.

G.7– Recolección automatizada de métricas SEO

```

1 name: Recoleccion Diaria Metricas SEO
2
3 on:
4   schedule:
5     - cron: '0 8 * * *'
6   workflow_dispatch:
7
8 jobs:
9   audit_seo:
10    runs-on: ubuntu-latest
11    permissions:
12      contents: write
13
14    steps:
15      - name: Checkout del repositorio
16        uses: actions/checkout@v4
17
18      - name: Configurar Python
19        uses: actions/setup-python@v4
20        with:
21          python-version: '3.9'
22
23      - name: Instalar dependencias
24        run: pip install requests
25
26      - name: Ejecutar Script de Auditoria
27        env:
28          PAGESPEED_API_KEY: ${ secrets.PAGESPEED_API_KEY }
29        run: python collect_metrics/collect_seo.py
30
31      - name: Guardar cambios (Commit & Push)
32        run: |
33          git config --global user.name 'GitHub Action Bot'
34          git config --global user.email 'action@github.com'
35          git add data/seo/*.json
36          git diff --quiet && git diff --staged --quiet || \
37            (git commit -m "Auto: Nuevas metricas SEO capturadas" && git push)

```

Código G.12: Workflow de GitHub Actions para la recolección diaria de métricas SEO.

G.8– Configuración y prompts

```

1 {
2   "version": "1.1.0",
3   "target_url": "https://programamos.es",
4   "target_brand": "Programamos",
5
6   "discovery": {
7     "model": "gemini-2.5-flash",
8     "provider": "google",

```

```

9     "grounding": "google_search"
10 },
11 "rag_simulator": {
12     "model": "gemini-2.5-flash",
13     "temperature": 0.0,
14     "max_tokens": 2000,
15     "response_format": "json_object",
16     "mode": "agent"
17 },
18 "embeddings": {
19     "model": "models/gemini-embedding-001",
20     "dimensions": 3072,
21     "provider": "google"
22 },
23 "chunking": {
24     "default_chunk_size_tokens": 256,
25     "default_overlap_tokens": 64,
26     "separators": ["\n## ", "\n### ", "\n\n", "\n", ". ", " "]
27 },
28 "retrieval": {
29     "top_k": 5,
30     "search_type": "similarity"
31 },
32 "crawler": {
33     "request_delay": 1.0,
34     "timeout": 10,
35     "max_retries": 3,
36     "respect_robots_txt": true,
37     "target": { "enabled": true, "max_pages": null, "priority": "pages_first" },
38     "competitor": { "enabled": true, "max_pages": 100, "priority": "discovered_urls_first" }
39 }
40 }

```

Código G.13: Fichero de configuración del experimento experiment_config.json.

```

1 def get_queries_for_run(block: Optional[str] = None) -> list[str]:
2     """Devuelve las queries para un run: core 20 + bloque rotativo seleccionado."""
3     data = load_queries()
4
5     if "rotation" not in data or "queries" not in data:
6         return get_all_queries() # v1 fallback
7
8     rotation = data["rotation"]
9     queries_db = data["queries"]
10
11     # Core queries (siempre)
12     core_ids = rotation["core"]
13     result = [queries_db[qid]["text"] for qid in core_ids]
14
15     # Bloque rotativo (opcional)
16     if block is not None:
17         if block not in rotation:
18             valid = [k for k in rotation if k != "core"]
19             raise ValueError(f"Bloque '{block}' no valido. Disponibles: {valid}")
20             block_ids = rotation[block]
21             result.extend([queries_db[qid]["text"] for qid in block_ids])
22

```

```

23     return result
24
25
26 def get_discovery_queries() -> list[str]:
27     """Solo queries informacionales + comparativas (sin navegacionales)."""
28     data = load_queries()
29
30     if "queries" in data and isinstance(data["queries"], dict):
31         return [
32             q["text"] for q in data["queries"].values()
33             if q["category"] in ("informacional", "comparativa")
34         ]
35
36     # v1 format
37     categories = data["categories"]
38     return categories.get("informacional", []) + categories.get("comparativa",
39         [])

```

Código G.14: Función get_queries_for_run con soporte de rotación.

```

1  PROMPT_REGISTRY = {
2      "rag_judge_agent": {
3          "version": "1.0.0",
4          "description": "Prompt para el RAG Judge en modo agente con herramienta de
5              búsqueda",
6          "system": (
7              "Eres un motor de búsqueda generativo (similar a Perplexity AI). "
8              "Tienes acceso a una herramienta de búsqueda web. Cuando recibes una "
9              "pregunta, DEBES buscar informacion relevante usando la herramienta "
10             "antes de responder.\n\n"
11             "PROCESO:\n"
12             "1. Analiza la pregunta del usuario.\n"
13             "2. Usa la herramienta 'search' para buscar informacion relevante. "
14             "Puedes hacer multiples busquedas si necesitas mas informacion.\n"
15             "3. Sintetiza la informacion encontrada en una respuesta coherente.\n\n"
16             "FORMATO DE RESPUESTA:\n"
17             "Una vez tengas suficiente informacion, responde EXCLUSIVAMENTE "
18             "en formato JSON:\n"
19             "# ... schema JSON ..."
20             "REGLAS DE CITACION:\n"
21             "1. Cada afirmacion factual DEBE tener una cita numerada [N].\n"
22             "2. El campo 'quote' debe contener el texto EXACTO de la fuente.\n"
23             "# ..."
24         ),
25         "model": "gemini-2.5-flash",
26         "temperature": 0.0,
27         "max_tokens": 2000,
28         "changelog": [
29             "1.0.0: Prompt del RAG Judge agente con herramienta de búsqueda (ADR
30                 -012)",
31         ],
32     },
33     # ... otros prompts: rag_judge, sentiment_analyzer
34 }

```

Código G.15: Registro centralizado de prompts con versionado.

G.9— Scripts de automatización

```

1 def main():
2     config = load_experiment_config()
3
4     # 1. Discovery: queries -> Gemini with grounding -> URLs
5     queries = get_discovery_queries()
6     finder = CompetitorFinder(config)
7     results = finder.discover_competitors(queries)
8     finder.save_results(results, str(discovery_path))
9
10    # 2. Select top competitors
11    top_competitors = results["aggregated_domains"][[:args.top]]
12
13    # 3. Crawl target + competitors
14    crawler = SiteCrawler(config)
15    target_docs = crawler.crawl_site(target_url, is_target=True)
16    for comp in top_competitors:
17        docs = crawler.crawl_site(f"https://{comp['domain']}", is_target=False,
18                                discovered_urls=comp.get("urls", []))
19        competitor_docs.extend(docs)
20
21    # 4. Chunk competitor documents
22    chunker = TokenAwareChunker(config)
23    competitor_chunks = chunker.chunk_documents(competitor_docs)
24
25    # 5. Embed + build frozen vectorstore (competitors only)
26    embeddings = create_embeddings(config)
27    frozen_vs = FAISS.from_documents(competitor_chunks, embeddings)
28    frozen_vs.save_local(str(vs_dir))
29
30    # 6. Summary

```

Código G.16: Orquestación del pipeline de descubrimiento en run_discovery.py.

```

1 def main():
2     # 1-2. Load frozen vectorstore
3     frozen_vs = FAISS.load_local(str(vs_dir), embeddings,
4                                allow_dangerous_deserialization=True)
5
6     # 3-4. Crawl + chunk + embed target
7     target_docs = crawler.crawl_site(target_url, is_target=True)
8     target_chunks = chunker.chunk_documents(target_docs)
9     target_vs = FAISS.from_documents(target_chunks, embeddings)
10
11    # Merge: target + frozen competitors
12    frozen_vs.merge_from(target_vs)
13    merged_vs = frozen_vs
14
15    # 5. RAG Judge + Metrics
16    judge = RAGJudge(config)
17    extractor = CitationExtractor(target_url, target_brand)
18
19    for query in queries:
20        answer = judge.generate_answer_with_agent(query, merged_vs)
21        metrics = extractor.extract_metrics(answer)
22
23    # 6. Save scorecard

```

```
24     scorecard = {
25         "run_id": run_id,
26         "visibility_rate": round(n_visible / len(successful) * 100, 2),
27         "avg_som": round(sum(m["som"] for m in successful) / len(successful), 2),
28         "per_query_metrics": metrics_list,
29     }
```

Código G.17: Orquestación del pipeline experimental en `run_experimental.py`.

Referencias

- Advanced Web Ranking. (2024). *Google organic CTR research*. Descargado de <https://www.advancedwebranking.com/ctrstudy/> (Consultado en marzo de 2026)
- Aggarwal, P., Murahari, V., Rajpurohit, T., Kalyan, A., Narasimhan, K., y Deshpande, A. (2024). GEO: Generative engine optimization. En *Proceedings of the 30th acm sigkdd conference on knowledge discovery and data mining*. New York, NY: ACM. (arXiv preprint arXiv:2311.09735 (2023)) doi: 10.1145/3637528.3671900
- Anthropic. (2025). *Claude API documentation: Tool use and web search*. Descargado de <https://docs.anthropic.com/> (Consultado en abril de 2026)
- Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., y Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. *arXiv preprint arXiv:2401.05856*. Descargado de <https://arxiv.org/abs/2401.05856>
- BrightEdge Research. (2024). *Generative AI and SEO: How search engines process content chunks*. Descargado de <https://www.brightedge.com> (Consultado en marzo de 2026)
- Broder, A. (2002). A taxonomy of web search. *ACM SIGIR Forum*, 36(2), 3–10.
- Chitika Inc. (2013). *The value of Google result positioning*. Descargado de <https://chitika.com/google-positioning-value> (Consultado en marzo de 2026)
- Fishkin, R. (2023). *Sparktoro founder's blog: SEO in the age of generative AI*. Descargado de <https://sparktoro.com> (Consultado en marzo de 2026)
- Google LLC. (2025). *Gemini API documentation: Grounding with Google search*. Descargado de <https://ai.google.dev/gemini-api/docs/grounding> (Consultado en abril de 2026)
- Google Search Central. (2025). *Structured data: Introduction and best practices*. Descargado de <https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data> (Consultado en abril de 2026)
- Kim, H., Jeong, S., Kim, H., Lee, J., y Lee, M. (2026). SAGEO arena: A realistic environment for evaluating search-augmented generative engine optimization. *arXiv preprint arXiv:2602.12187*. Descargado de <https://arxiv.org/abs/2602.12187>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. En *Advances in neural information processing systems* (Vol. 33, pp. 9459–9474). Curran Associates, Inc.
- Lüttgenau, J., Colic, L., y Ramirez, D. (2025). Beyond SEO: A transformer-based approach for reinventing web content optimisation. *arXiv preprint arXiv:2507.03169*. Descargado de <https://arxiv.org/abs/2507.03169>
- Metzler, D., Tay, Y., Bahri, D., y Najork, M. (2021). Rethinking search: Making domain experts out of dilettantes. En *Sigir forum* (Vol. 55). ACM.
- Moz. (2024). *Search engine ranking factors*. Descargado de <https://moz.com/search-ranking-factors> (Consultado en marzo de 2026)
- OpenAI. (2024). *ChatGPT: Web browsing and search capabilities*. Descargado de <https://openai.com/chatgpt> (Consultado en abril de 2026)

- Pan, B., Hembrooke, H., Joachims, T., Lorigo, L., Gay, G., y Granka, L. (2007). In Google we trust: Users' decisions on rank, position, and relevance. *Journal of Computer-Mediated Communication*, 12(3), 801–823.
- Tan, J., Dou, Z., Wang, W., Wang, M., Chen, W., y Wen, J.-R. (2025). HtmlRAG: HTML is better than plain text for modeling retrieved knowledge in RAG systems. En *Proceedings of the acm web conference 2025 (WWW 2025)*. ACM. Descargado de <https://arxiv.org/abs/2411.02959> doi: 10.1145/3696410.3714546